



Seminario de Práctica de Analista en Licenciatura en Informática.

**Sistema de Gestión Integral para RRHH, Ministerio de Gobierno, Infraestructura y
Desarrollo Territorial del Gobierno de Mendoza**

Alumno: Rodrigo A. Quinteros

Legajo: VINF09630

DNI: 27.949.990

Fecha: 28/06/2026

Docente/Tutor: Virgolini, Pablo Alejandro

Materia: INF275 – Seminario Práctica de Informática

Versión: 4 (Cuarta Entrega)

Contenido

Raquinteros/rrhh-ministerio: Sistema de Gestión Integral para RRHH	6
1 - INTRODUCCION	7
1.1 - ANTECEDENTES Y DESCRIPCION DE LA PROBLEMÁTICA	7
1.2 - FORMULACION DE LA PROBLEMÁTICA	7
2 - JUSTIFICACION	8
3 - DEFICIONES DEL PROYECTO	8
3.1 - OBJETIVOS GENERALES DEL PROYECTO	8
3.2 – DIAGRAMA GANTT CRONOGRAMA DEL PROYECTO	8
4 - DEFINICIONES DEL SISTEMA	9
4.1 - OBJETIVO GENERAL DEL SISTEMA	9
4.2 - LIMITES DEL SISTEMA	9
4.3 - ALCANCE	9
4.4 - NO CONTEMPLA	10
5 - ELICITACION	10
5.1 - TÉCNICAS UTILIZADAS	10
5.2 - PASOS SEGUIDOS	11
5.3 - RESULTADO DE LA ELICITACIÓN	12
5.4 - DOMINIO DEL PROBLEMA	12
5.5 - TIC (TECNOLOGÍA DE LA INFORMACIÓN Y COMUNICACIÓN)	12
5.6 - COMPETENCIA	13
6 – CONOCIMIENTO DEL NEGOCIO	14
6.1 – RELEVAMIENTO ESTRUCTURAL	14
6.2 – RELEVAMIENTO FUNCIONAL	14
6.3 - PROCESOS DEL NEGOCIO	15
6.4 – DIAGNOSTICO	20
6.5– DIAGRAMA DE DOMINIO	21
7 – PROPUESTA DE SOLUCION	22

7.1 -PROPUESTA FUNCIONAL.....	22
7.2 – PROPUESTA TECNICA	23
7.3 – LISTADO DE REQUERIMIENTOS FUNCIONALES	23
7.4 - LISTADO DE REQUERIMIENTOS NO FUNCIONALES	24
7.5 – LISTADO DE REQUERIMIENTOS CANDIDATOS	24
8 – INICIO DEL ANALISIS: CASOS DE USO.....	25
8.1 - DIAGRAMA DE CASOS DE USO	25
8.2 – DESCRIPCION DE CASOS DE USO.....	26
9 – ETAPA DE ANALISIS	30
9.1 - ALCANCE DEL PROTOTIPO	30
9.2 – DIAGRAMA DE SECUENCIA	31
10 – ETAPA DE DISEÑO	32
10.1 – CLASES PRINCIPALES DEL DISEÑO	32
10.2 – DIAGRAMA DE CLASES DE DISEÑO.....	33
11 – ETAPA DE IMPLEMENTACION	34
11.1 - ALCANCE IMPLEMENTADO	34
11.2 - DEFINICIONES TÉCNICAS DE IMPLEMENTACIÓN	34
11.3 - REQUERIMIENTOS NO FUNCIONALES DEL PROTOTIPO	35
11.4 – DIAGRAMA DE DESPLIEGUE	36
12 - ETAPA DE PRUEBAS	36
12.1 – PLAN DE PRUEBAS	37
12.2 - MODELO DE PRUEBAS	38
12.3 - CASO DE PRUEBA CPF02: GESTIONAR AGENTE.....	38
12.4 - CASO DE PRUEBA CPF04: REGISTRAR LICENCIA	39
12.5 - CASO DE PRUEBA CPV05: VALIDACIÓN DE FECHAS EN LICENCIAS	40
12.6 - PARTICIÓN DE EQUIVALENCIA	40
12.6.1 - PARTICIÓN DE EQUIVALENCIA PARA CU02 - GESTIONAR AGENTE.....	40

12.6.2 - PARTICIÓN DE EQUIVALENCIA PARA CU04 - GESTIONAR LICENCIAS	41
12.7 - ANÁLISIS DE VALORES DE FRONTERA	42
12.7.1 - ANÁLISIS DE VALORES DE FRONTERA PARA CU02 - GESTIONAR AGENTE	42
12.7.2 - ANÁLISIS DE VALORES DE FRONTERA PARA CU04 - GESTIONAR LICENCIAS	42
12.8 - COMPORTAMIENTO ESPERADO DEL SISTEMA	42
12.8.1 - COMPORTAMIENTO ESPERADO PARA CU02 - GESTIONAR AGENTE	43
12.8.2 - COMPORTAMIENTO ESPERADO PARA CU04 - GESTIONAR LICENCIAS	43
13 – INTERFAZ GRAFICA	43
13.1 – RELACIONES: INTERFAZ GRÁFICA, CASOS DE USO Y PRUEBAS	48
14 - DEFINICIÓN DE BASE DE DATOS PARA EL SISTEMA	48
14.1 - DIAGRAMA ENTIDAD-RELACIÓN	49
14.2 - CREACIÓN DE TABLAS MYSQL	50
14.3 - INSERCIÓN, CONSULTA Y BORRADO DE REGISTROS	52
14.3.1 – INSERCIÓN DE DATOS DE PRUEBA	52
14.3.2 - CONSULTA DE DATOS DE PRUEBA	54
14.3.3 – BORRADO DE REGISTROS	55
14.4 - PRESENTACIÓN DE CONSULTAS SQL	56
15 - DEFINICIONES DE COMUNICACIÓN	58
15.1 - COMPONENTES DE COMUNICACIÓN	58
15.2 – DIAGRAMA DE COMUNICACIÓN	58
16 – EXPLICACIÓN DEL DESARROLLO EN JAVA	59
16.1 – ORGANIZACIÓN GENERAL DEL PROYECTO	60
16.2 – CLASES DEL PROYECTO	61
16.2.1 – CLASE MAIN	61
16.2.2 – CLASE PERSONA	63
16.2.3 – CLASE AGENTE	65
16.2.4 – CLASE REPOSITORIOMEMORIA	67

16.2.5 – CLASE LOGINVIEW	69
16.2.6 – CLASE MENUPRINCIPALVIEW	71
16.2.7 – CLASE AGENTEVUE	73
16.2.8 – CLASE LICENCIAVIEW	75
16.3 - CONCEPTOS DE PROGRAMACIÓN ORIENTADA A OBJETOS APLICADOS	77
17 – DEMOSTRACIÓN DE COMPILACIÓN Y EJECUCIÓN.....	78
17.1 – CAPTURAS DE LA APLICACIÓN CORRIENDO	78
18 - PERSISTENCIA DE DATOS MEDIANTE JDBC Y MARIADB	81
18.1 - ESTRATEGIA DE PERSISTENCIA Y ARQUITECTURA	82
18.2 - CENTRALIZACIÓN DEL ACCESO A DATOS	83
18.3 - GESTIÓN DINÁMICA DE LA INFORMACIÓN MEDIANTE ARRAYLIST	83
18.4 - CONSULTAS RELACIONALES E INTEGRACIÓN CON LA INTERFAZ	84
19 - JUSTIFICACIÓN DEL PATRÓN DE DISEÑO IMPLEMENTADO	86
19.1 - ORGANIZACIÓN DEL PROYECTO BASADA EN MVC.....	86
19.2 - RELACIÓN ENTRE LOS COMPONENTES	88
19.3 - BENEFICIOS OBTENIDOS	89
20 - IMPLEMENTACIÓN DE ESTRUCTURAS DE DATOS DINÁMICAS (ARRAYLIST)	89
20.1 - UTILIZACIÓN DE ARRAYLIST EN EL PROYECTO.....	90
20.2 - RECUPERACIÓN DINÁMICA DE AGENTES.....	90
20.3 - RECUPERACIÓN DINÁMICA DE TIPOS DE LICENCIA.....	90
20.4 - RECUPERACIÓN DINÁMICA DE LICENCIAS	91
20.5 - CONCLUSIÓN	91
21 - APLICACIÓN DE POO: ABSTRACCIÓN, HERENCIA Y POLIMORFISMO	92
21.1 - ABSTRACCIÓN MEDIANTE LA CLASE PERSONA	92
21.2 - HERENCIA MEDIANTE LA CLASE AGENTE	92
21.3 - POLIMORFISMO MEDIANTE LA SOBRESCRITURA DE MÉTODOS	93
21.4 - ENCAPSULAMIENTO DE LA INFORMACIÓN.....	93

21.5 - BENEFICIOS OBTENIDOS	94
22 - MANEJO DE EXCEPCIONES Y VALIDACIONES DE ROBUSTEZ	94
22.1 - MANEJO DE EXCEPCIONES EN LA CAPA DE PERSISTENCIA	95
22.2 - VALIDACIÓN DEL FORMATO DE FECHAS.....	95
22.3 - VALIDACIONES PREVENTIVAS DE INGRESO DE DATOS	96
22.4 - COMUNICACIÓN DE ERRORES AL USUARIO	97
22.5 - BENEFICIOS OBTENIDOS	97
23 - JUSTIFICACIÓN DE LA NO UTILIZACIÓN DE ARCHIVOS PLANOS	98
23.1 - PERSISTENCIA IMPLEMENTADA EN EL PROYECTO	98
23.2 - RAZONES PARA NO UTILIZAR ARCHIVOS PLANOS.....	98
23.3 - CONCLUSIÓN	99
24 - CONCLUSIONES	99

TODO EL TRABAJO Y SUS COMPONENTES SERAN SUBIDOS A GIT EN EL SIGUIENTE ENLACE:

[Raquinteros/rrhh-ministerio: Sistema de Gestión Integral para RRHH](https://github.com/Raquinteros/rrhh-ministerio)

LINK VIDEO PRESENTACION PROYECTO:

https://youtu.be/YYQ_ejXOsao

1 - INTRODUCCION

Las tareas de RRHH llevadas a cabo por los agentes del estado, es complicada, ya que tiene que oficiar de interlocutor entre el Gobierno y los empleados, y además interpretar lo mejor posible los problemas y cuestionamientos de ambas partes.

Y estas tareas son muy diversas, en el mejor de los casos cuentan con un sistema que lleva el control de alguna de ellas, pero en contraste también encontramos que otras son completamente analógicas llevando un formulario papel, con matices en el medio como planillas de cálculos.

Este arcoíris de tareas y con diversas maneras de “llevarlo al día” provoca una complicación, y más teniendo en cuenta cuando hay que cruzar información entre todas estas

Y puesto que un error material en algún dato, provocado por ej. una mala interpretación de un numero escrito en una tarjeta de cartón, puede provocar grandes inconvenientes sobre todo para el agente al cual se esté refiriendo la gestión.

Con este sistema se busca organizar de manera factible y rápida la información y el estado actual de los agentes a cargo del Ministerio y sus dependencias.

1.1 - ANTECEDENTES Y DESCRIPCION DE LA PROBLEMÁTICA

En cuanto a cómo se realizaba estas importantes tareas, históricamente era en papel. Luego se implementaron planillas de cálculos para algunas, otras siguen siendo en papel, para luego pasar algunas de estas tareas al uso de sistemas pequeños con una sola función cada uno.

Actualmente, algunas de las tareas se realizan en una planilla de cálculo, pero este método solamente sirve como ordenamiento visual y para mostrar el resultado final, el contenido como se lleva a cabo el proceso es meramente mental con la latente posibilidad de cometer errores de carga o de cruce de información, a pesar de la buena predisposición de las personas encargada de tales tareas.

Estas deficiencias en cuanto a la disparidad y calidad de los soportes usados para tales fines, provoca demoras en trámites que en su principio no deberían de tener.

Y además contamos con el grave problema de seguridad de estos soportes, ya que cualquier descuido de parte del personal de la oficina, puede provocar que otras personas accedan a información confidencial de los empleados, con el motivo que fuere

1.2 - FORMULACION DE LA PROBLEMÁTICA

Expresado todo lo anterior, ¿Será realmente posible desarrollar un sistema integral, que, teniendo un único identificador por empleado, pueda gestionar diversas tareas y centralizar todas en una sola interfaz?

2 - JUSTIFICACION

La necesidad de implementar un sistema que realice estas distintas tareas, pero a la vez entrelazadas, radican principalmente en mejorar el tiempo de respuesta y la calidad de la misma. Ya que se buscará con este proyecto satisfacer varias necesidades claves como, la velocidad de respuesta a consultas tanto de niveles superiores como de los agentes estatales sobre su situación, cruce de información, mejorar el catálogo de información de los empleados del ministerio, mejorar el control de licencias, faltas, etc.

El impacto de implementación de este sistema es trascendental en la administración pública, ya que ninguna repartición cuenta con algo parecido, por lo que posicionaría a la oficina de personal del Ministerio de Gobierno, Infraestructura y Desarrollo Territorial a la vanguardia en lo referido al manejo y guarda de la información de los agentes que se desempeñan bajo su órbita.

Pudiéndose tomar como ejemplo para su aplicación en otras reparticiones, por lo tanto la relevancia que podría tener es muy alta para todo el Poder Ejecutivo, ya que a pesar que el mismo cuenta con sistemas CORE de gran envergadura, como seria el sistema de liquidación de sueldos de los empleados públicos de la provincia de Mendoza, pero este no contiene información básica de los empleados, como la que aspira este desarrollo contener.

3 - DEFICIONES DEL PROYECTO

3.1 - OBJETIVOS GENERALES DEL PROYECTO

Diseñar y desarrollar un sistema informático de gestión integral para la Oficina de Personal del Ministerio de Gobierno, Infraestructura y Desarrollo Territorial, orientado a centralizar, organizar y consultar la información administrativa de los agentes, optimizando la gestión de legajos, licencias, inasistencias, certificaciones y documentación asociada.

3.2 – DIAGRAMA GANTT CRONOGRAMA DEL PROYECTO

Meses	MARZO				ABRIL				MAYO				JUNIO			
Semanas	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4
Introducción																
Justificación																
Definiciones del proyecto																
Definiciones del sistema																
Elicitación																
Conocimiento del Negocio																
Propuesta de solución																
Casos de Uso																
Etapas de Analisis																
Etapas de Diseño																
Etapas de Implementacion																
Etapas de pruebas																
Interfaz Grafica																
Definiciones de BD																
Definiciones de Comun.																
Explicacion Desarrollo																
Demostracion Compilacion																
Persistencia MySQL																
Patron de Diseño (MVC)																
ArrayList																
Demostracion uso de POO																
Uso de Excepciones y Validacion																
Conclusiones																

El diagrama de Gantt esta codificado para la 1ª Entrega en color verde, 2ª Naranja, 3ª Celeste, y amarillo para el 4ª

3.3 - OBJETIVOS ESPECIFICOS DEL PROYECTO

A fin de lograr una concreción eficiente del objetivo general del proyecto se seguirán los pasos detallados a continuación:

- Centralizar la información administrativa de los agentes en un repositorio único y estructurado.
- Reducir la dependencia de planillas de cálculo, archivos físicos y sistemas aislados.
- Mejorar la trazabilidad de los legajos, licencias, faltas, certificaciones y documentación asociada.
- Disminuir los errores de carga, pérdida de información y duplicidad de datos.
- Facilitar la generación de informes administrativos para autoridades y personal autorizado.
- Proteger la información sensible de los agentes mediante mecanismos de autenticación, control de acceso y auditoría.
- Permitir una futura integración con sistemas externos del Gobierno de Mendoza, sin que dicha integración forme parte del alcance inicial.

4 - DEFINICIONES DEL SISTEMA

4.1 - OBJETIVO GENERAL DEL SISTEMA

Gestionar de manera centralizada la información administrativa de los agentes del Ministerio, permitiendo registrar, consultar, actualizar y controlar legajos, licencias, inasistencias, certificaciones y documentación asociada, con el fin de mejorar la eficiencia operativa de la Oficina de Personal.

4.2 - LIMITES DEL SISTEMA

El sistema inicia cuando un agente es incorporado administrativamente a la órbita del Ministerio y se registra su legajo digital inicial. A partir de ese momento, permite gestionar los datos administrativos vinculados a su situación de revista, documentación, licencias, faltas, certificaciones y novedades relevantes.

El sistema finaliza su intervención cuando el agente es dado de baja definitivamente por jubilación, renuncia, traslado, fallecimiento u otra causal administrativa, conservando el historial de información para futuras consultas autorizadas.

El sistema no reemplaza los sistemas provinciales centrales ni ejecuta actos administrativos formales. Su función es registrar, organizar, consultar y respaldar información utilizada por la Oficina de Personal para el seguimiento administrativo de los agentes.

4.3 - ALCANCE

El sistema contempla:

- Alta, baja lógica, modificación y consulta de agentes.
- Registro y actualización de legajos digitales.
- Carga y consulta de licencias.
- Registro de faltas e inasistencias.
- Gestión de certificaciones administrativas.
- Adjuntado de documentación digital asociada al agente.
- Generación de informes para autoridades o personal autorizado.
- Registro de auditoría sobre acciones realizadas por usuarios.
- Consulta centralizada de información administrativa del agente.

4.4 - NO CONTEMPLA

El sistema no contempla:

- Liquidación de haberes.
- Gestión integral de expedientes electrónicos.
- Firma digital de actos administrativos.
- Resolución de sumarios administrativos.
- Integración automática con sistemas CORE provinciales en esta etapa.
- Portal de autogestión para agentes en la primera versión.
- Notificaciones automáticas externas en la primera versión.
- Reemplazo de los sistemas oficiales provinciales ya existentes.
- Ejecución automática de actos administrativos que requieran intervención formal de autoridad competente.

5 - ELICITACION

Las técnicas aplicadas fueron entrevistas, observación directa y revisión documental. La combinación de estas técnicas permitió obtener información cualitativa sobre los procesos administrativos y, a la vez, identificar datos concretos utilizados en la registración y consulta de agentes, licencias, faltas, certificaciones y documentación asociada.

5.1 - TÉCNICAS UTILIZADAS

1. Entrevistas

Se realizaron entrevistas con personal de la Oficina de Personal, con el objetivo de identificar las tareas principales, la documentación utilizada, los problemas habituales, los circuitos de consulta y las necesidades de información más frecuentes.

Durante las entrevistas se relevaron los siguientes aspectos:

- tareas administrativas realizadas por el personal de la oficina;
- datos requeridos para registrar y consultar agentes;
- documentación utilizada para legajos, licencias, faltas y certificaciones;
- problemas frecuentes derivados del uso de planillas, archivos físicos y sistemas aislados;
- necesidades de consulta e informes por parte de autoridades y personal autorizado;
- restricciones relacionadas con la confidencialidad de la información.

2. Observación directa

Se realizó observación directa de las tareas habituales de la Oficina de Personal, con el fin de identificar cómo se registran, consultan y resguardan actualmente los datos administrativos de los agentes.

Esta técnica permitió detectar el uso simultáneo de distintos soportes de información, como documentación en papel, planillas de cálculo y sistemas específicos no integrados. También permitió observar demoras, duplicación de carga, riesgo de errores y dificultades para obtener información consolidada.

3. Revisión documental

Se revisaron formularios, planillas, documentación administrativa y registros utilizados por la Oficina de Personal. Esta revisión permitió identificar los datos mínimos necesarios para la construcción del modelo de información del sistema.

Entre los documentos considerados se encuentran:

- registros de agentes;
- documentación de legajos;
- planillas de licencias;
- registros de faltas e inasistencias;
- certificados administrativos;
- documentación respaldatoria asociada a trámites del agente.

5.2 - PASOS SEGUIDOS

El proceso de elicitación se desarrolló mediante los siguientes pasos:

1. Identificación de los actores involucrados en la gestión administrativa de personal.
2. Relevamiento de las tareas principales realizadas por la Oficina de Personal.
3. Análisis de los soportes actualmente utilizados: papel, planillas de cálculo y sistemas aislados.
4. Identificación de problemas frecuentes en la carga, consulta y resguardo de información.

5. Determinación de necesidades funcionales del sistema.
6. Identificación de restricciones de seguridad, confidencialidad y acceso a la información.
7. Organización de la información relevada para definir requerimientos, casos de uso y entidades principales del sistema.

5.3 - RESULTADO DE LA ELICITACIÓN

Como resultado del proceso de elicitación, se identificó la necesidad de desarrollar un sistema que permita centralizar la información administrativa de los agentes y reducir la dependencia de soportes dispersos.

Los principales problemas detectados fueron:

- duplicidad de información;
- riesgo de errores de carga;
- dificultad para cruzar datos entre legajos, licencias, faltas y certificaciones;
- demora en la generación de informes;
- dependencia de archivos físicos y planillas de cálculo;
- falta de trazabilidad sobre las acciones realizadas;
- riesgo de acceso no autorizado a información sensible.

5.4 - DOMINIO DEL PROBLEMA

El dominio comprende la **Gestión de Recursos Humanos en el Sector Público**. Se rige por leyes administrativas específicas (como el Estatuto del Empleado Público) que definen derechos, deberes y procedimientos estandarizados que el sistema debe automatizar para evitar el error humano de las planillas actuales.

A continuación, se enumeran las normativas vigentes:

- **Ley 5126/1986** - Escalafón de empleados públicos. Regula régimen escalafonario.
- **Ley 8993** – Ley de Ética pública. Establece deberes, prohibiciones e incompatibilidades de funcionarios y empleados públicos y compromiso ético.
- **Ley 9003** – Ley de Procedimiento Administrativo de Mendoza. Regula las relaciones entre la administración pública y los empleados.
- **Ley 9324/2021** – Actualiza Ley 5811 de régimen de licencias y Remuneraciones
- **Ley 9665** – Estatuto del Empleado Público. Derechos y obligaciones de los empleados públicos.

5.5 - TIC (TECNOLOGÍA DE LA INFORMACIÓN Y COMUNICACIÓN)

Las tecnologías consideradas a ser usadas en el desarrollo del proyecto son las siguientes:

Lenguaje de Programación: Se usará **JAVA** que tiene las siguientes características:

- Lenguaje orientado a objetos.
- Open Source (para no incurrir en gastos de licencias).
- Multiplataforma (para ser usado tanto en entornos Windows o Linux).
- Compatibilidad con base de datos más usadas.
- Disponibilidad de bibliotecas y repositorios.
- Soporte por comunidades de programadores (para solventar ayuda cuando se requiera).
- Compatible con otros lenguajes modernos para complementarse.

Base de datos relacionales: MySQL será el motor de base de datos, tiene los siguientes requisitos

- Open Source (para no incurrir en gastos de licencias).
- Multiplataforma, se buscará que sea compatible y de fácil instalación para asegurar mejor despliegue tanto en Linux como Windows.
- Soporte por comunidades de desarrolladores amplia.
- Compatibilidad con lenguajes de programación más extendidos.

5.6 - COMPETENCIA

Para explicar mejor la competencia del sistema a desarrollar con los otros métodos de trabajo, se presentan en forma de tabla para una mejor comparación.

Solución	Planillas de Cálculo (Actual)	Sistemas CORE	Meta 4	Sistema Propuesto RRHH
Centralización Datos	No (archivos dispares)	Limitada	Completa	Completa
Cruce de datos	Manual	Incompleto	Automático	Automático
Interfaz	Celdas	Compleja	Compleja	Amigable/Adaptable
Adaptabilidad	Posible - Complicado	Posible - Lento	Nula / Software propietario	Completa
Hardware Necesario	Pc local	Datacenter Provincial	Servidor/es	Pc Local/ Contenedor
Costo	Hay Software Open Source	Alto Mantenimiento	Muy alto licenciamiento	Desarrollado como Open Source

6 – CONOCIMIENTO DEL NEGOCIO

La Oficina de Personal del Ministerio funciona como el núcleo de gestión del capital humano, siendo responsable de la trazabilidad administrativa de cada agente estatal. A diferencia de una empresa comercial en la cual el "producto" no es un artículo, sino el **Legajo Único del Agente**, el cual debe reflejar con exactitud jurídica y administrativa en su situación de revista, asistencia y cumplimiento de normas. El proceso de negocio se rige por varias normas legales, lo que convierte a la oficina en un ente validador de derechos y obligaciones.

Esta oficina actúa como un receptor de eventos externos por parte de los agentes del gobierno, que deben ser procesados y almacenados; y con estos un emisor de eventos que puede repercutir en la situación del empleado.

6.1 – RELEVAMIENTO ESTRUCTURAL

- La oficina de personal del Ministerio de Gobierno, Infraestructura y Desarrollo Territorial, se encuentra en el **3º piso de la Casa de Gobierno de Mendoza**, emplazado en la calle Peltier 351 de la Capital de Mendoza.
- El hardware con el que se cuenta para el proyecto, son equipos de la línea Intel **Core i5 – 12ªG, 8 Gb RAM DDR4, SSD 120 Gb**. Equipamiento más que suficiente para correr el sistema a desarrollar.
- Con respecto a sistemas heredados, contamos con un **sistema realizado en VB6**, con una base de datos armada en un archivo delimitado con comas, que lleva el control de las faltas de los agentes.
- Una carpeta en la que se encuentran **escaneados algunos legajos** viejos, en formato **PDF**.
- Varias **planillas de cálculo** en donde se lleva el control de licencias, altas y bajas de agentes.
- El control de las licencias se lleva en unas **tarjetas de cartón**, a razón de una por agente (o el número que necesite)

6.2 – RELEVAMIENTO FUNCIONAL



Imagen 1 - Organigrama del Ministerio del Gobierno, se ha dejado las áreas más importantes para no desvirtuarlo y se mantienen las implicadas directamente con la oficina destinataria del presente proyecto.

- **Funciones de las Areas:**

- Ministerio: Máxima repartición (por debajo del Gobernador), es quien decide el ingreso o salida de agentes públicos dentro de las jurisdicciones del ministerio.
- Subsecretarias: reparticiones especialidades en distintas incumbencias del ministerio (suelen tener áreas de personal propias)
- Dirección General de Administración y despacho: Repartición a cargo de la oficina de personal, es quien interacciona directamente con ella, toma las decisiones finales en las cuestiones referidas a los agentes.
- Subdirección de Personal: es el destinatario de este proyecto.

6.3 - PROCESOS DEL NEGOCIO

Nombre de proceso: **Alta/Baja empleado**

Rol: Agente Administrativo de Personal

Ejecución:

1. Si es alta, crea un legajo nuevo / Si es baja, se va al paso 2 directamente
2. Incorpora la información al legajo papel
3. Se guarda legajo en archivador
4. “Crea” las entradas necesarias en todos “los sistemas”

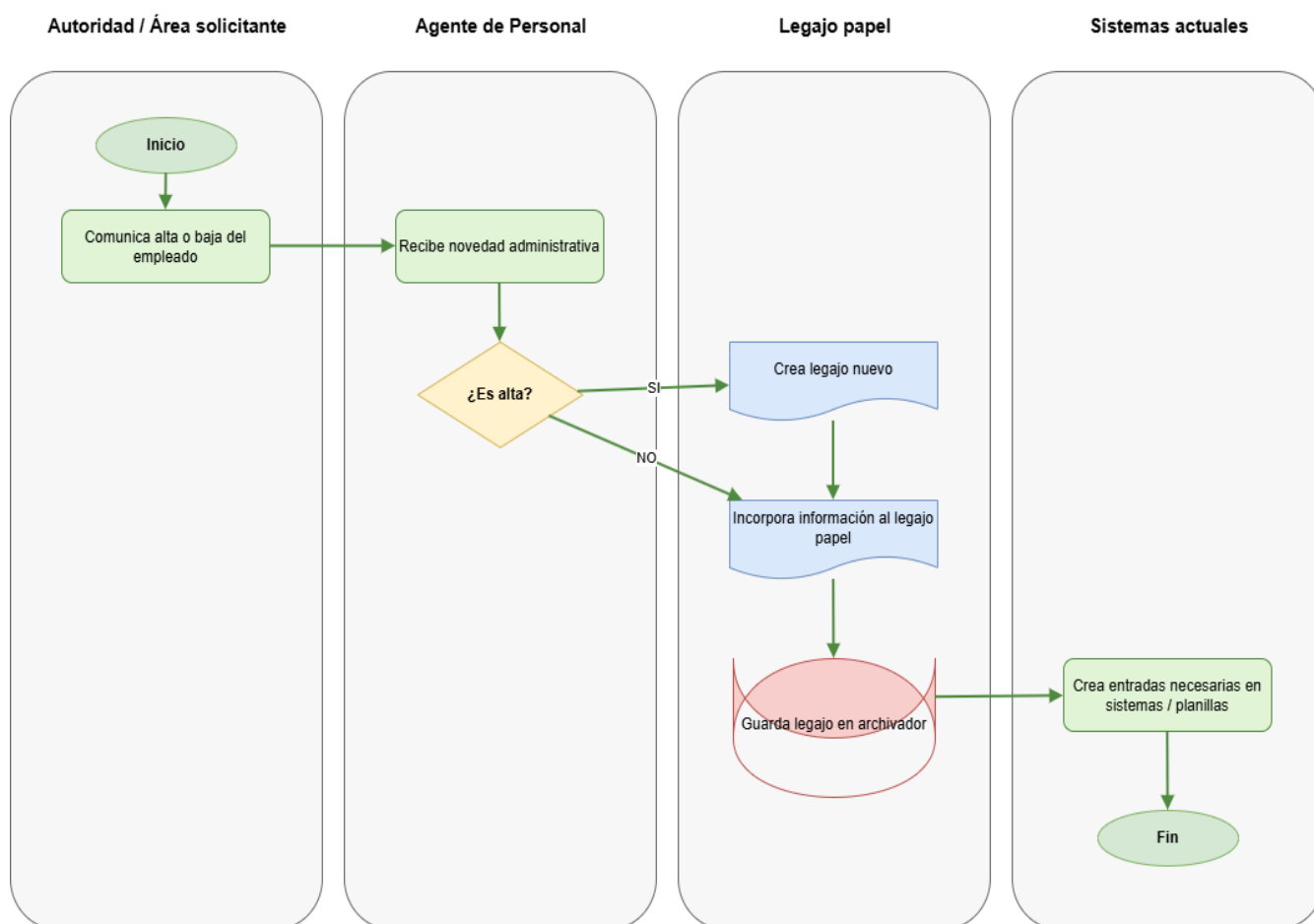


Imagen 2 – AS-IS Alta/Baja Agente

Nombre de proceso: **Carga Licencia**

Rol: Agente Administrativo de Personal

Ejecución:

1. Se recibe el pedido en papel de la licencia, del tipo que sea
2. Se registra en la planilla de cálculo correspondiente.
3. Se busca legajo en archivador
4. Se guarda el papel en su legajo.
5. Se guarda legajo en archivador

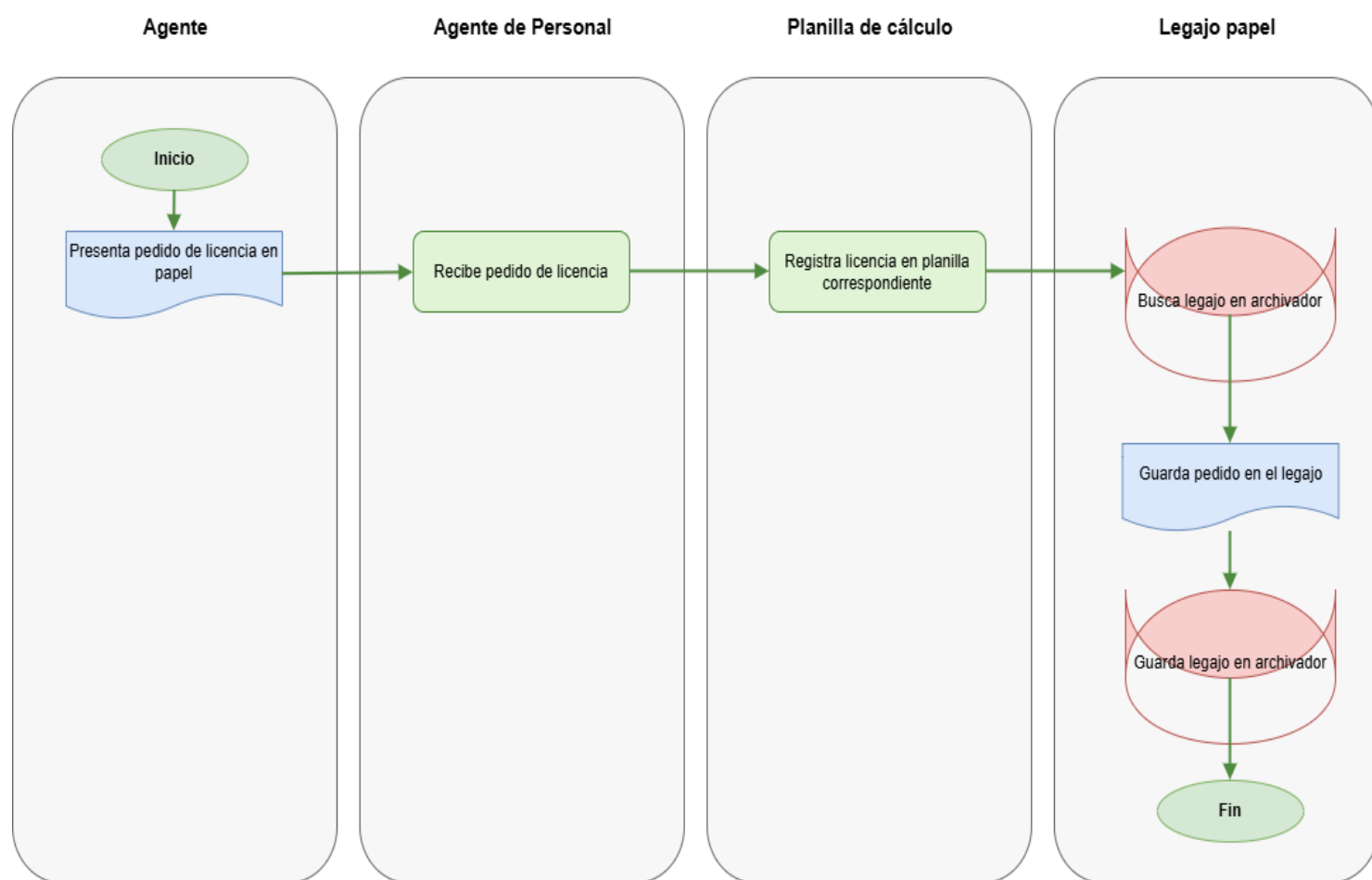


Imagen 3 – AS-IS Carga Licencia

Nombre de proceso: **Control faltas**

Rol: Agente Administrativo de Personal

Ejecución:

1. Se recibe aviso por mensaje/llamado/aviso de superior
2. Se registra en sistema VB – “Asistencia”
3. Si corresponde alguna sanción, Se busca legajo en archivador
4. Se incorpora al legajo papel
5. Se guarda legajo en archivador

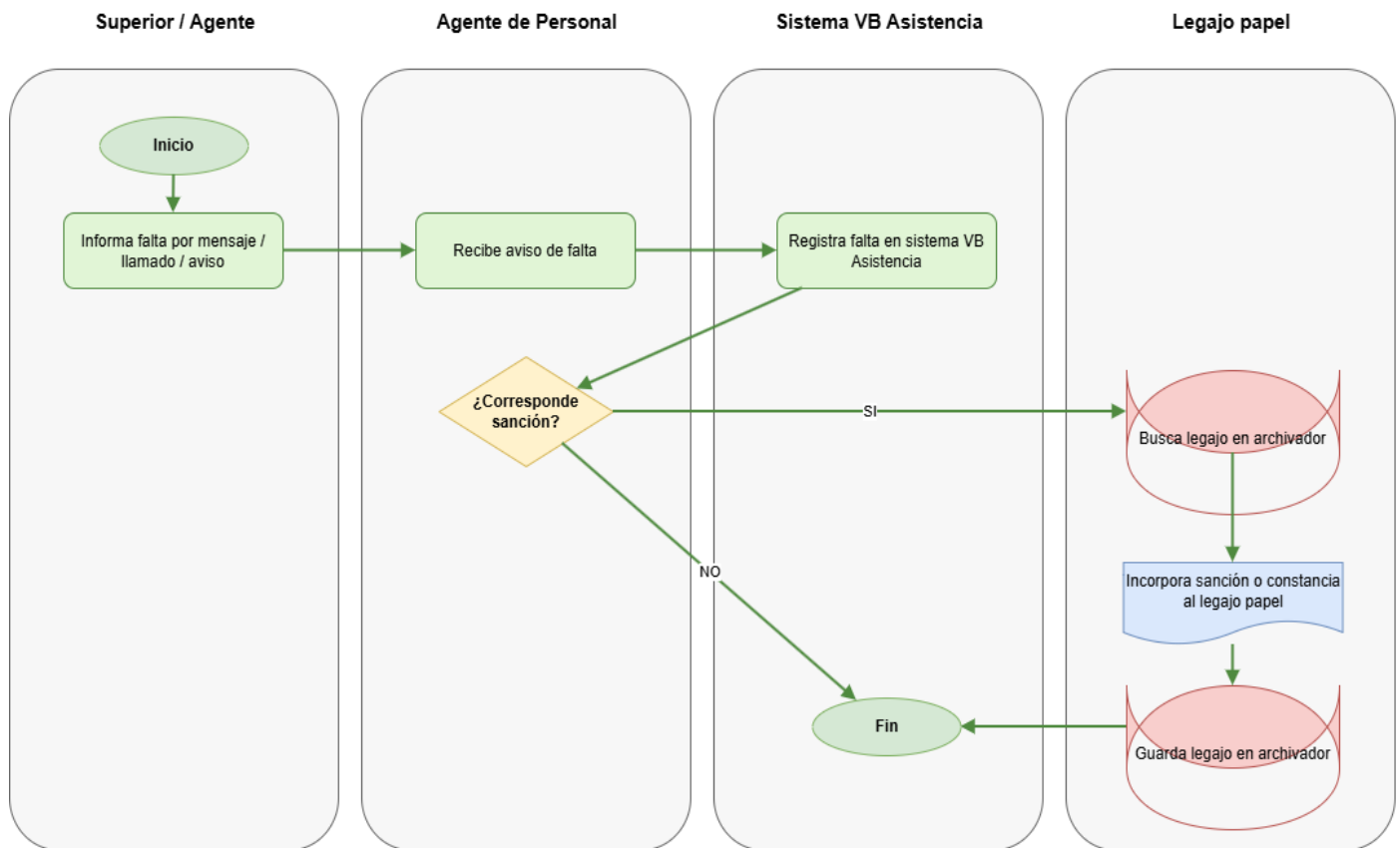


Imagen 4 – AS-IS Control de faltas

Nombre de proceso: **Emitir Certificado**

Rol: Agente Administrativo de Personal

Ejecución:

1. Se recibe el pedido de emisión de certificado con la información necesaria
2. Se busca legajo en archivador
3. Dentro del legajo se consulta la información necesaria
4. Se redacta en procesador de texto
5. Se guarda legajo en archivador
6. Se imprime y firma
7. Es entregado al solicitante.

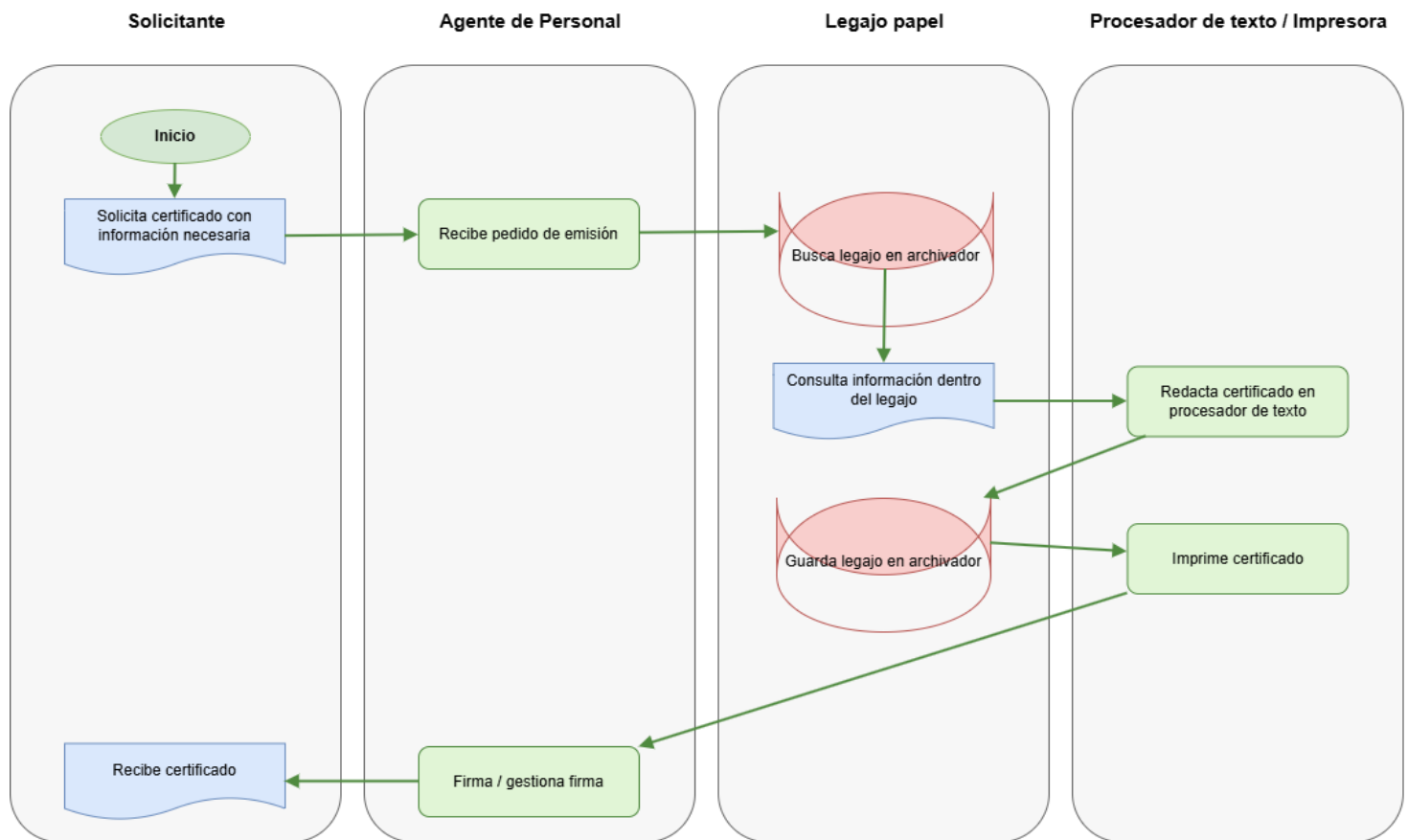


Imagen 5 – AS-IS Emitir certificado

Nombre del proceso: **Modificar Legajo**

Rol: Agente Administrativo de Personal

Ejecución:

1. Se busca legajo del agente en cuestión en archivador
2. Se incorpora la información necesaria, puede ser por alguno de los procesos anteriores.
3. Se guarda legajo en archivador

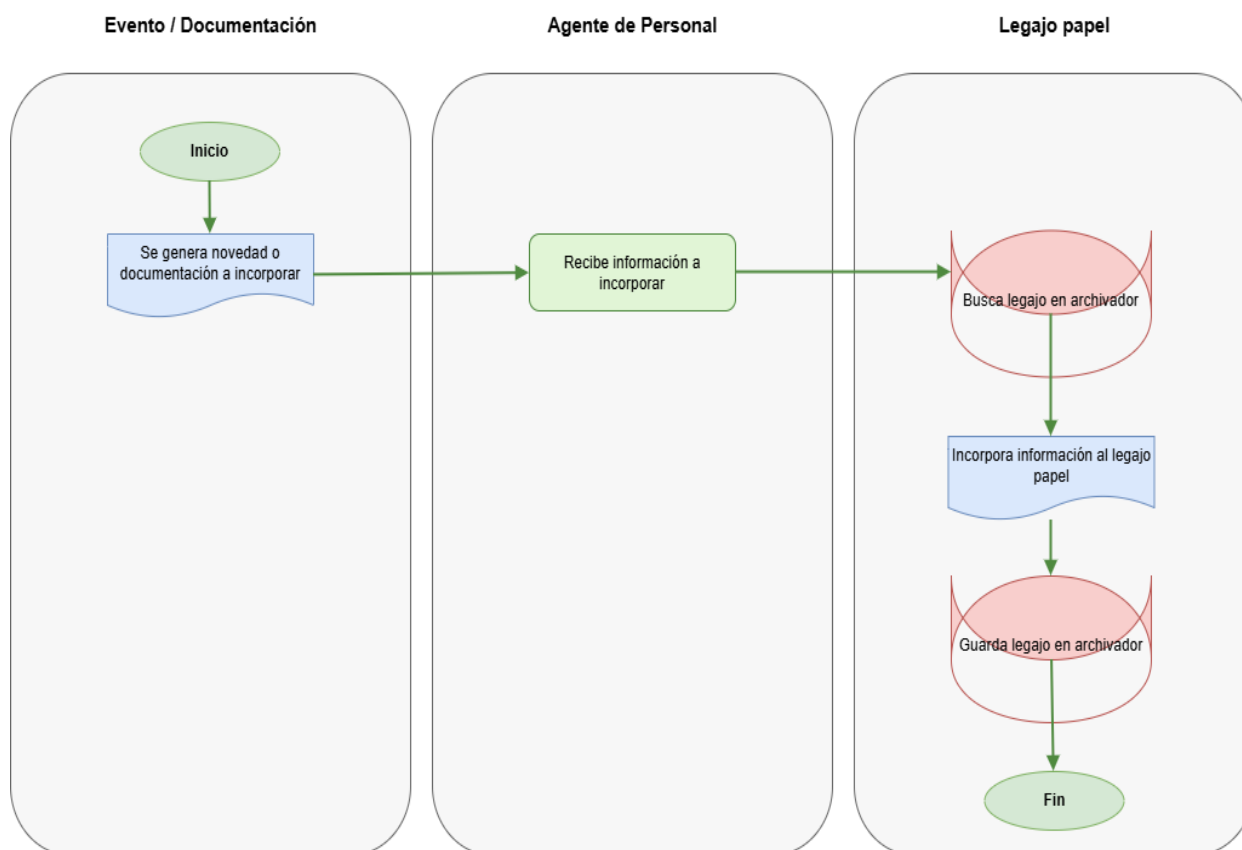


Imagen 6 – AS-IS Modificar Legajo

6.4 – DIAGNOSTICO

- **Proceso: Alta/Baja Empleado**

Problema: Se pierden o rompen legajos íntegramente o parcialmente

Causa: No se ha despapelizado la gestión de legajos, y al estar todos juntos archivados de manera física quedan expuestos a diversos factores.

- **Proceso: Carga Licencia**

Problema: Se “pierden” días de licencia o no se conmutan como corresponden. Produciendo depende del tipo de licencia, pérdidas económicas o perdidas de días disponibles para los agentes públicos.

Causa: al ser una planilla de cálculo, es muy común que por un mal empleo de fórmulas dentro de la misma resulten en situaciones complicadas.

- **Proceso: Control de faltas**

Problema: Muy lenta ejecución del sistema, y pérdidas ocasionales de información

Causa: es de lenta ejecución puesto que está programado con tecnologías viejas, apenas compatible con sistemas operativos actuales, se debe parchear para funcionar en equipos con S.O de 64 bits. Además, cuando se debe reinstalar la computadora que lo contiene, se corre el riesgo de perder archivos.

- **Proceso: Emitir Certificado**

Problema: Alta demora en la emisión de la nota solicitada.

Causa: No se tiene acceso a la información de forma eficaz, ya que cuando se requiere algún dato del empleado hay que recurrir a un archivador y buscar el legajo del agente en cuestión; y contar con la suerte que el papel con la información se encuentre, y en buenas condiciones

- **Proceso: Modificar Legajo**

Problema: Alta demora en la ejecución de esta tarea

Causa: Hay que buscar legajo en archivador y agregar la modificación

6.5– DIAGRAMA DE DOMINIO

El modelo de dominio permite representar los principales conceptos identificados dentro del contexto de la Oficina de Personal y las relaciones existentes entre ellos. Este modelo no representa todavía clases de programación, sino entidades conceptuales del negocio que serán utilizadas como base para el análisis, los casos de uso, el diseño de clases y el posterior modelo de base de datos.

En el sistema propuesto, el agente constituye la entidad central del dominio, ya que sobre él se registran legajos, licencias, faltas, certificaciones y documentación asociada. También se incorporan los usuarios del sistema, los roles y la auditoría, debido a que el sistema debe contemplar control de acceso y trazabilidad sobre las operaciones realizadas.

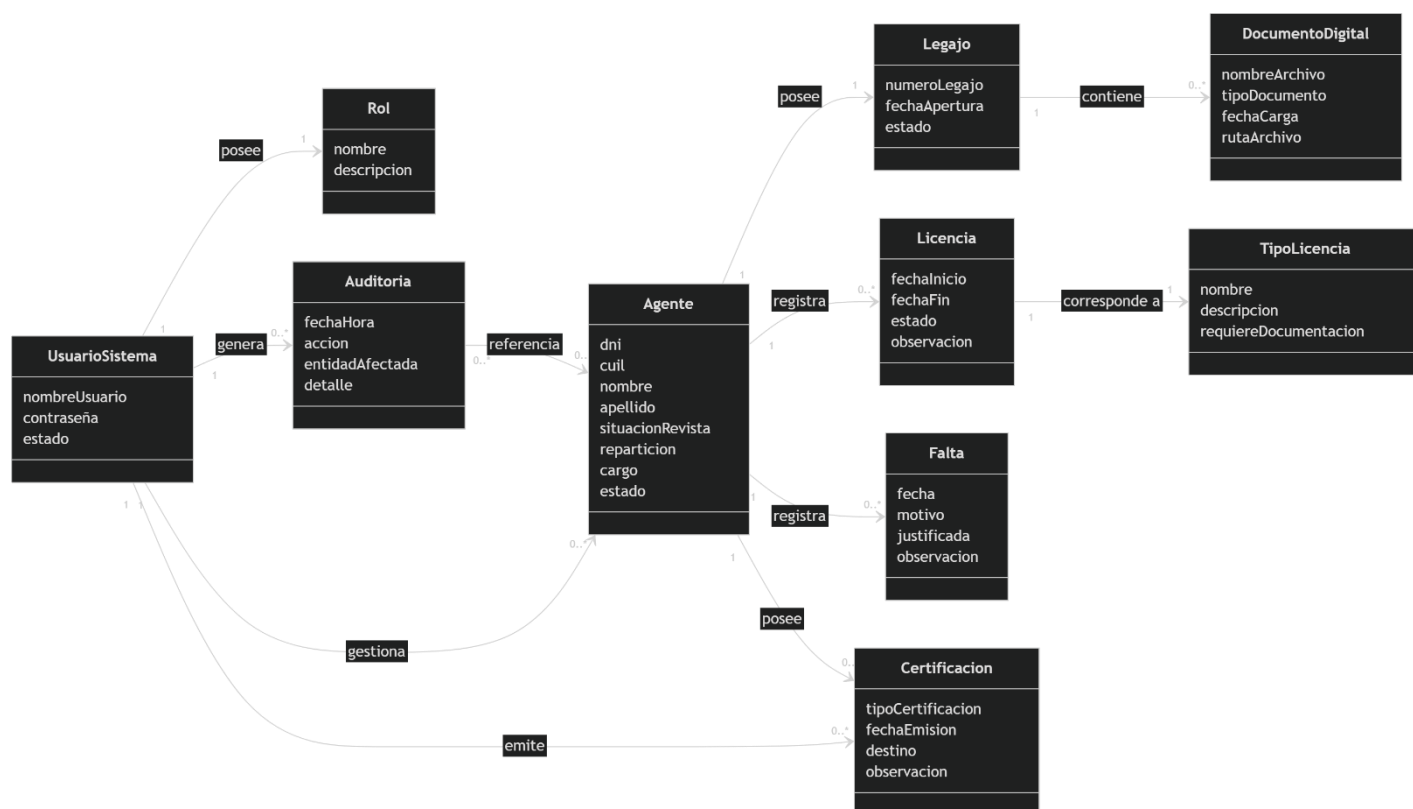


Imagen 7 – Diagrama de Dominio

El diagrama muestra que cada agente posee un legajo, al cual se vincula la documentación digital correspondiente. Además, sobre el agente se registran licencias, faltas y certificaciones administrativas.

Por otra parte, los usuarios del sistema operan de acuerdo con un rol determinado y sus acciones quedan registradas mediante auditoría. Esto permite mantener trazabilidad sobre las operaciones realizadas y contribuye a proteger la información sensible administrada por la Oficina de Personal.

7 – PROPUESTA DE SOLUCION

Una vez realizado el diagnostico de las tareas se desprende una serie de problemas que son completamente salvables con la realización del proyecto

7.1 -PROPUESTA FUNCIONAL

La propuesta contempla un sistema que integre todos los procesos relevados anteriormente

El operador ingresa con su usuario y contraseña, y se encuentra con una pantalla con accesos a las diversas funciones del sistema:

1. **LEGAJOS**: se pueden dar de alta agentes nuevos, cargando la información básica requerida, se puede adjuntar a un archivo.pdf, con legajo papel digitalizado. También se puede buscar entre legajos y modificar el que sea necesario. Este nuevo legajo digital, con cada modificación, adjunta una página nueva con la novedad a un legajo.pdf
2. **LICENCIAS**: se busca y selecciona el agente en cuestión. Se mostrará la licencia anual reglamentaria correspondiente y disponible, para que se haga una carga de los días solicitados junto con el comienzo y fin de la misma. En caso de que la licencia sea por otros motivos tendrá que fundamentarlo con un certificado explicando los motivos, que se agregará como una página más a su legajo.pdf
3. **ASISTENCIA**: se busca y selecciona el agente en cuestión y se le imputa los días faltados, como también se puede adjuntar un archivo.pdf con alguna notificación por sanción o una digitalización de un certificado o comprobante que solvante la falta laboral, en caso de que exista este, se adjuntará el archivo.pdf a su legajo.pdf
4. **CERTIFICADOS**: se selecciona la certificación a emitir, esto provocará que se mostrará una plantilla con la información del certificado seleccionado, luego se procede en un buscador interno, a seleccionar el agente en cuestión, al hacer clic sobre él, se cargará la plantilla con los datos del agente.

7.2 – PROPUESTA TECNICA

Para el sistema propuesto es elegido el lenguaje Java, que cumple con todos los requisitos mencionados anteriormente en el área de TICS, usando base de datos Mysql.

No es requerido en ninguno de los casos anteriormente mencionados la adquisición de licencias, y para el equipamiento informático no hace falta adecuaciones de ningún tipo, pudiendo correr sobre cualquier sistema operativo y hardware ya existente.

Se propondrá terminar de digitalizar el resto de legajos que aún está en papel, como así también los que ya están digitalizados, actualizarlos con los papeles nuevos. **ESTA TAREA NO ES PARTE DEL FUNCIONAMIENTO DEL SISTEMA A DESARROLLAR**

7.3 – LISTADO DE REQUERIMIENTOS FUNCIONALES

ID	Nombre	Descripción
----	--------	-------------

RQF01	Autenticación	El sistema deberá permitir el acceso de usuarios autorizados mediante nombre de usuario y contraseña.
RQF02	Gestión de agentes	El sistema deberá permitir registrar, modificar, consultar y realizar la baja lógica de agentes.
RQF03	Gestión de legajo	El sistema deberá permitir mantener la información administrativa integral del agente en un legajo digital.
RQF04	Registro de faltas	El sistema deberá permitir registrar faltas e inasistencias asociadas a los agentes.
RQF05	Gestión de licencias	El sistema deberá permitir registrar, consultar y actualizar licencias asociadas a los agentes.
RQF06	Gestión de certificados	El sistema deberá permitir emitir, consultar y registrar certificaciones administrativas de los agentes.
RQF07	Gestión de documentación	El sistema deberá permitir adjuntar, consultar y organizar documentación digital asociada al legajo del agente.
RQF08	Generación de informes	El sistema deberá permitir generar informes administrativos sobre agentes, legajos, licencias, faltas y certificaciones, según los permisos del usuario.

7.4 - LISTADO DE REQUERIMIENTOS NO FUNCIONALES

ID	Descripción
RQNF01	El sistema deberá contar con una interfaz simple e intuitiva, reduciendo la carga cognitiva del usuario operador.
RQNF02	El sistema deberá proteger los datos sensibles de los agentes mediante autenticación, control de acceso y restricción de permisos según rol.
RQNF03	El sistema deberá responder las operaciones habituales de consulta y registro en un tiempo máximo estimado de tres segundos, bajo condiciones normales de uso.
RQNF04	El sistema deberá permitir la emisión de informes y certificaciones mediante impresión física o generación de archivos PDF.
RQNF05	El sistema deberá registrar auditoría de las acciones relevantes realizadas por los usuarios, incluyendo fecha, hora, usuario, acción y entidad afectada.
RQNF06	El sistema deberá permitir el acceso concurrente de múltiples usuarios autorizados dentro de la red interna institucional.

7.5 – LISTADO DE REQUERIMIENTOS CANDIDATOS

ID	Descripción
RQC01	Permitir el acceso web al sistema para consultas autorizadas desde fuera del lugar físico de trabajo.
RQC02	Integrar el sistema con sistemas CORE del Gobierno de Mendoza en futuras etapas.
RQC03	Incorporar notificaciones automáticas a agentes y usuarios internos.
RQC04	Desarrollar un portal de autogestión para que los agentes puedan consultar su información administrativa.

8 – INICIO DEL ANALISIS: CASOS DE USO

8.1 - DIAGRAMA DE CASOS DE USO

Identificación de actores:

- **Personal de RRHH:** Es el actor principal del sistema, ya que es quien opera directamente la aplicación.
- **Agente:** es un actor secundario, debido a que puede interactúa constantemente con la oficina de RRHH. Pero no opera la aplicación.
- **Autoridad:** es también un actor secundario, ya que solicita información a la oficina de personal. Pero no opera la aplicación.

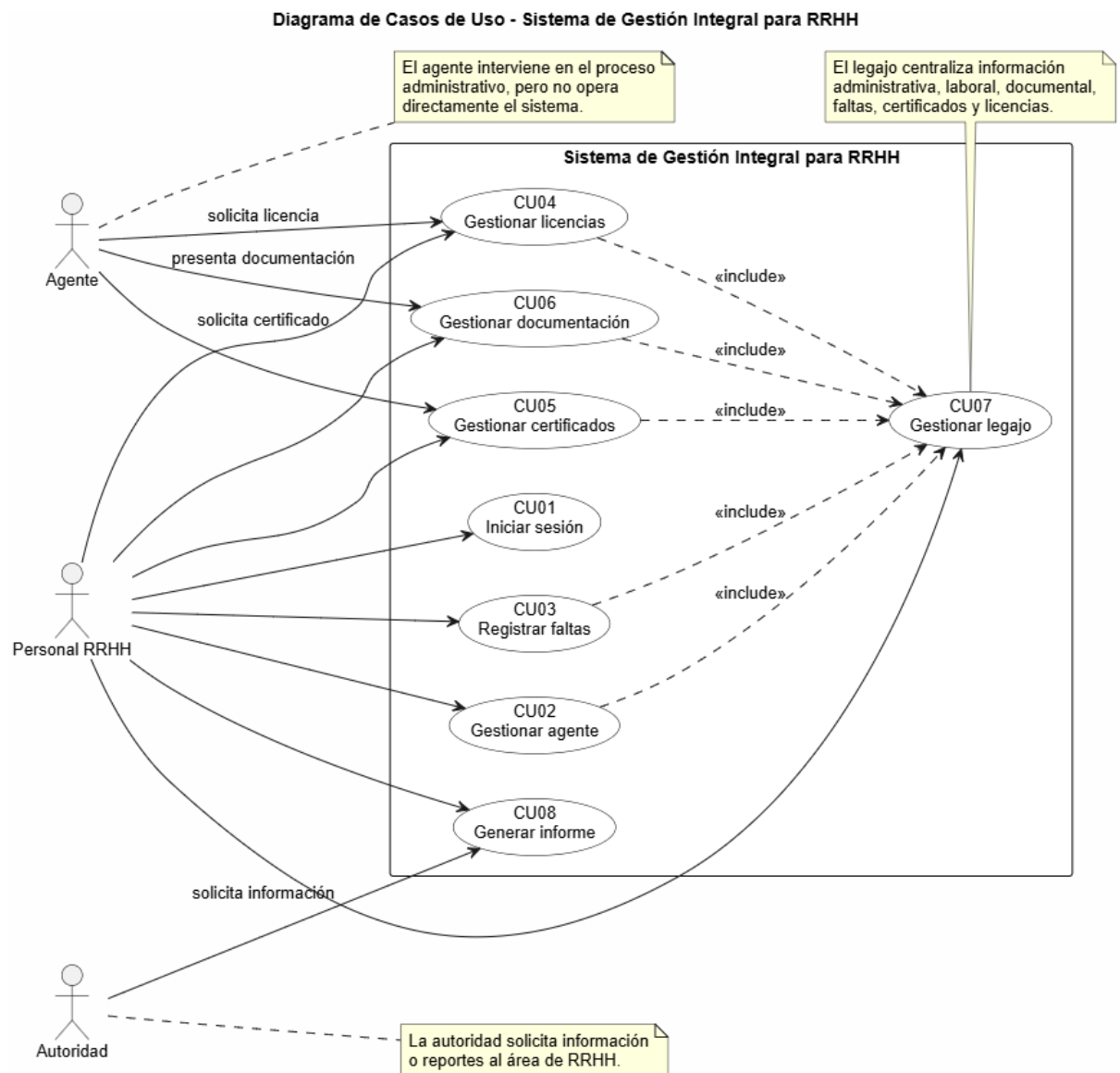


Imagen 8 – Diagrama de casos de Uso

ID	Caso de Uso	Actor Principal	Comentario
RQF01	CU01	Personal RRHH (Oficina de Personal)	Permitir el acceso mediante credenciales
RQF02	CU02	Personal RRHH (Oficina de Personal)	Gestionar agente
RQF03	CU07	Personal RRHH (Oficina de Personal)	Gestionar legajo
RQF04	CU03	Personal RRHH (Oficina de Personal)	Registrar faltas laborales
RQF05	CU04	Personal RRHH (Oficina de Personal)	Gestionar licencias
RQF06	CU05	Personal RRHH (Oficina de Personal)	Gestionar certificados
RQF07	CU06	Personal RRHH (Oficina de Personal)	Gestionar documentación
RQF08	CU08	Personal RRHH (Oficina de Personal)	Generar informe

8.2 – DESCRIPCION DE CASOS DE USO

Luego del análisis de los requerimientos funcionales, se desglosa la existencia de 8 casos de uso, que serán detallados a continuación:

Caso de uso	CU01	
Nombre	Iniciar Sesión	
Actores	Personal RRHH (Principal)	
Referencias	RQF01	
Descripción	Permitir Acceso mediante credenciales	
Precondición	El usuario debe estar registrado	
Flujo principal	1	Ingresa usuario y contraseña
	2	El sistema valida
	3	Acceso concedido
Postcondición	Acceso al sistema Habilitado	
Flujo alternativo	A1	Credenciales incorrectas -> mensaje de error

Caso de uso	CU02	
Nombre	Gestionar Agente	
Actores	Personal RRHH (Principal)	
Referencias	RQF02	
Descripción	Administrar datos de los agentes	
Precondición	El usuario debe estar autenticado	
Flujo principal	1	Seleccionar agente

	2	Ingresar / Modificar datos
	3	Guardar
Postcondición	Datos Actualizados	
Flujo alternativo	A1	Datos Incompletos -> solicitar corrección

Caso de uso	CU03	
Nombre	Registrar faltas	
Actores	Personal RRHH (Principal)	
Referencias	RQF04	
Descripción	Registrar faltas laborales de los agentes	
Precondición	El usuario debe estar autenticado	
Flujo principal	1	Seleccionar agente
	2	Ingresar falta
	3	Guardar
Postcondición	Datos Actualizados	
Flujo alternativo	A1	Agente inexistente - > cancelar
	A2	Fecha errónea -> consultar calendario

Caso de uso	CU04	
Nombre	Gestionar licencias	
Actores	Personal RRHH (Principal)	
Referencias	RQF05	
Descripción	Registrar y controlar licencias	
Precondición	Agente existente	
Flujo principal	1	Recibe solicitud
	2	Selecciona agente
	3	Registra licencia
	4	Actualiza saldo

Postcondición	Licencia registrada	
Flujo alternativo	A1	Licencia insuficientes -> rechazo

Caso de uso	CU05	
Nombre	Gestionar certificados	
Actores	Personal RRHH (Principal)	
Referencias	RQF06	
Descripción	Crear y editar certificados	
Precondición	Usuario autenticado	
Flujo principal	1	Selecciona plantilla
	2	Edita
	3	Adjunta Certificado
	3	Guarda
Postcondición	Certificado generado	
Flujo alternativo	A1	Error en datos -> corregir
	A2	Imprimir/enviar certificado
Otros	Extend CU08 – Generar Informe	

Caso de uso	CU06	
Nombre	Gestionar documentación	
Actores	Personal RRHH (Principal) - Agente (Secundario)	
Referencias	RQF07	
Descripción	Registrar documentación del agente	
Precondición	Agente existente	
Flujo principal	1	Recibe documentación
	2	selecciona agente
	3	Adjunta archivo
	3	Guarda
Postcondición	Archivo almacenado	
Flujo alternativo	A1	Archivo invalido -> rechazo

Caso de uso	CU07	
Nombre	Gestionar legajo	
Actores	Personal RRHH (Principal)	
Referencias	RQF03	
Descripción	Centralizar información del agente	
Precondición	Agente existente	
Flujo principal	1	Buscar agente
	2	Visualizar datos
	3	Actualizar
Postcondición	Información disponible y actualizada	
Flujo alternativo	A1	Legajo no encontrado -> error
Otros	-	

Caso de uso	CU08	
Nombre	Generar Informe	
Actores	Personal RRHH (Principal) - Autoridad (Secundario)	
Referencias	RQF08	
Descripción	Generar informes solicitados	
Precondición	Solicitud de autoridad	
Flujo principal	1	Recibe solicitud
	2	Define Criterios
	3	Genera Informe
	4	Entrega impresión/digital
Postcondición	Informe generado y entregado	
Flujo alternativo	A1	Datos insuficientes -> ajustar
Otros	extend: puede incluir certificados	

9 – ETAPA DE ANALISIS

En esta etapa se utilizará como caso de uso principal el **CU04 - Gestionar Licencias**, debido a que permite integrar los módulos de Gestión de Agentes, Gestión de Legajos y Gestión de Licencias. Toda licencia debe estar asociada a un agente previamente registrado y debe quedar incorporada en su legajo administrativo, por lo que este caso de uso permite demostrar adecuadamente la interacción entre los componentes principales del prototipo.

Además del CU04 - Gestionar Licencias, se considera el **CU02 - Gestionar Agente** como caso de uso complementario del prototipo. Este caso permite registrar y actualizar los datos básicos de los agentes, condición necesaria para que posteriormente puedan gestionarse sus legajos y licencias.

9.1 - ALCANCE DEL PROTOTIPO

Si bien el diseño general del Sistema de Gestión Integral para RRHH contempla una solución amplia para la administración centralizada de la información del personal del Ministerio, el prototipo operacional correspondiente se acota a tres módulos funcionales principales: Gestión de Agentes, Gestión de Legajos y Gestión de Licencias.

Este recorte responde a criterios de viabilidad técnica, alcance académico y plazos de desarrollo. La selección de estos módulos permite representar el núcleo funcional del sistema, ya que toda licencia debe estar asociada a un agente registrado y debe impactar en su legajo administrativo.

Los módulos definidos para el prototipo son los siguientes:

- **Gestión de Agentes:** permite registrar, consultar y actualizar los datos personales, identificatorios y laborales de los agentes pertenecientes a la institución.
- **Gestión de Legajos:** permite centralizar y consultar la información histórica, administrativa y documental vinculada a cada agente, facilitando el seguimiento de su trayectoria dentro del organismo.
- **Gestión de Licencias:** permite registrar solicitudes de licencias, validar fechas, consultar antecedentes asociados al agente y registrar el estado administrativo de la solicitud, ya sea aprobada, rechazada u observada.

De esta manera, el CU04 permite demostrar la trazabilidad entre los requerimientos funcionales, las clases de análisis, las clases de diseño, el modelo de base de datos y los casos de prueba definidos para esta segunda entrega.

9.2 – DIAGRAMA DE SECUENCIA

El siguiente diagrama de secuencia representa el caso de uso **CU04 - Gestionar Licencias**. Este caso de uso fue seleccionado porque permite integrar los tres módulos principales definidos para el prototipo: Gestión de Agentes, Gestión de Legajos y Gestión de Licencias.

El flujo comienza cuando el Agente Administrativo de Personal ingresa al módulo de licencias y busca un agente registrado. Luego, el sistema consulta los datos del agente y su legajo administrativo. Una vez seleccionado el tipo de licencia, el usuario carga las fechas correspondientes y el sistema valida la consistencia de los datos ingresados.

Si los datos son correctos, la licencia se registra, el legajo del agente se actualiza y el sistema informa que la operación fue realizada correctamente. En caso de que el agente no exista, las fechas sean inválidas o la licencia no cumpla las condiciones requeridas, el sistema informa el error y no registra la solicitud.

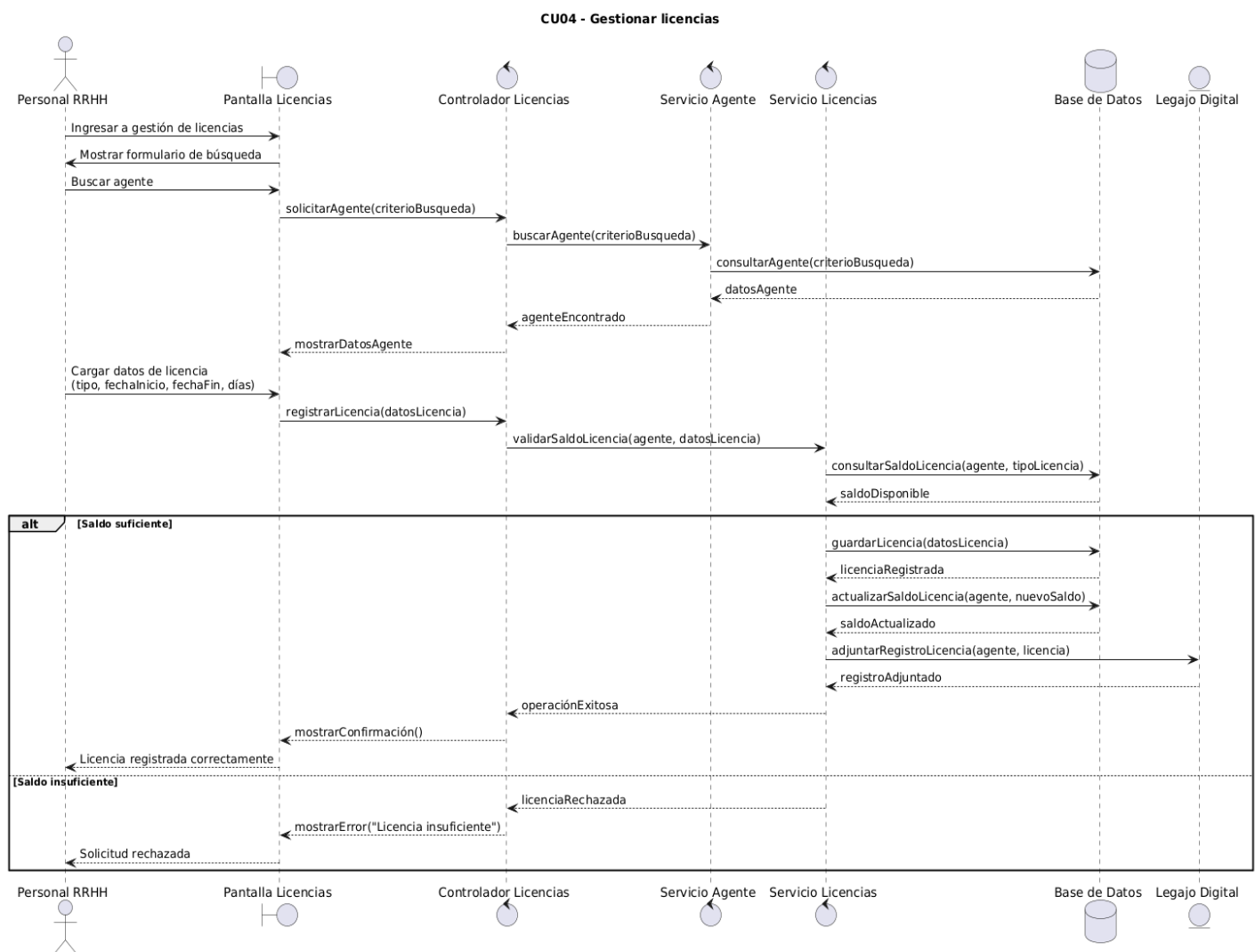


Imagen 9 – Diagrama de secuencia CU4

10 – ETAPA DE DISEÑO

El diseño se limita al alcance definido para el prototipo operacional: Gestión de Agentes, Gestión de Legajos y Gestión de Licencias. Esta delimitación permite construir un modelo coherente, verificable y alineado con el **CU04 - Gestionar Licencias**.

El diseño propuesto contempla las clases necesarias para registrar agentes, administrar su legajo, gestionar solicitudes de licencias, validar tipos de licencia y controlar el acceso de usuarios autorizados.

10.1 – CLASES PRINCIPALES DEL DISEÑO

A partir del alcance definido para el prototipo y del análisis realizado sobre el **CU04 - Gestionar Licencias**, se identifican las siguientes clases principales:

- **UsuarioSistema:** Representa a los usuarios autorizados para acceder al sistema. Contiene los datos necesarios para autenticación y control de permisos básicos.
- **Rol:** Representa el perfil asignado a cada usuario del sistema. Permite diferenciar permisos según el tipo de usuario, por ejemplo, administrador o personal de RRHH.
- **Agente:** Representa al agente perteneciente al Ministerio. Contiene datos personales, identificatorios y laborales necesarios para su gestión administrativa.
- **Legajo:** Representa el historial administrativo asociado a un agente. Permite centralizar información de carrera, situación de revista, documentación y novedades relevantes.
- **Licencia:** Representa una solicitud o registro de licencia vinculada a un agente. Contiene fechas, cantidad de días, estado, observaciones y relación con el tipo de licencia correspondiente.
- **TipoLicencia:** Representa la clasificación de las licencias disponibles. Define el nombre del tipo de licencia, si requiere documentación respaldatoria y si consume saldo disponible.
- **DocumentoLegajo:** Representa documentación asociada al legajo del agente. Puede corresponder a archivos respaldatorios, notas administrativas, constancias o documentación relacionada con una licencia.
- **Auditoria:** Representa el registro de acciones realizadas dentro del sistema. Permite mantener trazabilidad básica sobre operaciones relevantes, como altas, modificaciones o registros de licencias.

10.2 – DIAGRAMA DE CLASES DE DISEÑO

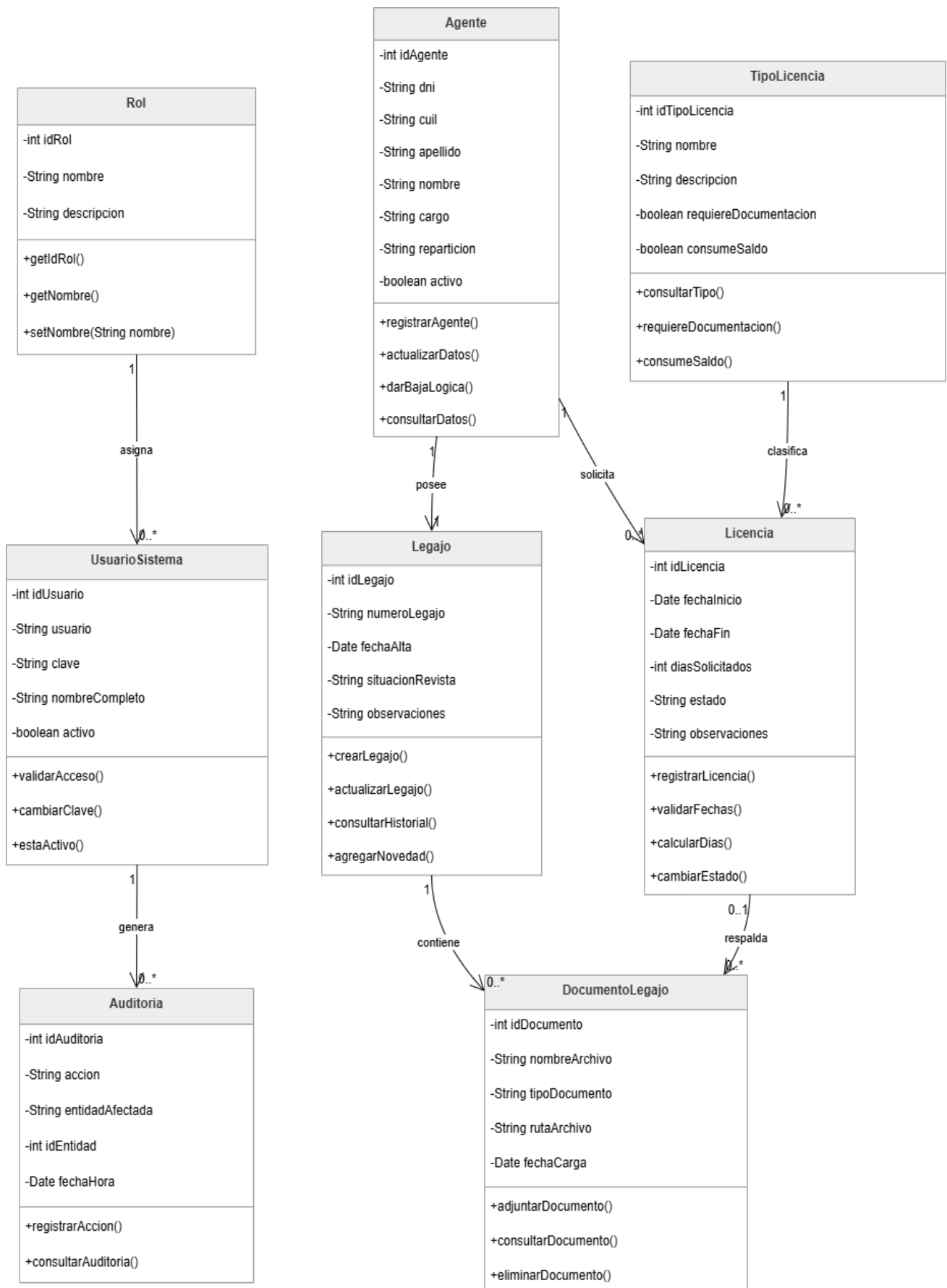


Imagen 10 – Diagrama de clases

11 – ETAPA DE IMPLEMENTACION

En esta instancia se establecen las tecnologías seleccionadas, la arquitectura general del sistema, los componentes principales y la forma en que estos interactúan para permitir el funcionamiento de los módulos de Gestión de Agentes, Gestión de Legajos y Gestión de Licencias.

El prototipo será desarrollado como una aplicación de escritorio en Java, utilizando una base de datos MySQL para la persistencia de la información.

La implementación no contempla el desarrollo de una aplicación web ni el uso de frameworks externos. El objetivo es construir un prototipo simple, funcional y verificable, que permita demostrar el registro, consulta y actualización de información administrativa asociada a agentes, legajos y licencias.

11.1 - ALCANCE IMPLEMENTADO

El prototipo implementado no cubre la totalidad del sistema final proyectado, sino un conjunto acotado de funcionalidades necesarias para validar el funcionamiento general del sistema.

Las funcionalidades implementadas son:

- Inicio de sesión de usuario autorizado.
- Visualización de menú principal.
- Registro de agentes.
- Listado de agentes cargados.
- Búsqueda de agente por DNI.
- Registro de licencia asociada a un agente existente.
- Validación básica de datos obligatorios.
- Validación de fechas en la carga de licencias.
- Visualización de licencias registradas.

Las funcionalidades no implementadas en esta etapa quedan previstas para ampliaciones posteriores del sistema. Entre ellas se encuentran la persistencia completa desde Java hacia MySQL, la gestión avanzada de usuarios, la emisión automática de reportes, la integración con otros sistemas institucionales y la administración completa del legajo digital.

11.2 - DEFINICIONES TÉCNICAS DE IMPLEMENTACIÓN

Para la implementación del prototipo se utilizaron las siguientes herramientas y tecnologías:

- Lenguaje de programación: Java.
- Entorno de desarrollo: NetBeans IDE.
- Tipo de aplicación: Aplicación Java Desktop.
- Biblioteca gráfica: Swing.
- Base de datos definida para el sistema: MySQL / MariaDB.
- Entorno de base de datos utilizado para pruebas: XAMPP y phpMyAdmin.
- Organización del proyecto: paquetes separados para clase principal, modelo, datos y vistas.

El prototipo fue desarrollado siguiendo una organización básica por responsabilidades. La clase principal inicia la aplicación, las clases del modelo representan las entidades principales del dominio, las clases de vista contienen las pantallas gráficas y la clase de datos permite administrar temporalmente la información durante la ejecución del programa.

Esta organización permite separar parcialmente la lógica de presentación, la representación de los datos y el almacenamiento temporal, facilitando la comprensión del prototipo y su posterior ampliación.

11.3 - REQUERIMIENTOS NO FUNCIONALES DEL PROTOTIPO

ID	Descripción
RNF01	El sistema debe ser desarrollado en Java, utilizando una aplicación de escritorio como medio de interacción con el usuario
RNF02	El sistema debe utilizar una base de datos MySQL para almacenar la información correspondiente a usuarios, agentes, legajos, licencias, tipos de licencia, documentos asociados y auditoría básica.
RNF03	El sistema utilizará XAMPP como entorno de apoyo para disponer del servicio MySQL y la herramienta phpMyAdmin para administración visual de la base de datos durante el desarrollo.
RNF04	El prototipo contará con una interfaz gráfica de escritorio desarrollada con Java Swing, permitiendo al usuario interactuar con las funcionalidades principales sin necesidad de operar directamente sobre la base de datos.
RNF05	La aplicación deberá conectarse a la base de datos mediante JDBC, permitiendo registrar, consultar, actualizar y validar información persistente
RNF06	El acceso al sistema deberá estar restringido mediante autenticación de usuarios. Las acciones principales deberán estar asociadas al usuario que las ejecuta.
RNF07	Las operaciones relevantes, como altas, modificaciones y registros de licencias, deberán poder ser registradas para mantener trazabilidad básica de uso.
RNF08	El prototipo no implementa la totalidad del sistema integral, sino únicamente los módulos definidos para el AP2: Gestión de Agentes, Gestión de Legajos y Gestión de Licencias.

11.4 – DIAGRAMA DE DESPLIEGUE

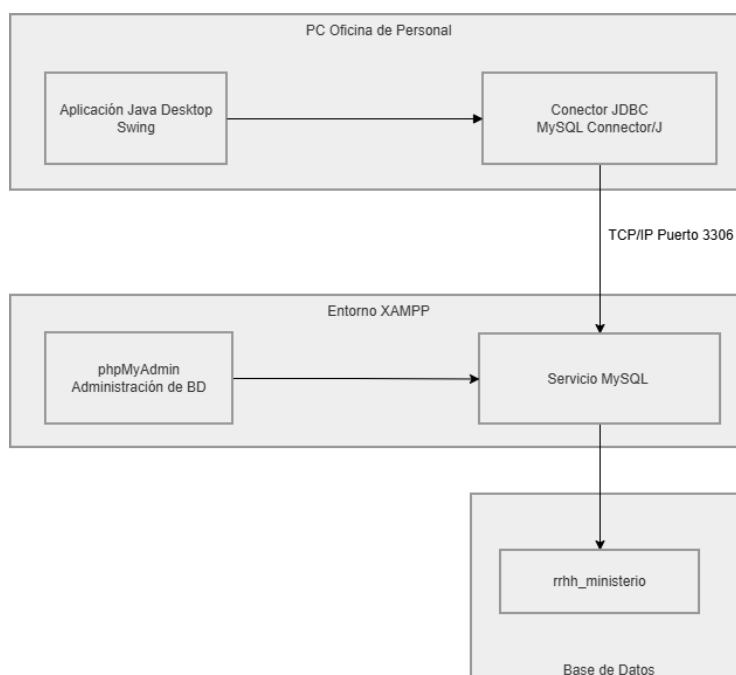


Imagen 11 – Diagrama de despliegue

El diagrama muestra los componentes principales necesarios para ejecutar el prototipo operacional.

La aplicación Java Desktop representa el sistema utilizado por el personal autorizado de la Oficina de Personal. Desde esta aplicación se accede a las pantallas de inicio de sesión, gestión de agentes, consulta de legajos y registro de licencias.

El conector JDBC permite que la aplicación Java se comuniquen con la base de datos. Este componente actúa como intermediario entre el código Java y el motor MySQL.

El entorno XAMPP permite iniciar y administrar el servicio MySQL durante el desarrollo del prototipo.

La base de datos rrhh_ministerio almacena la información persistente utilizada por el sistema, incluyendo usuarios, roles, agentes, legajos, tipos de licencia y licencias registradas.

Para este prototipo, todos los componentes pueden ejecutarse en una misma computadora. Sin embargo, el diseño permite que en una futura etapa la aplicación se ejecute en distintos puestos de trabajo y la base de datos se aloje en un servidor interno de la red institucional.

12 - ETAPA DE PRUEBAS

Los objetivos del plan de pruebas son los siguientes:

- Verificar que solo usuarios autorizados puedan ingresar al sistema.
- Validar que un agente pueda ser registrado correctamente.

- Comprobar que el sistema permita consultar el legajo de un agente existente.
- Verificar que una licencia pueda registrarse cuando los datos ingresados sean válidos.
- Comprobar que el sistema rechace licencias con fechas incorrectas.
- Comprobar que el sistema informe error cuando se intente registrar una licencia para un agente inexistente.
- Verificar que la información registrada quede persistida en la base de datos.

Las pruebas serán ejecutadas de forma manual sobre el prototipo desarrollado en Java Desktop, utilizando una base de datos MySQL con datos de prueba previamente cargados.

Para cada caso de prueba se define un resultado esperado concreto, verificable y observable por el usuario o por el evaluador del prototipo.

El resultado esperado indica qué debe hacer el sistema cuando se ejecuta una acción determinada. Puede consistir en el registro correcto de datos, la visualización de información, la emisión de un mensaje de confirmación, la presentación de un mensaje de error o el rechazo de una operación inválida.

12.1 – PLAN DE PRUEBAS

Para identificar los casos de prueba se utilizará la siguiente nomenclatura:

- CP: caso de prueba.
- F: prueba funcional.
- V: prueba de validación.
- BD: prueba de persistencia en base de datos.

Caso de uso	Código prueba	Tipo de prueba	Técnica propuesta	Observaciones
CU01 - Iniciar sesión	CPS01	Seguridad / Sistema	Prueba funcional de acceso	Validar ingreso con usuario activo y rechazo de credenciales incorrectas
CU02 - Gestionar Agente	CPF02	Sistema	Prueba funcional	Verificar alta, consulta y modificación de agente
CU03 - Registrar faltas	CPF03	Sistema	Prueba funcional	Verificar registro de falta asociada a un agente existente
CU04 - Gestionar Licencias	CPF04	Sistema	Prueba funcional	Verificar registro correcto de licencia
CU04 - Gestionar Licencias	CPV05	Validación	Partición de equivalencia y frontera	Validar fechas de licencia
CU04 - Gestionar Licencias	CPBD06	Base de datos	Verificación SQL	Confirmar persistencia del registro de licencia

CU05 - Gestionar Certificados	CPF07	Sistema	Prueba funcional	Verificar emisión y registro de certificado
CU06 - Gestionar Documentación	CPF08	Sistema	Prueba funcional	Verificar carga de documentación asociada al agente
CU07 - Gestionar Legajo	CPF09	Sistema	Prueba funcional	Verificar consulta y actualización de legajo existente
CU08 - Generar Informe	CPF10	Sistema	Prueba funcional	Verificar generación de informe según criterio seleccionado

12.2 - MODELO DE PRUEBAS

El plan de pruebas anterior contempla la totalidad de los casos de uso definidos para el sistema integral. Los casos seleccionados para el desarrollo detallado de pruebas son:

CU02 - Gestionar Agente: Se utiliza como caso complementario, ya que permite registrar, consultar y actualizar los datos básicos de los agentes. Esta funcionalidad es necesaria para que posteriormente puedan gestionarse legajos y licencias.

CU04 - Gestionar Licencias: Se utiliza como caso principal de prueba, ya que integra los tres módulos definidos para el prototipo: Gestión de Agentes, Gestión de Legajos y Gestión de Licencias.

Así podemos validar un flujo funcional mínimo del sistema: primero se registra o consulta un agente, luego se accede a su información administrativa y finalmente se registra una licencia asociada.

El resto de los casos de uso queda contemplado dentro del plan general de pruebas del sistema integral, pero no será desarrollado en profundidad en esta instancia.

12.3 - CASO DE PRUEBA CPF02: GESTIONAR AGENTE

	Descripción
Código de prueba	CPF02
Caso de uso asociado	CU02 - Gestionar Agente
Tipo de prueba	Sistema / Funcional
Técnica propuesta	Prueba funcional manual
Objetivo	Verificar que el sistema permita registrar un agente nuevo y consultar posteriormente sus datos.
Precondiciones	El usuario debe haber iniciado sesión correctamente. El agente no debe encontrarse registrado previamente. La base de datos debe estar disponible.
Datos de entrada	DNI: 27949990. CUIL: 20-27949990-1 Apellido: Quinteros. Nombre: Rodrigo. Cargo: Administrativo. Repartición: Oficina de Personal. Estado: Activo.

Procedimiento	1. Ingresar al sistema con un usuario autorizado. 2. Acceder al módulo Gestión de Agentes. 3. Seleccionar la opción de alta o nuevo agente. 4. Completar los datos personales, identificatorios y laborales. 5. Confirmar la operación. 6. Consultar el agente registrado utilizando DNI o apellido.
Resultado esperado	El sistema valida que los campos obligatorios estén completos, registra el agente en memoria y lo muestra en el listado de agentes cargados
Mensaje esperado	Informa mediante un mensaje que el agente fue registrado correctamente.
Resultado no esperado	Si faltan datos obligatorios o el DNI ya existe, el sistema no registra el agente y muestra un mensaje de advertencia.

12.4 - CASO DE PRUEBA CPF04: REGISTRAR LICENCIA

	Descripción
Código de prueba	CPF04
Caso de uso asociado	CU04 - Gestionar Licencias
Tipo de prueba	Sistema / Funcional
Técnica propuesta	Prueba funcional manual
Objetivo	Verificar que el sistema permita registrar correctamente una licencia asociada a un agente existente.
Precondiciones	El usuario debe haber iniciado sesión correctamente. El agente debe existir en el sistema. El agente debe contar con legajo asociado. Debe existir el tipo de licencia seleccionado. La base de datos debe estar disponible.
Datos de entrada	Agente: DNI 27949990. Tipo de licencia: Licencia Anual Reglamentaria. Fecha de inicio: 20/05/2026. Fecha de finalización: 21/05/2026. Observaciones: Prueba de registro de licencia.
Procedimiento	1. Ingresar al sistema con un usuario autorizado. 2. Acceder al módulo Gestión de Licencias. 3. Buscar el agente por DNI. 4. Seleccionar el agente encontrado. 5. Verificar la información del legajo. 6. Seleccionar el tipo de licencia. 7. Ingresar fecha de inicio. 8. Ingresar fecha de finalización. 9. Ingresar observaciones. 10. Confirmar la operación.
Resultado esperado	El sistema localiza el agente, muestra sus datos identificatorios, valida la información ingresada y registra la licencia asociada al agente. La licencia queda visible en el listado de licencias registradas, mostrando nombre y apellido del agente, tipo de licencia, fechas y cantidad de días.
Mensaje esperado	Licencia registrada correctamente.
Resultado no esperado	Si el DNI no corresponde a un agente registrado, el sistema informa que el agente no fue encontrado y no permite registrar la licencia.

12.5 - CASO DE PRUEBA CPV05: VALIDACIÓN DE FECHAS EN LICENCIAS

	Descripción
Código de prueba	CPV05
Caso de uso asociado	CU04 - Gestionar Licencias
Tipo de prueba	Validación
Técnica propuesta	Partición de equivalencia y análisis de valores de frontera
Objetivo	Verificar que el sistema permita registrar una licencia cuando la fecha de finalización sea igual o posterior a la fecha de inicio, y que rechace la operación cuando la fecha de finalización sea anterior a la fecha de inicio.
Funcionalidad evaluada	Validación de fechas en el registro de licencias.
Precondiciones	El usuario debe haber iniciado sesión correctamente. El agente debe existir en el sistema. El agente debe tener un legajo asociado. Debe existir el tipo de licencia seleccionado. La base de datos debe estar disponible.
Datos generales	Agente: DNI 27949990. Tipo de licencia: Licencia Anual Reglamentaria. Observaciones: Prueba de validación de fechas.
Regla evaluada	La fecha de finalización de la licencia debe ser igual o posterior a la fecha de inicio.
Resultado esperado	El sistema acepta fechas válidas y rechaza fechas inválidas.
Resultado no esperado	El sistema no debe registrar licencias con fecha de finalización anterior a la fecha de inicio.

12.6 - PARTICIÓN DE EQUIVALENCIA

Para los casos de uso seleccionados dentro del modelo de pruebas se aplica la técnica de partición de equivalencia. Esta técnica permite dividir los datos de entrada en clases válidas e inválidas, verificando que el sistema responda correctamente ante cada grupo de datos.

En esta etapa se aplica partición de equivalencia sobre los casos CU02 - Gestionar Agente y CU04 - Gestionar Licencias, debido a que ambos forman parte del flujo funcional mínimo del prototipo. El CU02 permite registrar y consultar agentes, mientras que el CU04 permite registrar licencias asociadas a dichos agentes.

12.6.1 - PARTICIÓN DE EQUIVALENCIA PARA CU02 - GESTIONAR AGENTE

Para el CU02 - Gestionar Agente, se evalúa la carga de datos básicos del agente. La regla principal consiste en que el agente debe registrarse con datos obligatorios completos y con identificadores únicos, especialmente DNI y CUIL.

Clases de equivalencia válidas

Clase	Condición	Resultado esperado
CEV01	DNI válido, CUIL válido y datos obligatorios completos	El sistema permite registrar el agente
CEV02	Agente existente consultado por DNI válido	El sistema muestra los datos del agente
CEV03	Modificación de agente existente con datos válidos	El sistema actualiza los datos del agente

Clases de equivalencia inválidas

Clase	Condición	Resultado esperado
CEI01	DNI vacío	El sistema solicita completar el DNI
CEI02	CUIL vacío	El sistema solicita completar el CUIL
CEI03	Apellido o nombre vacío	El sistema solicita completar los datos obligatorios
CEI04	DNI duplicado	El sistema rechaza el registro
CEI05	CUIL duplicado	El sistema rechaza el registro
CEI06	DNI con formato inválido	El sistema rechaza el dato ingresado
CEI07	Búsqueda de agente inexistente	El sistema informa que no se encontró el agente

12.6.2 - PARTICIÓN DE EQUIVALENCIA PARA CU04 - GESTIONAR LICENCIAS

Para el CU04 - Gestionar Licencias, se evalúa la validación de fechas y la existencia previa del agente.

La regla principal consiste en que la fecha de finalización de la licencia debe ser igual o posterior a la fecha de inicio, y que la licencia debe asociarse a un agente existente.

Clases de equivalencia válidas

Clase	Condición	Resultado esperado
CEV01	Fecha fin igual a fecha inicio	El sistema permite registrar una licencia de un día
CEV02	Fecha fin posterior a fecha inicio	El sistema permite registrar una licencia de varios días
CEV03	Agente existente con legajo asociado	El sistema permite continuar con la carga de licencia

Clases de equivalencias invalidas

Clase	Condición	Resultado esperado
CEI01	Fecha fin anterior a fecha inicio	El sistema rechaza la solicitud
CEI02	Fecha de inicio vacía	El sistema solicita completar la fecha de inicio

CEI03	Fecha de finalización vacía	El sistema solicita completar la fecha de finalización
CEI04	Agente inexistente	El sistema no permite registrar la licencia
CEI05	Tipo de licencia no seleccionado	El sistema solicita seleccionar un tipo de licencia

12.7 - ANÁLISIS DE VALORES DE FRONTERA

El análisis de valores de frontera permite verificar el comportamiento del sistema en los límites de aceptación de las reglas definidas. Para el CU02 se aplican valores de frontera sobre la longitud del DNI, mientras que para el CU04 se aplican sobre la relación entre fecha de inicio y fecha de finalización.

12.7.1 - ANÁLISIS DE VALORES DE FRONTERA PARA CU02 - GESTIONAR AGENTE

Para el CU02 - Gestionar Agente, se toma como referencia el DNI del agente. En el contexto del prototipo, se considera válido un DNI numérico de 8 dígitos.

Caso	Dato ingresado	Condición	Resultado esperado
VF01	2794999	DNI con 7 dígitos	Rechaza el registro
VF02	27949990	DNI con 8 dígitos	Acepta el registro
VF03	279499901	DNI con 9 dígitos	Rechaza el registro
VF04	vacío	DNI no informado	Rechaza el registro
VF05	27949990 duplicado	DNI ya existente	Rechaza el registro

12.7.2 - ANÁLISIS DE VALORES DE FRONTERA PARA CU04 - GESTIONAR LICENCIAS

Para el CU04 - Gestionar Licencias, se toma como fecha de inicio de referencia el día 20/05/2026. El límite principal se encuentra en la igualdad entre la fecha de inicio y la fecha de finalización.

Caso	Fecha inicio	Fecha fin	Condición	Resultado esperado
VF01	20/5/2026	19/5/2026	Fecha fin anterior	Rechaza la solicitud
VF02	20/5/2026	20/5/2026	Fecha fin igual	Acepta la solicitud
VF03	20/5/2026	21/5/2026	Fecha fin posterior	Acepta la solicitud
VF04	Vacía	21/5/2026	Falta fecha inicio	Rechaza la solicitud
VF05	20/5/2026	Vacía	Falta fecha fin	Rechaza la solicitud

12.8 - COMPORTAMIENTO ESPERADO DEL SISTEMA

Con base en las clases de equivalencia y los valores de frontera definidos, se establece el comportamiento esperado para cada caso de uso evaluado.

12.8.1 - COMPORTAMIENTO ESPERADO PARA CU02 - GESTIONAR AGENTE

Dato de entrada	Comportamiento esperado	Mensaje emitido por el sistema
DNI: 27949990, CUIL válido y datos completos	Registra agente	Agente registrado correctamente
DNI vacío	No registra agente	Debe ingresar DNI
CUIL vacío	No registra agente	Debe ingresar CUIL
Apellido o nombre vacío	No registra agente	Debe completar los datos obligatorios
DNI con 7 dígitos	No registra agente	El DNI ingresado no tiene un formato válido
DNI con 9 dígitos	No registra agente	El DNI ingresado no tiene un formato válido
DNI duplicado	No registra agente	Ya existe un agente registrado con ese DNI
Búsqueda de DNI inexistente	No muestra datos	No se encontró el agente solicitado

12.8.2 - COMPORTAMIENTO ESPERADO PARA CU04 - GESTIONAR LICENCIAS

Dato de entrada	Comportamiento esperado	Mensaje emitido por el sistema
Inicio: 20/05/2026 - Fin: 19/05/2026	No registra licencia	La fecha de finalización no puede ser anterior a la fecha de inicio
Inicio: 20/05/2026 - Fin: 20/05/2026	Registra licencia	Licencia registrada correctamente
Inicio: 20/05/2026 - Fin: 21/05/2026	Registra licencia	Licencia registrada correctamente
Fecha inicio vacía	No registra licencia	Debe ingresar fecha de inicio
Fecha fin vacía	No registra licencia	Debe ingresar fecha de finalización
Agente inexistente	No registra licencia	No se encontró el agente solicitado
Tipo de licencia no seleccionado	No registra licencia	Debe seleccionar un tipo de licencia

13 – INTERFAZ GRAFICA

La interfaz gráfica del prototipo fue desarrollada como parte de la implementación del Sistema de Gestión Integral para RRHH. Su objetivo es representar visualmente la interacción básica entre el usuario y los módulos definidos dentro del alcance operativo del AP2.

El prototipo no constituye la versión final del sistema, sino una representación funcional preliminar que permite validar la navegación, la carga de datos, las validaciones principales y la respuesta del sistema ante operaciones correctas o incorrectas

Pantalla de Inicio de sesión

La pantalla de inicio de sesión permite restringir el acceso al sistema mediante usuario y contraseña. Esta pantalla se vincula con el requerimiento funcional RQF01 - Autenticación y permite validar que solo usuarios autorizados puedan acceder al menú principal del prototipo.

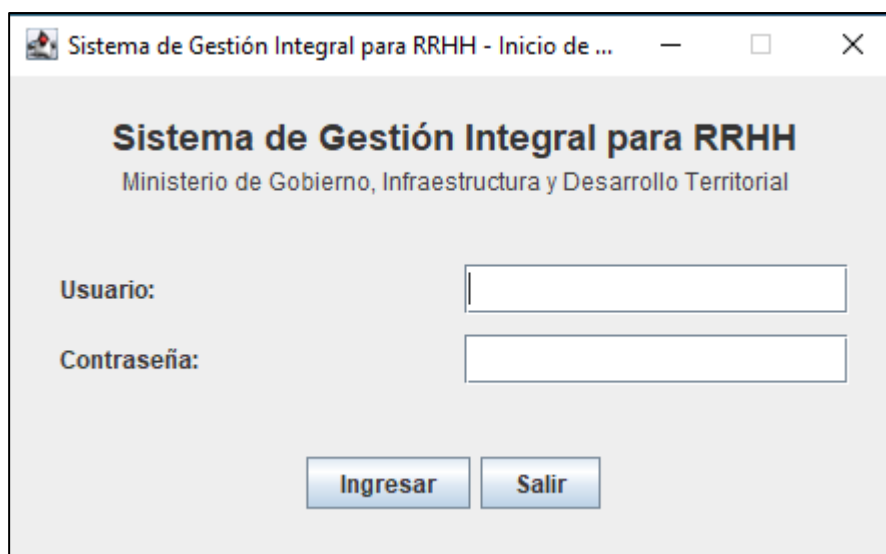
La imagen muestra una ventana de aplicación con el título "Sistema de Gestión Integral para RRHH - Inicio de ...". El contenido de la ventana tiene un fondo gris claro. En el centro, se muestra el título "Sistema de Gestión Integral para RRHH" en una fuente grande y bold, con el subtítulo "Ministerio de Gobierno, Infraestructura y Desarrollo Territorial" debajo. Hay dos campos de entrada de texto: uno etiquetado "Usuario:" y otro etiquetado "Contraseña:". Debajo de estos campos, hay dos botones: "Ingresar" y "Salir".

Imagen 12 – Pantalla de Login

Menú Principal

El menú principal permite acceder a los módulos disponibles dentro del prototipo. Desde esta pantalla, el usuario puede ingresar a Gestión de Agentes, Gestión de Legajos y Gestión de Licencias. Esta pantalla permite representar la organización general de la aplicación y facilita la navegación del usuario entre las funcionalidades principales.

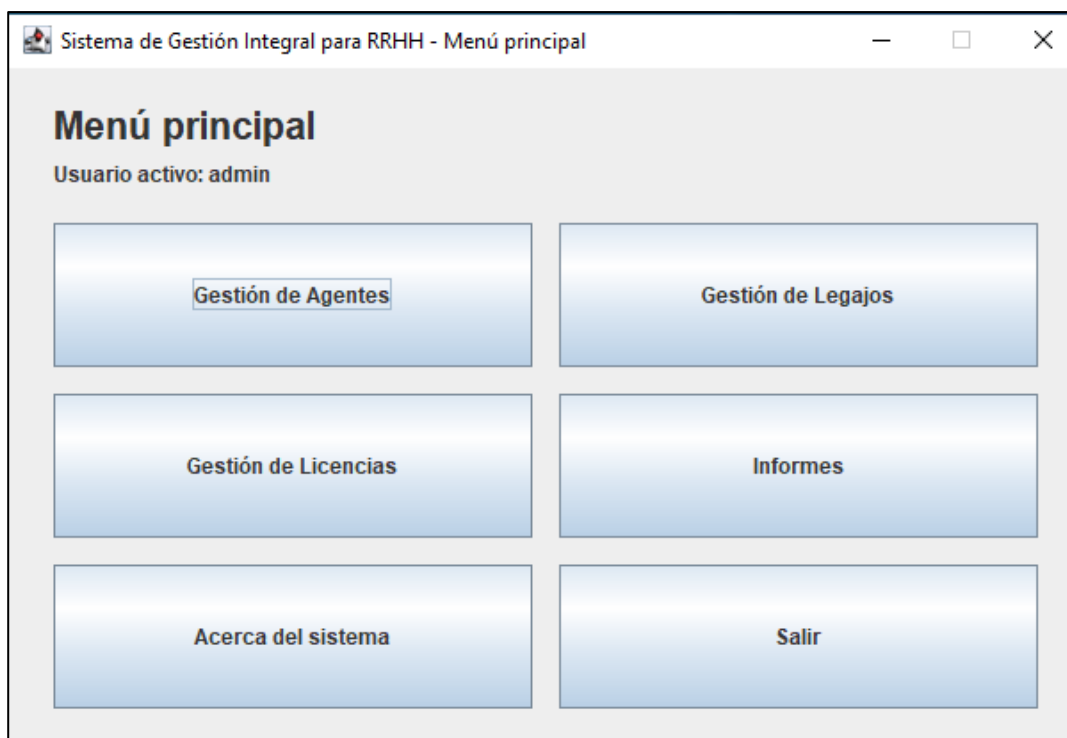


Imagen 13 – Pantalla Menú Principal

Pantalla de gestión de agentes

La pantalla de Gestión de Agentes permite registrar y consultar información básica de los agentes pertenecientes al Ministerio. Esta funcionalidad se vincula con el requerimiento funcional RQF02 - Gestión de agentes y con el caso de uso CU02 - Gestionar Agente.

DNI	CUIL	Apellido	Nombre	Cargo	Repartición	Activo

Imagen 14 – Pantalla Gestion Agentes

Gestión de Agentes

Datos del agente

DNI: 27949990

CUIL: 20279499906

Apellido: Quinteros

Nombre: Rodrigo

Cargo: Jefe División Informática

Repartición: Dirección General de Administración y Despacho

Estado: ☒ Activo

Operación exitosa
Agente registrado correctamente.
Aceptar

Agentes registrados

DNI	CUIL	Apellido	Nombre	Cargo	Repartición	Activo
27949990	20279499906	Quinteros	Rodrigo	Jefe División Infor...	Dirección General ...	Sí

Guardar agente Limpiar Volver

Imagen 15 – Registro de Agente Exitoso

Pantalla de gestión de licencias

La pantalla de Gestión de Licencias permite registrar solicitudes de licencia asociadas a un agente existente. Esta funcionalidad se vincula con el requerimiento funcional RQF05 - Gestión de licencias y con el caso de uso CU04 - Gestionar Licencias.

Gestión de Licencias

Datos de la licencia

DNI del agente:

Agente encontrado:

Tipo de licencia: Licencia Anual Reglamentaria

Fecha inicio (dd/MM/yyyy):

Fecha fin (dd/MM/yyyy):

Observaciones:

Licencias registradas

DNI	Apellido	Nombre	Tipo	Inicio	Fin	Días	Estado	Observaciones
-----	----------	--------	------	--------	-----	------	--------	---------------

Registrar licencia Limpiar Volver

Imagen 16 – Pantalla gestión licencias

La pantalla permite demostrar la integración entre los módulos de Gestión de Agentes, Gestión de Legajos y Gestión de Licencias, ya que toda licencia debe quedar asociada a un agente y reflejarse dentro de su información administrativa.

The screenshot shows the 'Gestión de Licencias' window. The 'Datos de la licencia' section contains the following information:

- DNI del agente: 27949990
- Agente encontrado: Quinteros, Rodrigo
- Tipo de licencia: Licencia Anual Reglamentaria
- Fecha inicio (dd/MM/yyyy): 18/05/2026
- Fecha fin (dd/MM/yyyy): 24/05/2026
- Observaciones: Licencia Ordinaria

Below this, a table titled 'Licencias registradas' displays the following data:

DNI	Apellido	Nombre	Tipo	Inicio	Fin	Días	Estado	Observaciones
27949990	Quinteros	Rodrigo	Licencia Anual ...	18/05/2026	24/05/2026	7	Registrada	Licencia Ordinar...

An 'Operación exitosa' dialog box is displayed in the center, stating: 'Licencia registrada correctamente para el agente Quinteros, Rodrigo.' with an 'Aceptar' button.

At the bottom of the window are three buttons: 'Registrar licencia', 'Limpiar', and 'Volver'.

Imagen 17 – Licencia registrada correctamente

El sistema valida que las fechas ingresadas sean consistentes, impidiendo registrar una licencia cuya fecha de finalización sea anterior a la fecha de inicio. Esta validación se encuentra relacionada con la partición de equivalencia y el análisis de valores de frontera definidos en la etapa de pruebas.

The screenshot shows the 'Gestión de Licencias' window with the following data:

- DNI del agente: 27949990
- Agente encontrado: Quinteros, Rodrigo
- Tipo de licencia: Licencia por Enfermedad
- Fecha inicio (dd/MM/yyyy): 18/05/2026
- Fecha fin (dd/MM/yyyy): 14/05/2026
- Observaciones: Gripe

The 'Licencias registradas' table is empty.

An error dialog box titled 'Fechas inválidas' is displayed, with a warning icon and the message: 'La fecha de finalización no puede ser anterior a la fecha de inicio.' with an 'Aceptar' button.

At the bottom of the window are three buttons: 'Registrar licencia', 'Limpiar', and 'Volver'.

Imagen 18 – Licencia rechazada por validación de fechas

13.1 – RELACIONES: INTERFAZ GRÁFICA, CASOS DE USO Y PRUEBAS

La pantalla de inicio de sesión se relaciona con el CU01 - Iniciar Sesión, permitiendo representar el acceso controlado al sistema.

La pantalla de Gestión de Agentes se relaciona con el CU02 - Gestionar Agente, ya que permite registrar y consultar agentes. Esta pantalla se vincula con las pruebas funcionales definidas para verificar la carga correcta de datos y el rechazo de datos inválidos.

La pantalla de Gestión de Licencias se relaciona con el CU04 - Gestionar Licencias, caso de uso principal seleccionado para el análisis. Esta pantalla permite registrar licencias, validar fechas y mostrar mensajes de confirmación o error.

Pantalla	Caso de uso relacionado	Requerimiento asociado	Caso de Prueba
Inicio de sesión	CU01 - Iniciar sesión	RQF01 - Autenticación	CPS01
Gestión de Agentes	CU02 - Gestionar Agente	RQF02 - Gestión de agentes	CPF02
Gestión de Licencias	CU04 - Gestionar Licencias	RQF05 - Gestión de licencias	CPF04 / CPV05
Validación de fechas	CU04 - Gestionar Licencias	RQF05 - Gestión de licencias	CPV05

14 - DEFINICIÓN DE BASE DE DATOS PARA EL SISTEMA

Para el desarrollo del prototipo se define una base de datos denominada: **rrhh_ministerio**. Esta base será utilizada para almacenar la información principal correspondiente a los módulos incluidos en el alcance del AP2: Gestión de Agentes, Gestión de Legajos y Gestión de Licencias.

La base de datos será implementada en MySQL, utilizando XAMPP como entorno de apoyo para el desarrollo y administración local. La aplicación Java Desktop se conectará a la base mediante JDBC, permitiendo registrar, consultar y validar la información utilizada por el prototipo.

Las tablas principales previstas para el prototipo son:

- roles
- usuarios
- agentes
- legajos
- tipos_licencia
- licencias
- documentos_legajo
- auditoria

14.1 - DIAGRAMA ENTIDAD-RELACIÓN

El siguiente diagrama entidad-relación representa la estructura de datos definida para el prototipo operacional del Sistema de Gestión Integral para RRHH.

El modelo se encuentra acotado a los módulos: **Gestión de Agentes**, **Gestión de Legajos y Gestión de Licencias**. También se incorporan las tablas necesarias para usuarios, roles, documentación asociada y auditoría básica.

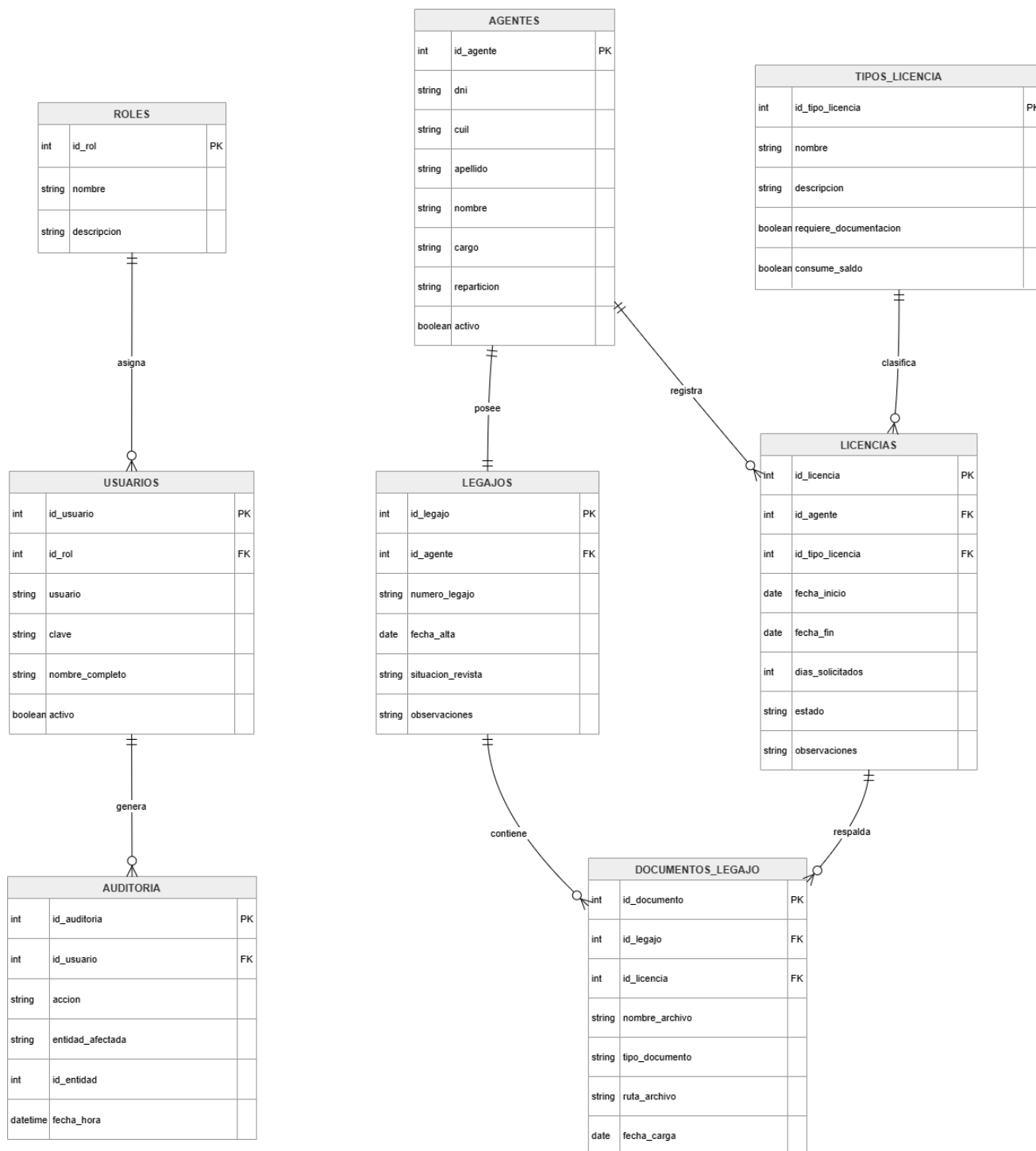


Imagen 19 – Diagrama de Entidad-Relación

El diagrama muestra las relaciones principales entre las tablas del prototipo. La tabla **agentes** se vincula directamente con **legajos** y **licencias**, ya que cada licencia registrada debe pertenecer a un agente existente y quedar asociada a su información administrativa.

La tabla **tipos_licencia** funciona como catálogo para clasificar las licencias disponibles. La tabla **documentos_legajo** permite registrar documentación vinculada al legajo y, cuando corresponde, a una licencia específica.

Las tablas **usuarios** y **roles** permiten representar el control básico de acceso al sistema, mientras que **auditoria** permite conservar trazabilidad sobre las acciones realizadas por los usuarios.

14.2 - CREACIÓN DE TABLAS MYSQL

A partir del diagrama entidad-relación definido, se procede a crear la base de datos y las tablas principales del prototipo. La base de datos se denomina `rrhh_ministerio`.

Las tablas creadas representan las entidades principales del sistema: roles, usuarios, agentes, legajos, tipos de licencia, licencias, documentos asociados al legajo y auditoría.

Para mantener la integridad de los datos se utilizan claves primarias, claves foráneas y restricciones de unicidad en campos identificatorios como usuario, DNI, CUIL y número de legajo.

ACLARACIÓN IMPORTANTE: TODAS LAS VISTAS, EJECUCIONES Y CAPTURAS DE MYSQL, HAN SIDO REALIZADAS CON VISUAL STUDIO CODE.

PARA NO EXTENDER ESTA SECCION NO SE MOSTRARÁN TODAS LAS CAPTURAS DE LA BASE DE DATOS

Creación tablas:

```
CREATE TABLE agentes (  
  id_agente INT AUTO_INCREMENT PRIMARY KEY,  
  dni VARCHAR(8) NOT NULL UNIQUE,  
  cuil VARCHAR(20) NOT NULL UNIQUE,  
  apellido VARCHAR(80) NOT NULL,  
  nombre VARCHAR(80) NOT NULL,  
  cargo VARCHAR(100),  
  reparticion VARCHAR(120),  
  activo BOOLEAN NOT NULL DEFAULT TRUE  
);
```

Imagen 20 – Creación tabla "agentes"

```
CREATE TABLE legajos (
    id_legajo INT AUTO_INCREMENT PRIMARY KEY,
    id_agente INT NOT NULL UNIQUE,
    numero_legajo VARCHAR(30) NOT NULL UNIQUE,
    fecha_alta DATE NOT NULL,
    situacion_revista VARCHAR(100),
    observaciones TEXT,
    CONSTRAINT fk_legajos_agentes
    FOREIGN KEY (id_agente) REFERENCES agentes(id_agente)
);
```

Imagen 21 – Creación tabla “legajos”

```
CREATE TABLE licencias (
    id_licencia INT AUTO_INCREMENT PRIMARY KEY,
    id_agente INT NOT NULL,
    id_tipo_licencia INT NOT NULL,
    fecha_inicio DATE NOT NULL,
    fecha_fin DATE NOT NULL,
    dias_solicitados INT NOT NULL,
    estado VARCHAR(30) NOT NULL,
    observaciones TEXT,
    CONSTRAINT fk_licencias_agentes
    FOREIGN KEY (id_agente) REFERENCES agentes(id_agente),
    CONSTRAINT fk_licencias_tipos
    FOREIGN KEY (id_tipo_licencia) REFERENCES tipos_licencia(id_tipo_licencia)
);
```

Imagen 22 – Creación tabla “licencias”

```
CREATE TABLE usuarios (
    id_usuario INT AUTO_INCREMENT PRIMARY KEY,
    id_rol INT NOT NULL,
    usuario VARCHAR(50) NOT NULL UNIQUE,
    clave VARCHAR(100) NOT NULL,
    nombre_completo VARCHAR(100) NOT NULL,
    activo BOOLEAN NOT NULL DEFAULT TRUE,
    CONSTRAINT fk_usuarios_roles
    FOREIGN KEY (id_rol) REFERENCES roles(id_rol)
);
```

Imagen 23 – Creación tabla “usuarios”

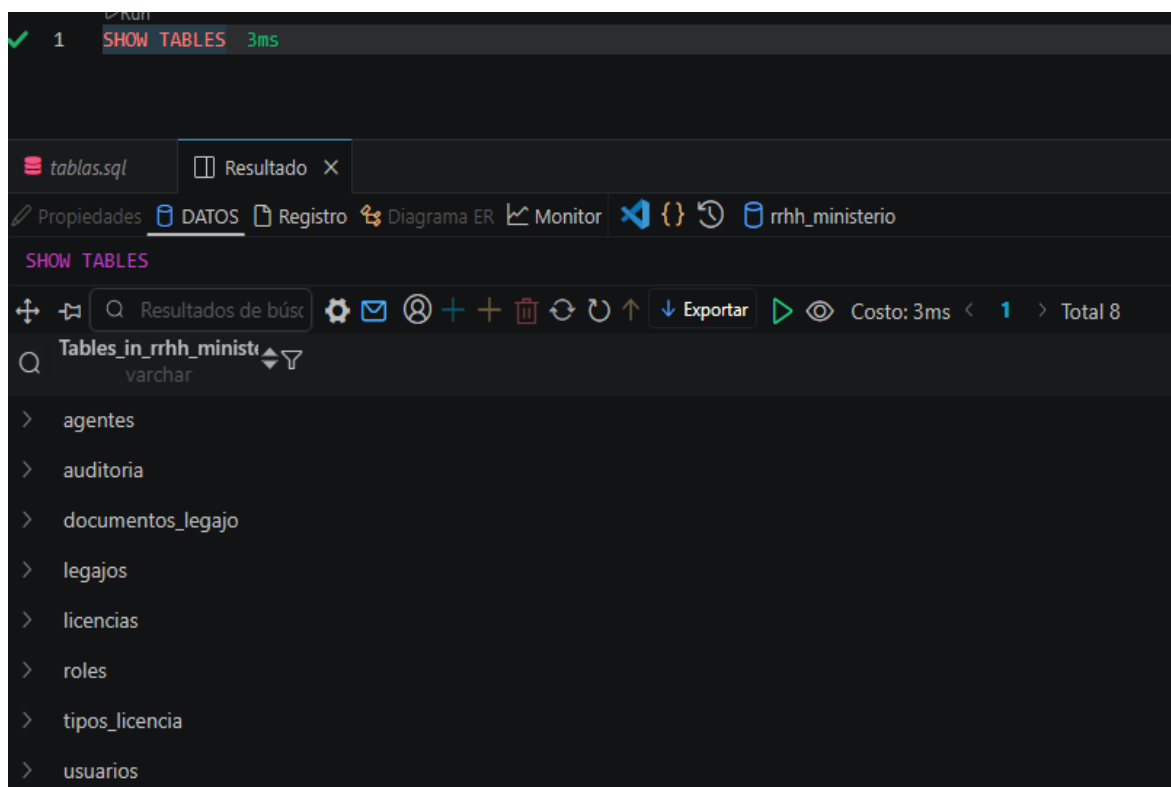


Imagen 24 – Vista de Visual Studio Code mostrando tablas creadas

14.3 - INSERCIÓN, CONSULTA Y BORRADO DE REGISTROS

Una vez creadas las tablas principales de la base de datos **rrhh_ministerio**, se procede a realizar operaciones básicas de inserción, consulta y borrado de registros.

Para esta prueba se cargan registros correspondientes a **roles**, **usuarios**, **agentes**, **legajos**, **tipos_licencia** y **licencias**. Luego se ejecutan consultas SQL para verificar los datos cargados y se realiza un borrado controlado sobre un registro de prueba.

14.3.1 – INSERCIÓN DE DATOS DE PRUEBA

En primer lugar, se insertan registros de prueba en las tablas principales. Estos datos permiten simular el funcionamiento básico del prototipo y verificar la relación entre usuarios, agentes, legajos y licencias.

1 `SELECT * FROM agentes;` 11ms

Propiedades DATOS Registro Diagrama ER Monitor rrhh_ministerio

`SELECT * FROM agentes LIMIT 100`

Resultados de búsqueda Costo: 18ms < 1 > Total 3

	<code>id_agente</code> int(11)	<code>* dni</code> varchar(8)	<code>* cuil</code> varchar(20)	<code>* apellido</code> varchar(80)	<code>* nombre</code> varchar(80)	<code>cargo</code> varchar(100)	<code>reparticion</code> varchar(120)	<code>* activo</code> tinyint(1)
> 1		27949990	20-27949990-6	Quinteros	Rodrigo	Administrativo	Oficina de Personal	1
> 2		30111222	20-30111222-3	Gonzalez	Mariana	Analista Administrativo	Dirección de Recursos Hum	1
> 3		28666777	20-28666777-5	Pereyra	Carlos	Auxiliar Administrativo	Subsecretaría Administrativ	1

Imagen 25 – Vista tabla “agentes”

1 `SELECT * FROM legajos;` 4ms

Propiedades DATOS Registro Diagrama ER Monitor rrhh_ministerio

`SELECT * FROM legajos LIMIT 100`

Resultados de búsqueda Costo: 8ms < 1 > Total 3

	<code>id_legajo</code> int(11)	<code>* id_agente</code> int(11)	<code>* numero_legajo</code> varchar(30)	<code>* fecha_alta</code> date	<code>situacion_revista</code> varchar(100)	<code>observaciones</code> text
> 1		1	LEG-0001	2020-03-10	Planta permanente	Legajo inicial del agente
> 2		2	LEG-0002	2021-07-15	Contratado	Legajo administrativo activo
> 3		3	LEG-0003	2019-11-05	Planta permanente	Sin observaciones

Imagen 26 – Vista tabla “legajos”

1 `SELECT * FROM licencias` 5ms

Propiedades DATOS Registro Diagrama ER Monitor rrhh_ministerio

`SELECT * FROM licencias LIMIT 100`

Resultados de búsqueda Costo: 9ms < 1 > Total 3

	<code>id_licenc</code> int(11)	<code>* id_agen</code> int(11)	<code>* id_tipo_licencia</code> int(11)	<code>* fecha_inicio</code> date	<code>* fecha_fin</code> date	<code>* dias_solicitados</code> int(11)	<code>* estado</code> varchar(30)	<code>observaciones</code> text
> 1		1	1	2026-05-20	2026-05-21	2	Registrada	Prueba de licencia anual reglamentaria
> 2		2	2	2026-06-03	2026-06-05	3	Registrada	Licencia por enfermedad con documentación pendiente
> 3		3	3	2026-08-10	2026-08-10	1	Registrada	Licencia por examen académico

Imagen 27 – Vista tabla “licencias”

14.3.2 - CONSULTA DE DATOS DE PRUEBA

Luego de insertar los registros, se ejecutó una consulta SQL para verificar que la información se encuentre correctamente relacionada entre las tablas **agentes**, **licencias** y **tipos_licencia**.

La consulta permite visualizar las licencias registradas junto con el DNI, apellido y nombre del agente, el tipo de licencia, las fechas correspondientes, la cantidad de días solicitados y el estado de la licencia.

```

1  SELECT
2      l.id_licencia,
3      a.dni,
4      a.apellido,
5      a.nombre,
6      tl.nombre AS tipo_licencia,
7      l.fecha_inicio,
8      l.fecha_fin,
9      l.dias_solicitados,
10     l.estado
11 FROM licencias l
12 INNER JOIN agentes a
13     ON l.id_agente = a.id_agente
14 INNER JOIN tipos_licencia tl
15     ON l.id_tipo_licencia = tl.id_tipo_licencia; 7ms
  
```

id_licencia	dni	apellido	nombre	tipo_licencia	fecha_inicio	fecha_fin	dias_solicitados	estado
1	27949990	Quinteros	Rodrigo	Licencia Anual Reglamentar	2026-05-20	2026-05-21	2	Registrada
2	30111222	Gonzalez	Mariana	Licencia por Enfermedad	2026-06-03	2026-06-05	3	Registrada
3	28666777	Pereyra	Carlos	Licencia por Estudio	2026-08-10	2026-08-10	1	Registrada

Imagen 28 – Consulta de licencias registradas asociadas a agentes y tipos de licencia

```

1  SELECT
2      l.id_licencia,
3      a.dni,
4      a.apellido,
5      a.nombre,
6      tl.nombre AS tipo_licencia,
7      l.fecha_inicio,
8      l.fecha_fin,
9      l.dias_solicitados,
10     l.estado
11 FROM licencias l
12 INNER JOIN agentes a
13     ON l.id_agente = a.id_agente
14 INNER JOIN tipos_licencia tl
15     ON l.id_tipo_licencia = tl.id_tipo_licencia
16 WHERE a.dni = '27949990'; 8ms
  
```

id_licencia	dni	apellido	nombre	tipo_licencia	fecha_inicio	fecha_fin	dias_solicitados	estado
1	27949990	Quinteros	Rodrigo	Licencia Anual Reglamentar	2026-05-20	2026-05-21	2	Registrada

Imagen 29 – Consulta de licencia registrada, filtrando por DNI

14.3.3 – BORRADO DE REGISTROS

Para verificar el borrado de registros se insertó previamente una licencia temporal. Luego se realizó una consulta de verificación para ver el registro de prueba.

Se ejecutó una sentencia DELETE sobre dicho registro, evitando afectar los datos principales utilizados para las pruebas del prototipo. Posteriormente, se realizó una consulta de verificación para comprobar que el registro temporal ya no se encontraba en la tabla licencias.

Esta operación permite demostrar que la base de datos admite operaciones básicas de mantenimiento sobre los registros almacenados. Para no borrar datos importantes, se insertó primero una licencia temporal.

```

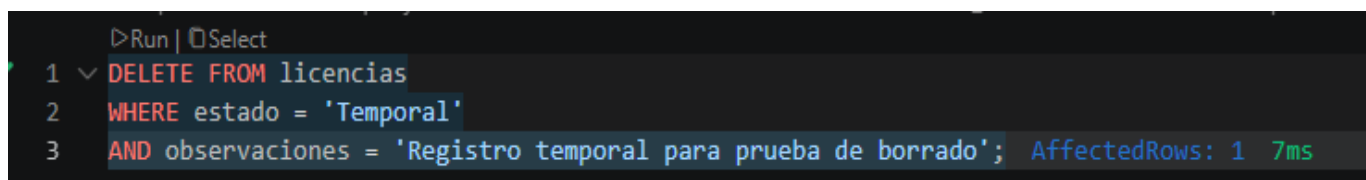
>Run | Select
1  INSERT INTO licencias (
2      id_agente,
3      id_tipo_licencia,
4      fecha_inicio,
5      fecha_fin,
6      dias_solicitados,
7      estado,
8      observaciones
9  ) VALUES (
10     1,
11     4,
12     '2026-09-01',
13     '2026-09-01',
14     1,
15     'Temporal',
16     'Registro temporal para prueba de borrado'
17 ); 10ms AffectedRows: 1
    
```

Imagen 30 – Inserción de registro de licencia temporal

The screenshot shows a SQL query result in a database client. The query is a SELECT statement filtering for 'Temporal' licenses. The result table has 8 columns: id_licencia, id_agente, id_tipo_licencia, fecha_inicio, fecha_fin, dias_solicitados, estado, and observaciones. A single row is displayed with the following values: 8, 1, 4, 2026-09-01, 2026-09-01, 1, Temporal, and Registro temporal para prueba de borrado.

id_licencia	id_agente	id_tipo_licencia	fecha_inicio	fecha_fin	dias_solicitados	estado	observaciones
8	1	4	2026-09-01	2026-09-01	1	Temporal	Registro temporal para prueba de borrado

Imagen 31 – Consulta de Registro temporal insertado



```
1 DELETE FROM licencias
2 WHERE estado = 'Temporal'
3 AND observaciones = 'Registro temporal para prueba de borrado'; AffectedRows: 1 7ms
```

Imagen 32 – Borrado de registro temporal insertado

Las operaciones realizadas permiten verificar el funcionamiento básico de la base de datos **rrhh_ministerio**. La inserción de registros permite cargar datos de prueba, las consultas permiten comprobar la relación entre agentes, legajos y licencias, y el borrado controlado permite validar la eliminación de registros temporales.

El conjunto completo de sentencias SQL utilizadas se conserva en el archivo **rrhh_ministerio.sql**, que forma parte del respaldo técnico del prototipo.

14.4 - PRESENTACIÓN DE CONSULTAS SQL

En este apartado se presentan las consultas SQL utilizadas para verificar el funcionamiento básico de la base de datos **rrhh_ministerio**.

Para no sobrecargar el informe, se incluyen las consultas más representativas. El conjunto completo de sentencias utilizadas se conserva en el archivo **rrhh_ministerio.sql** como respaldo técnico del prototipo.

Crear tabla roles

```
CREATE TABLE roles (  
id_rol INT AUTO_INCREMENT PRIMARY KEY,  
nombre VARCHAR(50) NOT NULL,  
descripcion VARCHAR(150)  
);
```

Insertar datos en tabla agentes

```
INSERT INTO agentes (  
dni,  
cuil,  
apellido,  
nombre,  
cargo,  
reparticion,  
activo
```

```
) VALUES  
( '27949990', '20-27949990-1', 'Quinteros', 'Rodrigo', 'Administrativo', 'Oficina de Personal', TRUE),  
( '30111222', '20-30111222-3', 'Gonzalez', 'Mariana', 'Analista Administrativo', 'Dirección de Recursos Humanos', TRUE),  
( '28666777', '20-28666777-5', 'Pereyra', 'Carlos', 'Auxiliar Administrativo', 'Subsecretaría Administrativa', TRUE);
```

Consulta de agentes registrados

```
SELECT  
id_agente,  
dni,  
cuil,  
apellido,  
nombre,  
cargo,  
reparticion,  
activo  
FROM agentes;
```

Insertar registro temporal para pruebas

```
INSERT INTO licencias (  
    id_agente,  
    id_tipo_licencia,  
    fecha_inicio,  
    fecha_fin,  
    dias_solicitados,  
    estado,  
    observaciones  
) VALUES  
(1, 4, '2026-09-01', '2026-09-01', 1, 'Temporal', 'Registro temporal para prueba de borrado');
```

Borrado de registro temporal para pruebas

```
DELETE FROM licencias  
  
WHERE estado = 'Temporal'  
  
AND observaciones = 'Registro temporal para prueba de borrado';
```

15 - DEFINICIONES DE COMUNICACIÓN

Para el prototipo operacional del Sistema de Gestión Integral para RRHH se define una comunicación simple entre la aplicación Java Desktop y la base de datos MySQL.

La aplicación se ejecuta en una computadora de la Oficina de Personal o equipo de desarrollo y se conecta a la base de datos **rrhh_ministerio** mediante JDBC. Esta comunicación utiliza el protocolo TCP/IP y el puerto 3306, correspondiente al servicio MySQL.

En una futura implementación institucional, la base de datos podría alojarse en un servidor interno.

El prototipo no contempla comunicación con sistemas externos, portales web, servicios de mensajería, firma digital, sistemas CORE provinciales ni integración automática con expedientes electrónicos.

15.1 - COMPONENTES DE COMUNICACIÓN

Componente	Descripción
Aplicación Java Desktop	Interfaz utilizada por el personal autorizado de RRHH.
JDBC	Medio de conexión entre Java y MySQL.
MySQL / MariaDB	Motor de base de datos donde se almacena la información del prototipo.
XAMPP	Entorno utilizado para ejecutar MySQL durante el desarrollo.
TCP/IP	Protocolo utilizado para la comunicación entre la aplicación y la base de datos.
Puerto 3306	Puerto utilizado por el servicio MySQL.

15.2 – DIAGRAMA DE COMUNICACIÓN

El esquema representa la comunicación básica del prototipo. El usuario autorizado opera la aplicación Java Desktop desarrollada con Swing. La aplicación se comunica con la base de datos mediante JDBC, utilizando TCP/IP y el puerto 3306 correspondiente al servicio MySQL. La base de datos rrhh_ministerio almacena la información de usuarios, agentes, legajos, licencias y demás entidades principales del sistema.

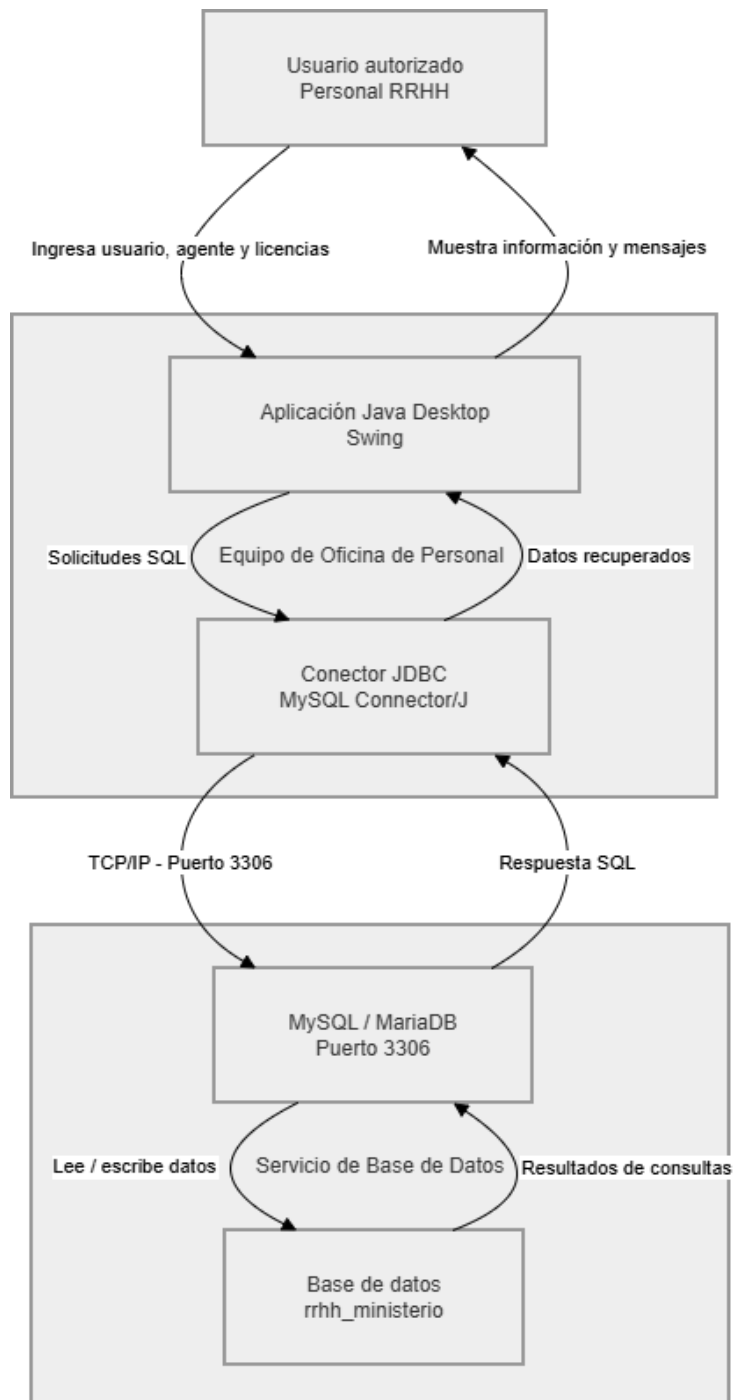


Imagen 33 – Diagrama de Comunicación

16 – EXPLICACIÓN DEL DESARROLLO EN JAVA

El desarrollo del prototipo se realizó utilizando el lenguaje de programación **Java**, con el objetivo de representar de forma funcional las operaciones principales del **Sistema de Gestión Integral para RRHH del Ministerio de Gobierno, Infraestructura y Desarrollo Territorial del Gobierno de Mendoza**.

El sistema fue planteado como una aplicación de escritorio, desarrollada con interfaz gráfica mediante **Java Swing**. Esta elección permite construir ventanas, formularios, botones, tablas y mensajes de interacción con el usuario sin incorporar frameworks externos complejos.

El desarrollo implementado permite validar el funcionamiento básico del sistema mediante:

- inicio de sesión;
- menú principal de navegación;
- carga de agentes;
- listado de agentes registrados;
- búsqueda de agente por DNI;
- validación de datos obligatorios;
- validación de formato de DNI;
- registro de licencias asociadas a agentes;
- validación de fechas;
- manejo de errores mediante mensajes;
- almacenamiento temporal en memoria.

Para facilitar la demostración de los conceptos de Programación Orientada a Objetos, se incorporó una clase abstracta denominada **Persona**. Esta clase permite representar los datos generales de una persona y sirve como base para la clase Agente. La incorporación **no modifica el alcance funcional del prototipo**, sino que mejora la estructura interna del código y permite evidenciar con mayor claridad los conceptos de **abstracción, herencia y polimorfismo**.

16.1 – ORGANIZACIÓN GENERAL DEL PROYECTO

El proyecto se encuentra organizado en paquetes, separando las responsabilidades principales del sistema. Esta organización permite diferenciar el punto de entrada de la aplicación, las clases del modelo, el almacenamiento temporal y las ventanas gráficas.

La estructura general del código fuente es la siguiente:

```
src
├── rrhh
│   ├── Main.java
│   ├── modelo
│   │   ├── Persona.java
│   │   └── Agente.java
│   ├── datos
│   │   └── RepositorioMemoria.java
│   └── vista
│       ├── LoginView.java
│       ├── MenuPrincipalView.java
│       ├── AgenteView.java
│       └── LicenciaView.java
```

Imagen 34 – Tree del proyecto java

La distribución por paquetes queda definida de esta manera:

Paquete	Contenido	Responsabilidad
rrhh	Main	Inicio de la aplicación.
rrhh.modelo	Persona, Agente	Representación de los datos principales del sistema.
rrhh.datos	RepositorioMemoria	Almacenamiento temporal y búsqueda de agentes.
rrhh.vista	LoginView, MenuPrincipalView, AgenteView, LicenciaView	Interfaz gráfica del prototipo.

Esta división permite que el prototipo sea más claro y mantenible. Las ventanas se encargan de la interacción con el usuario, las clases del modelo representan los datos y el repositorio concentra las operaciones de almacenamiento temporal.

La idea general ha sido que, con las distintas clases presentadas, se demuestren todos los aspectos solicitados de la programación orientada a objetos

16.2 – CLASES DEL PROYECTO

En cada clase se pondrá una captura de pantalla del código, en el caso de que la clase tenga muchas líneas de código, se harán capturas, cortas para mostrar parte del código.

16.2.1 – CLASE MAIN

Es la clase principal del sistema. Su función es iniciar la aplicación y abrir la ventana de inicio de sesión.

La clase **Main** cumple la función de iniciar el prototipo. Desde esta clase se crea la ventana **LoginView**, que representa el primer punto de interacción entre el usuario y el sistema. Se utiliza **SwingUtilities.invokeLater** para iniciar correctamente la interfaz gráfica en el hilo de eventos de Swing.

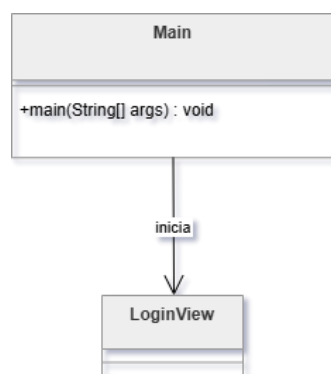


Imagen 35 – Diagrama de Clase Main

Atributos privados:

No posee atributos privados.

Constructor:

No posee constructor definido explícitamente. Java utiliza el constructor por defecto.

Métodos:

Método	Descripción
main(String[] args)	Punto de entrada del programa. Ejecuta la interfaz gráfica inicial mediante <code>SwingUtilities.invokeLater</code> .

```

J Main.java 1 x
J Main.java > Main
1  package rrhh;
2
3  import javax.swing.JOptionPane;
4  import javax.swing.SwingUtilities;
5  import rrhh.datos.ConexionBD;
6  import rrhh.vista.LoginView;
7
8  /**
9   * Clase principal del sistema.
10  *
11  * Responsabilidades:
12  * - Verificar la conexión con MariaDB.
13  * - Iniciar la interfaz gráfica.
14  * - Abrir la ventana de inicio de sesión.
15  */
16  public class Main {
17
18      /**
19       * Punto de entrada del programa.
20       *
21       * @param args argumentos recibidos por consola.
22       */
23      public static void main(String[] args) {
24
25          /**
26           * Antes de abrir el sistema se prueba la conexión con MariaDB.
27           * Si la base no está disponible, se informa al usuario y no se continúa.
28           */
29          if (!ConexionBD.probarConexion()) {
30              JOptionPane.showMessageDialog(
31                  parentComponent: null,
32                  "No se pudo conectar con la base de datos MariaDB.\n"
33                  + "Verifique que XAMPP/MySQL esté iniciado y que exista la base rrhh_ministerio.",
34                  title: "Error de conexión",
35                  JOptionPane.ERROR_MESSAGE
36              );
37              return;
38          }
39
40          /**
41           * La interfaz gráfica de Swing debe iniciarse en el hilo de eventos.
42           */
43          SwingUtilities.invokeLater(() -> {
44              LoginView login = new LoginView();
45              login.setVisible(b: true);
46          });
47      }
48  }

```

Imagen 36 – Clase Main

16.2.2 – CLASE PERSONA

La clase Persona se incorpora para evitar repetir atributos comunes y para representar una entidad general del dominio. Al declararse como clase abstracta, no se instancia directamente, sino que sirve como base para clases más específicas.

En el prototipo, Persona permite demostrar el uso de **abstracción**, porque concentra características generales, y también permite aplicar **herencia**, ya que la clase Agente hereda sus atributos y métodos comunes.

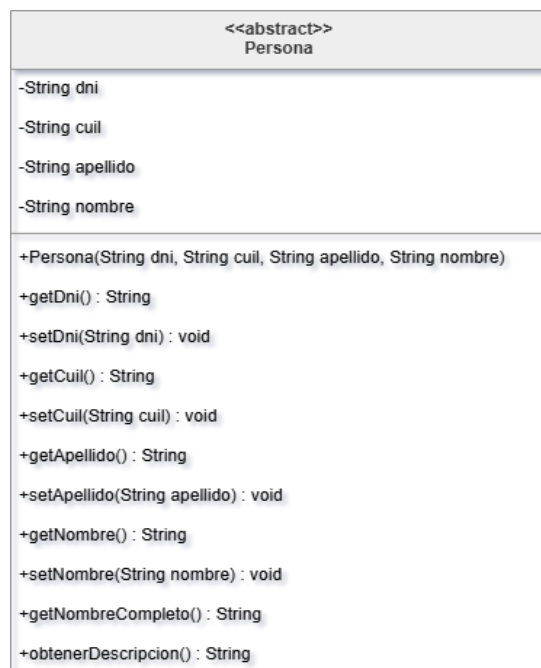


Imagen 37 –Diagrama de Clase Persona

Atributos privados:

private String dni;

private String cuil;

private String apellido;

private String nombre;

Constructor:

public Persona(String dni, String cuil, String apellido, String nombre)

El constructor inicializa los datos básicos de una persona.

Métodos:

Método	Descripción
getDni() / setDni(...)	Permiten consultar o modificar el DNI.
getCuil() / setCuil(...)	Permiten consultar o modificar el CUIL.

getApellido() / setApellido(...)	Permiten consultar o modificar el apellido.
getNombre() / setNombre(...)	Permiten consultar o modificar el nombre.
getNombreCompleto()	Devuelve apellido y nombre en un único texto.
mostrarDatos()	Método abstracto que deberá ser implementado por las clases hijas.

```

J Persona.java 1 X
modelo > J Persona.java > {} rrhh.modelo
1 package rrhh.modelo;
2
3 /**
4  * Clase abstracta que representa una persona dentro del sistema.
5  *
6  * Se utiliza como clase base para concentrar atributos comunes como DNI,
7  * CUIL, apellido y nombre. Permite aplicar herencia y abstracción dentro
8  * del prototipo.
9  */
10 public abstract class Persona {
11
12     private String dni;
13     private String cuil;
14     private String apellido;
15     private String nombre;
16
17     public Persona(String dni, String cuil, String apellido, String nombre) {
18         this.dni = dni;
19         this.cuil = cuil;
20         this.apellido = apellido;
21         this.nombre = nombre;
22     }
23
24     public String getDni() {
25         return dni;
26     }
27
28     public void setDni(String dni) {
29         this.dni = dni;
30     }
31
32     public String getCuil() {
33         return cuil;
34     }
35
36     public void setCuil(String cuil) {
37         this.cuil = cuil;
38     }
39
40     public String getApellido() {
41         return apellido;
42     }
43
44     public void setApellido(String apellido) {
45         this.apellido = apellido;
46     }
47
48     public String getNombre() {
49         return nombre;
50     }
51
52     public void setNombre(String nombre) {
53         this.nombre = nombre;
54     }
55
56     /**
57      * Método abstracto que obliga a las clases hijas a definir
58      * cómo se muestran sus datos principales.
59      */
60     public abstract String mostrarDatos();
61 }

```

Imagen 38 –clase Persona

16.2.3 – CLASE AGENTE

Representa al agente o empleado gestionado por el sistema.

La clase **Agente** aplica **herencia**, porque extiende la clase abstracta **Persona**. De esta forma, reutiliza los atributos generales de una persona y agrega datos específicos del contexto laboral, como cargo, repartición y estado activo.

También aplica **polimorfismo**, porque implementa el método obtenerDescripcion() definido en Persona. Esto permite que cada clase derivada pueda devolver una descripción propia según su tipo.

Además, mantiene **encapsulamiento**, ya que sus atributos son privados y se accede a ellos mediante métodos públicos.

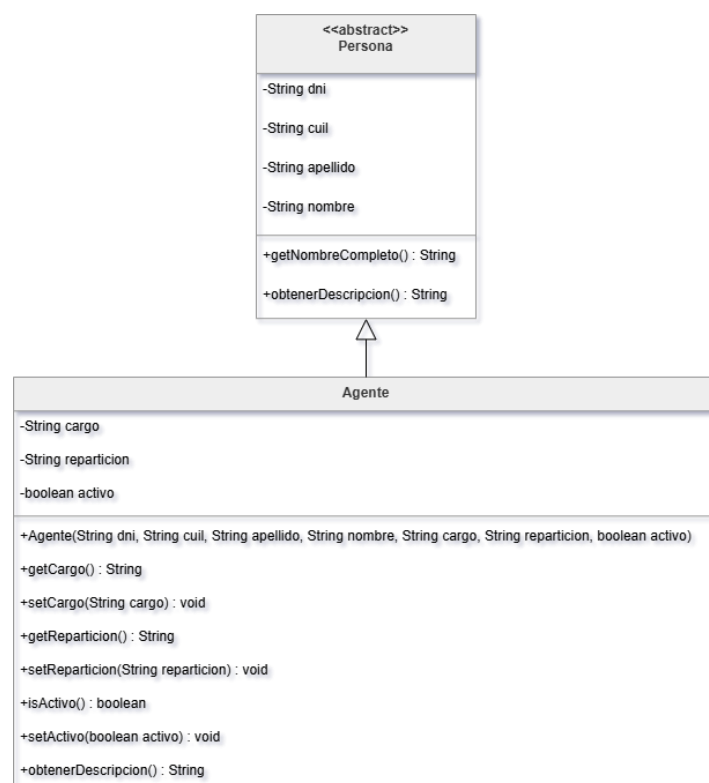


Imagen 39 – Diagrama de Clase Agente

Relación con Persona:

```
public class Agente extends Persona
```

Esto significa que Agente hereda los atributos y métodos comunes definidos en Persona, como DNI, CUIL, apellido y nombre.

Atributos privados propios:

```
private String cargo;
```

```
private String reparticion;
```

```
private boolean activo;
```

Los atributos dni, cuil, apellido y nombre ya no se declaran directamente en Agente, porque pertenecen a la clase base Persona.

Constructor:

```
public Agente(String dni, String cuil, String apellido, String nombre,  
              String cargo, String reparticion, boolean activo)
```

Métodos:

Método	Descripción
getCargo() / setCargo(...)	Permiten consultar o modificar el cargo.
getReparticion() / setReparticion(...)	Permiten consultar o modificar la repartición.
isActivo() / setActivo(...)	Permiten consultar o modificar el estado del agente.
obtenerDescripcion()	Implementa el método abstracto heredado de Persona.

```
J Agente.java 1 x
modelo > J Agente.java > {} rhh.modelo
1  package rhh.modelo;
2
3  /**
4   * Clase que representa a un agente o empleado gestionado por el sistema.
5   *
6   * Hereda de Persona los datos comunes: DNI, CUIL, apellido y nombre.
7   * Mantiene como atributos propios los datos laborales del agente.
8   */
9  public class Agente extends Persona {
10
11      private String cargo;
12      private String reparticion;
13      private boolean activo;
14
15      public Agente(String dni, String cuil, String apellido, String nombre, String cargo, String reparticion, boolean activo) {
16          super(dni, cuil, apellido, nombre);
17          this.cargo = cargo;
18          this.reparticion = reparticion;
19          this.activo = activo;
20      }
21
22      public String getCargo() {
23          return cargo;
24      }
25
26      public void setCargo(String cargo) {
27          this.cargo = cargo;
28      }
29
30      public String getReparticion() {
31          return reparticion;
32      }
33
34      public void setReparticion(String reparticion) {
35          this.reparticion = reparticion;
36      }
37
38      public boolean isActivo() {
39          return activo;
40      }
41
42      public void setActivo(boolean activo) {
43          this.activo = activo;
44      }
45
46      @Override
47      public String mostrarDatos() {
48          return getApellido() + ", " + getNombre()
49              + " | DNI: " + getDni()
50              + " | CUIL: " + getCuil()
51              + " | Cargo: " + cargo
52              + " | Repartición: " + reparticion
53              + " | Estado: " + (activo ? "Activo" : "Inactivo");
54      }
55
56      @Override
57      public String toString() {
58          return mostrarDatos();
59      }
60  }
```

Imagen 40 – Diagrama de clase Agente

16.2.4 – CLASE REPOSITORIOMEMORIA

Inicialmente su responsabilidad era mantener los agentes en una colección ArrayList durante la ejecución del programa. En la versión actual, su lógica fue adaptada para conectarse con MariaDB mediante JDBC, respetando la base de datos definida previamente en el proyecto.

A pesar de que la persistencia definitiva se realiza en MariaDB, la clase sigue utilizando ArrayList como estructura temporal de trabajo.

De esta forma, el prototipo demuestra tanto el uso de una base de datos persistente como el empleo de colecciones dinámicas en Java. Esta clase permite demostrar:

- uso de colecciones con ArrayList;
- estructuras repetitivas mediante recorridos for;
- búsqueda lineal por DNI;
- validación de duplicidad;
- ordenamiento de agentes por apellido y nombre.

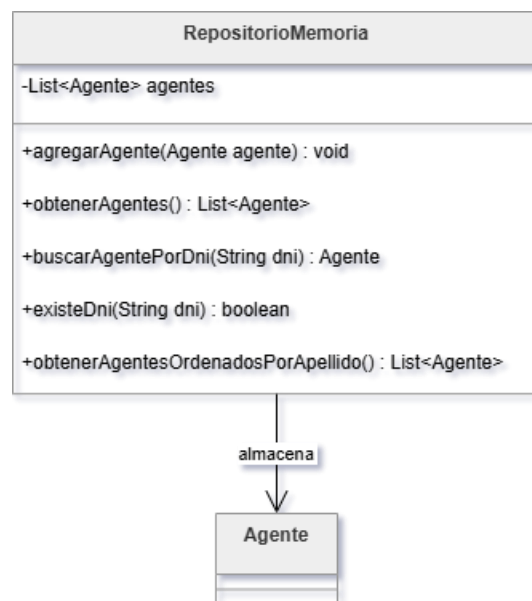


Imagen 41 – Diagrama de Clase RepositorioMemoria

Atributos privados:

```
private static final List<Agente> agentes = new ArrayList<>();
```

Constructor:

No posee constructor público relevante, ya que sus métodos y la lista de agentes son estáticos.

Métodos:

Método	Descripción
agregarAgente(Agente agente)	Agrega un agente a la lista en memoria.
obtenerAgentes()	Devuelve la lista de agentes registrados.
buscarAgentePorDni(String dni)	Busca un agente por DNI mediante recorrido de la lista.
existeDni(String dni)	Verifica si ya existe un agente registrado con el DNI indicado.
obtenerAgentesOrdenadosPorApellido()	Devuelve los agentes ordenados alfabéticamente por apellido y nombre.

```

J RepositorioMemoria.java 1 X
datos > J RepositorioMemoria.java > {} rrhh.datos
1  package rrhh.datos;
2
3  import java.sql.Connection;
4  import java.sql.Date;
5  import java.sql.PreparedStatement;
6  import java.sql.ResultSet;
7  import java.sql.SQLException;
8  import java.time.LocalDate;
9  import java.util.ArrayList;
10 import java.util.List;
11 import rrhh.modelo.Agente;
12
13 /**
14  * Repositorio de datos del sistema.
15  *
16  * El nombre RepositorioMemoria se conserva para respetar la estructura ya usada
17  * en el prototipo y evitar modificar innecesariamente las pantallas.
18  *
19  * A partir de esta versión, la clase ya no guarda datos en memoria mediante
20  * ArrayList, sino que trabaja contra MariaDB usando JDBC.
21  *
22  * Tablas usadas:
23  * - usuarios
24  * - agentes
25  * - tipos_licencia
26  * - licencias
27  */
28 public class RepositorioMemoria {
29
30     /**
31      * Constructor privado.
32      *
33      * La clase se utiliza de manera estática desde las ventanas del sistema.
34      */
35     private RepositorioMemoria() {
36     }
37
38     /**
39      * Valida usuario y contraseña contra la tabla usuarios.
40      *
41      * Respeta los usuarios cargados en el archivo SQL:
42      * - admin / admin123
43      * - rrhh / rrhh123
44      *
45      * @param usuario usuario ingresado.
46      * @param clave clave ingresada.
47      * @return true si el usuario existe, está activo y la clave coincide.
48      */
49     public static boolean validarUsuario(String usuario, String clave) {
50         String sql = ""
51             SELECT COUNT(*) AS total
52             FROM usuarios
53             WHERE usuario = ?
54             AND clave = ?
55             AND activo = TRUE
56             """;

```

Imagen 42 – Clase RepositorioMemoria

```
/**
 * Obtiene los agentes ordenados por apellido y nombre.
 *
 * Este método permite demostrar el uso de ordenamiento dentro del prototipo.
 *
 * @return lista de agentes ordenada alfabéticamente.
 */
public static List<Agente> obtenerAgentesOrdenadosPorApellido() {
    List<Agente> agentes = new ArrayList<>();

    String sql = """
        SELECT
            id_agente,
            dni,
            cuil,
            apellido,
            nombre,
            cargo,
            reparticion,
            activo
        FROM agentes
        ORDER BY apellido ASC, nombre ASC
        """;

    try (Connection conexion = ConexionBD.obtenerConexion();
        PreparedStatement ps = conexion.prepareStatement(sql);
        ResultSet rs = ps.executeQuery()) {
        while (rs.next()) {
            agentes.add(mapearAgente(rs));
        }
    } catch (SQLException e) {
        System.err.println("Error al obtener agentes ordenados: " + e.getMessage());
    }

    return agentes;
}
```

Imagen 43 – Clase RepositorioMemoria

16.2.5 – CLASE LOGINVIEW

La clase LoginView permite validar el ingreso al sistema. Utiliza estructuras condicionales para verificar si el usuario y la contraseña son correctos. Si la validación es exitosa, se crea una instancia de MenuPrincipalView; si falla, se muestra un mensaje de error mediante JOptionPane.

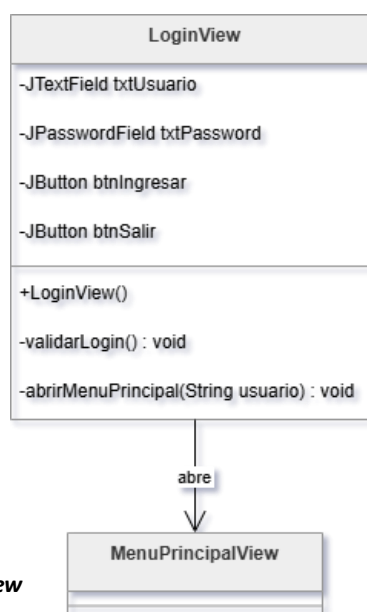


Imagen 44 – Diagrama de clase LoginView

Credenciales de prueba:

Usuario: admin / Contraseña: admin123

Atributos privados:

Contiene componentes gráficos como campos de texto, campo de contraseña y botones. Su función principal es capturar las credenciales ingresadas por el usuario.

Constructor:

```
public LoginView()
```

Inicializa la ventana de login y sus componentes gráficos.

Métodos principales:

Método	Descripción
Constructor LoginView()	Configura la pantalla de inicio de sesión.
Validación de usuario	Verifica usuario y contraseña.
Apertura de menú	Si las credenciales son correctas, abre MenuPrincipalView.



```

J LoginView.java 1 X
vista > J LoginView.java > LoginView > configurarVentana()
1 package rrhh.vista;
2
3 import java.awt.BorderLayout;
4 import java.awt.Color;
5 import java.awt.Font;
6 import java.awt.GridLayout;
7 import javax.swing.BorderFactory;
8 import javax.swing.JButton;
9 import javax.swing.JFrame;
10 import javax.swing.JLabel;
11 import javax.swing.JOptionPane;
12 import javax.swing.JPanel;
13 import javax.swing.JPasswordField;
14 import javax.swing.JTextField;
15 import javax.swing.SwingConstants;
16 import rrhh.datos.RepositorioMemoria;
17
18 /**
19  * Ventana de inicio de sesión.
20  *
21  * Esta pantalla valida el usuario contra la tabla usuarios de MariaDB.
22  */
23 public class LoginView extends JFrame {
24
25     private JTextField txtUsuario;
26     private JPasswordField txtClave;
27     private JButton btnIngresar;
28     private JButton btnSalir;
29
30     /**
31      * Constructor de la ventana de login.
32      */
33     public LoginView() {
34         configurarVentana();
35         inicializarComponentes();
36     }

```

Imagen 45 – Clase LoginView

16.2.6 – CLASE MENUPRINCIPALVIEW

La clase MenuPrincipalView cumple con el requisito de disponer de un **menú de selección**. Desde esta pantalla el usuario puede acceder a los módulos principales del prototipo. La navegación hacia otras ventanas se realiza mediante eventos asociados a botones.

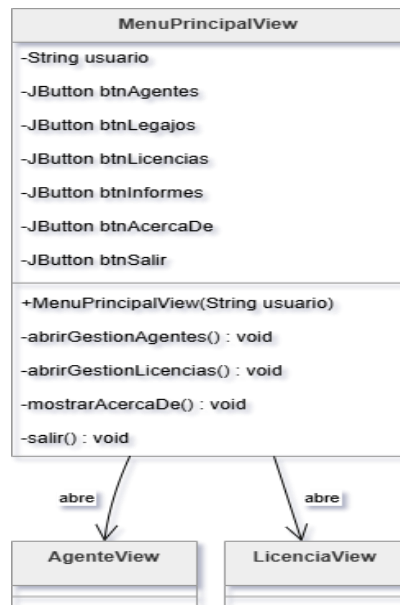


Imagen 46 – Diagrama de clase MenuPrincipalView

Atributos privados:

Contiene componentes gráficos asociados a las opciones del menú y mantiene el usuario activo recibido desde el login.

Constructor:

```
public MenuPrincipalView(String usuario)
```

Inicializa el menú principal y recibe el usuario autenticado.

Opciones del menú:

Gestión de Agentes | Gestión de Legajos | Gestión de Licencias | Informes | Acerca del sistema
Salir

Métodos principales:

Método	Descripción
Constructor MenuPrincipalView(String usuario)	Configura la pantalla principal.
Acceso a gestión de agentes	Abre la ventana AgentView.
Acceso a gestión de licencias	Abre la ventana LicenciaView.
Salir	Cierra la aplicación o la ventana activa.

```

J MenuPrincipalView.java X
vista > J MenuPrincipalView.java
1  package rrhh.vista;
2
3  import java.awt.BorderLayout;
4  import java.awt.Font;
5  import java.awt.GridLayout;
6  import javax.swing.BorderFactory;
7  import javax.swing.JButton;
8  import javax.swing.JFrame;
9  import javax.swing.JLabel;
10 import javax.swing.JOptionPane;
11 import javax.swing.JPanel;
12
13 /**
14  * Ventana principal del sistema.
15  *
16  * Presenta el menú de navegación del prototipo y permite acceder
17  * a los módulos implementados o previstos dentro del alcance del sistema.
18  */
19 public class MenuPrincipalView extends JFrame {
20
21     private String usuarioActual;
22
23     public MenuPrincipalView(String usuarioActual) {
24         this.usuarioActual = usuarioActual;
25         configurarVentana();
26         inicializarComponentes();
27     }
28
29     private void configurarVentana() {
30         setTitle("Sistema de Gestión Integral para RRHH - Menú principal");
31         setSize(620, 420);
32         setLocationRelativeTo(null);
33         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
34         setResizable(false);
35     }

```

Imagen 47 – Clase MenuPrincipalView - 1

```

71     /**
72      * Navegación hacia los módulos funcionales del prototipo.
73      * Se oculta el menú principal y se pasa su referencia para permitir el retorno.
74      */
75     btnAgentes.addActionListener(e -> {
76         AgenteView agenteView = new AgenteView(this);
77         agenteView.setVisible(b: true);
78         setVisible(b: false);
79     });
80
81     btnLicencias.addActionListener(e -> {
82         LicenciaView licenciaView = new LicenciaView(this);
83         licenciaView.setVisible(b: true);
84         setVisible(b: false);
85     });
86
87     /**
88      * Opciones visibles en el menú, pero reservadas para futuras iteraciones.
89      * Se mantienen para representar el alcance general del sistema declarado.
90      */
91     btnLegajos.addActionListener(e -> {
92         JOptionPane.showMessageDialog(
93             this,
94             message: "Módulo de legajos previsto para consulta del prototipo.",
95             title: "Gestión de Legajos",
96             JOptionPane.INFORMATION_MESSAGE
97         );
98     });

```

Imagen 48 – Clase MenuPrincipalView - 2

16.2.7 – CLASE AGENTEVUEW

La clase AgenteView permite demostrar la creación de objetos en Java. Cuando el usuario completa los datos y confirma el registro, el sistema crea un objeto Agente mediante su constructor y lo almacena temporalmente en RepositorioMemoria.

Como Agente hereda de Persona, al crear un agente también se inicializan los datos generales de la persona y los datos específicos del agente. Esta clase utiliza estructuras condicionales para validar campos obligatorios, formato de DNI y duplicidad de registros.



Imagen 49 – Diagrama de Clase AgentView

Atributos privados:

Contiene campos gráficos para ingresar: (cito algunos)

DNI

CUIL

Apellido

Nombre

También contiene una tabla para listar los agentes registrados.

Constructor:

```
public AgenteView()
```

Métodos principales:

Método	Descripción
Constructor AgenteView()	Configura la ventana y sus componentes.
Registrar agente	Toma los datos ingresados y crea un objeto Agente.
Validar campos	Controla que los datos obligatorios estén completos.
Validar DNI	Controla que el DNI tenga un formato válido.
Verificar duplicado	Consulta RepositorioMemoria.existeDni(...).
Actualizar tabla	Muestra los agentes registrados.
Listar ordenado	Puede mostrar los agentes ordenados por apellido y nombre.

```

J AgenteView.java 1 X
vista > J AgenteView.java > AgenteView > modeloTabla
1  package rrhh.vista;
2
3  import java.awt.BorderLayout;
4  import java.awt.GridLayout;
5  import javax.swing.BorderFactory;
6  import javax.swing.JButton;
7  import javax.swing.JCheckBox;
8  import javax.swing.JFrame;
9  import javax.swing.JLabel;
10 import javax.swing.JOptionPane;
11 import javax.swing.JPanel;
12 import javax.swing.JScrollPane;
13 import javax.swing.JTable;
14 import javax.swing.JTextField;
15 import javax.swing.table.DefaultTableModel;
16 import rrhh.datos.RepositorioMemoria;
17 import rrhh.modelo.Agente;
18
19 /**
20  * Ventana de gestión de agentes.
21  *
22  * Permite registrar agentes y consultar los agentes persistidos en MariaDB.
23  */
24 public class AgenteView extends JFrame {
25
26     private MenuPrincipalView menuPrincipal;
27
28     private JTextField txtDni;
29     private JTextField txtCuil;
30     private JTextField txtApellido;
31     private JTextField txtNombre;
32     private JTextField txtCargo;
33     private JTextField txtReparticion;
34     private JCheckBox chkActivo;
35
36     private JTable tablaAgentes;
37     private DefaultTableModel modeloTabla;
38
39     /**
40      * Constructor de la pantalla de gestión de agentes.
41      *
42      * @param menuPrincipal referencia al menú principal para poder volver.
43      */
44     public AgenteView(MenuPrincipalView menuPrincipal) {
45         this.menuPrincipal = menuPrincipal;
46
47         configurarVentana();
48         inicializarComponentes();
49         cargarTablaAgentes();
50     }

```

Imagen 50 – Clase AgenteView

16.2.8 – CLASE LICENCIAVIEW

La clase LicenciaView demuestra el uso de validaciones, búsqueda de objetos y manejo de excepciones. Antes de registrar una licencia, el sistema busca el agente por DNI. Si el agente existe, se validan las fechas ingresadas.

Si el formato de fecha es incorrecto, el error se controla mediante una excepción `DateTimeParseException`, evitando que el programa se cierre inesperadamente. El cálculo de días se realiza con `ChronoUnit.DAYS`.

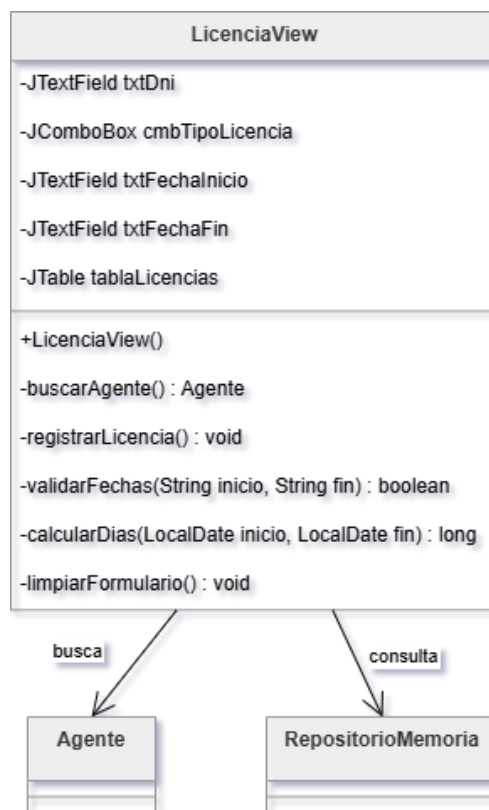


Imagen 51 – Diagrama de Clase LicenciaView

Atributos privados:

Contiene campos gráficos para:

- DNI del agente
- Tipo de licencia
- Fecha de inicio
- Fecha de fin
- Tabla de licencias registradas

Constructor:

```
public LicenciaView()
```

Tipos de licencia:

Licencia Anual Reglamentaria

Licencia por Enfermedad

Licencia por Estudio

Licencia por Razones Particulares

Manejo de excepciones:

La clase utiliza try/catch para controlar errores en el formato de fecha:

```
try {
    fechaInicio = LocalDate.parse(fechaInicioTexto, formatter);
    fechaFin = LocalDate.parse(fechaFinTexto, formatter);
} catch (DateTimeParseException ex) {
    JOptionPane.showMessageDialog(
        this,
        message: "Las fechas deben tener el formato dd/MM/yyyy.",
        title: "Formato de fecha inválido",
        JOptionPane.WARNING_MESSAGE
    );
}
return;
```

Imagen 52 – Manejo excepciones clase LicenciaView

Métodos principales:

Método	Descripción
Constructor LicenciaView()	Configura la pantalla de licencias.
Buscar agente por DNI	Consulta RepositorioMemoria.buscarAgentePorDni(dni).
Validar fechas	Verifica formato y coherencia de fechas.
Registrar licencia	Registra visualmente la licencia en una tabla.
Calcular días	Calcula la cantidad de días entre fecha inicio y fecha fin.

```
J LicenciaView.java 1 X
vista > J LicenciaView.java > LicenciaView > registrarLicencia()
1  package rrhh.vista;
2
3  import java.awt.BorderLayout;
4  import java.awt.GridLayout;
5  import java.time.LocalDate;
6  import java.time.format.DateTimeFormatter;
7  import java.time.format.DateTimeParseException;
8  import java.util.List;
9  import javax.swing.BorderFactory;
10 import javax.swing.JButton;
11 import javax.swing.JComboBox;
12 import javax.swing.JFrame;
13 import javax.swing.JLabel;
14 import javax.swing.JOptionPane;
15 import javax.swing.JPanel;
16 import javax.swing.JScrollPane;
17 import javax.swing.JTable;
18 import javax.swing.JTextField;
19 import javax.swing.table.DefaultTableModel;
20 import rrhh.datos.RepositorioMemoria;
21 import rrhh.modelo.Agente;
22
23 /**
24  * Ventana de gestión de licencias.
25  *
26  * Permite buscar un agente por DNI y registrar licencias en MariaDB,
27  * respetando la estructura aprobada:
28  *
29  * licencias.id_agente
30  * licencias.id_tipo_licencia
31  */
```

Imagen 53 – Clase LicenciaView - 1

```

155     /**
156      * Carga los tipos de licencia desde la tabla tipos_licencia.
157      */
158     private void cargarTiposLicencia() {
159         List<String> tipos = RepositorioMemoria.obtenerTiposLicencia();
160
161         cmbTipoLicencia.removeAllItems();
162
163         for (String tipo : tipos) {
164             cmbTipoLicencia.addItem(tipo);
165         }
166
167         /**
168          * Respaldo defensivo.
169          * Si por algún motivo la tabla tipos_licencia no devolviera datos,
170          * se cargan los tipos previstos en el SQL aprobado.
171          */
172         if (cmbTipoLicencia.getItemCount() == 0) {
173             cmbTipoLicencia.addItem(item: "Licencia Anual Reglamentaria");
174             cmbTipoLicencia.addItem(item: "Licencia por Enfermedad");
175             cmbTipoLicencia.addItem(item: "Licencia por Estudio");
176             cmbTipoLicencia.addItem(item: "Licencia por Razones Particulares");
177         }
178     }
179
180     /**
181      * Busca un agente por DNI y muestra apellido y nombre.
182      */
183     private void buscarAgente() {
184         String dni = txtDniAgente.getText().trim();
185
186         if (dni.isEmpty()) {
187             JOptionPane.showMessageDialog(
188                 this,
189                 message: "Debe ingresar el DNI del agente.",
190                 title: "Dato obligatorio",
191                 JOptionPane.WARNING_MESSAGE
192             );
193             return;
194         }

```

Imagen 54 – Clase LicenciaView - 2

16.3 - CONCEPTOS DE PROGRAMACIÓN ORIENTADA A OBJETOS APLICADOS

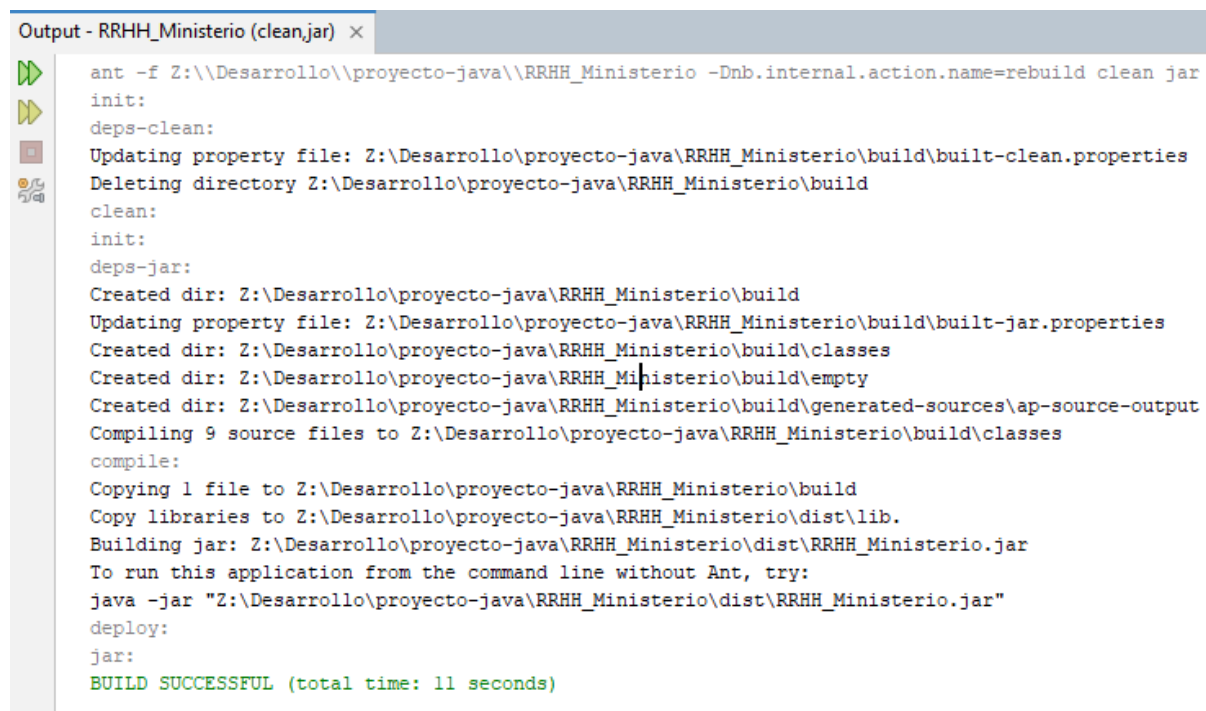
Concepto	Aplicación en el proyecto
Encapsulamiento	Atributos privados en Persona y Agente , accedidos mediante getters y setters.
Abstracción	Persona representa una entidad general y abstracta del dominio.
Herencia	Agente hereda de Persona .
Polimorfismo	Agente implementa el método obtenerDescripcion() definido en Persona .
Constructores	Persona , Agente y las ventanas inicializan sus objetos mediante constructores.
Creación de objetos	Se crean instancias de LoginView , MenuPrincipalView , AgenteView , LicenciaView y Agente .
Búsqueda	RepositorioMemoria.buscarAgentePorDni(...).
Ordenamiento	RepositorioMemoria.obtenerAgentesOrdenadosPorApellido().
Manejo de excepciones	Validación de fechas en LicenciaView con try/catch.

Menú de selección

Implementado en MenuPrincipalView.

17 – DEMOSTRACIÓN DE COMPILACIÓN Y EJECUCIÓN.

La siguiente captura muestra el resultado del proceso **Clean and Build** ejecutado desde **NetBeans**. El entorno compiló correctamente las clases Java del proyecto, y generó el archivo ejecutable **RRHH_Ministerio.jar** dentro de la carpeta **dist**. El mensaje **BUILD SUCCESSFUL** confirma que el proyecto compila sin errores.



```
Output - RRHH_Ministerio (clean.jar) x
ant -f Z:\Desarrollo\proyecto-java\RRHH_Ministerio -Dnb.internal.action.name=rebuild clean jar
init:
deps-clean:
Updating property file: Z:\Desarrollo\proyecto-java\RRHH_Ministerio\build\build-clean.properties
Deleting directory Z:\Desarrollo\proyecto-java\RRHH_Ministerio\build
clean:
init:
deps-jar:
Created dir: Z:\Desarrollo\proyecto-java\RRHH_Ministerio\build
Updating property file: Z:\Desarrollo\proyecto-java\RRHH_Ministerio\build\build-jar.properties
Created dir: Z:\Desarrollo\proyecto-java\RRHH_Ministerio\build\classes
Created dir: Z:\Desarrollo\proyecto-java\RRHH_Ministerio\build\empty
Created dir: Z:\Desarrollo\proyecto-java\RRHH_Ministerio\build\generated-sources\ap-source-output
Compiling 9 source files to Z:\Desarrollo\proyecto-java\RRHH_Ministerio\build\classes
compile:
Copying 1 file to Z:\Desarrollo\proyecto-java\RRHH_Ministerio\build
Copy libraries to Z:\Desarrollo\proyecto-java\RRHH_Ministerio\dist\lib.
Building jar: Z:\Desarrollo\proyecto-java\RRHH_Ministerio\dist\RRHH_Ministerio.jar
To run this application from the command line without Ant, try:
java -jar "Z:\Desarrollo\proyecto-java\RRHH_Ministerio\dist\RRHH_Ministerio.jar"
deploy:
jar:
BUILD SUCCESSFUL (total time: 11 seconds)
```

Imagen 55 – Compilación exitosa del proyecto Java

17.1 – CAPTURAS DE LA APLICACIÓN CORRIENDO



Imagen 56 – Captura ventana de Login – Fondo logo Netbeans



Imagen 57 – Captura Menú Principal

Gestión de Agentes

Datos del agente

DNI:

CUIL:

Apellido:

Nombre:

Cargo:

Repartición:

Estado: ☒ Activo

Agentes registrados

DNI	CUIL	Apellido	Nombre	Cargo	Repartición	Activo
27949990	20-27949990-1	Quinteros	Rodrigo	Administrativo	Oficina de Personal	Sí
30111222	20-30111222-3	Gonzalez	Mariana	Analista Administr...	Direcci??n de Rec...	Sí
28666777	20-28666777-5	Pereyra	Carlos	Auxiliar Administra...	Subsecretar??a A...	Sí
31454814	27314548146	Granau	Cecilia	Jefa	RRHH	Sí

Guardar agente Limpiar Listar ordenado Volver

Imagen 58 – Captura Gestión de Agentes

Gestión de Licencias

Datos de la licencia

DNI del agente: **Buscar agente**

Agente encontrado:

Tipo de licencia: **Licencia Anual Reglamentaria**

Fecha inicio (dd/MM/yyyy):

Fecha fin (dd/MM/yyyy):

Observaciones:

Licencias registradas

DNI	Apellido	Nombre	Tipo	Inicio	Fin	Días	Estado	Observaciones
31454814	Granau	Cecilia	Licencia por Raz...	2026-05-25	2026-05-30	6	Registrada	Licencia tranquila
28666777	Pereyra	Carlos	Licencia por Est...	2026-08-10	2026-08-10	1	Registrada	Licencia por exa...
30111222	Gonzalez	Mariana	Licencia por Enf...	2026-06-03	2026-06-05	3	Registrada	Licencia por enf...
27949990	Quinteros	Rodrigo	Licencia Anual ...	2026-05-20	2026-05-21	2	Registrada	Prueba de licen...

Registrar licencia **Limpiar** **Volver**

Imagen 59 – Captura Gestión de Licencia

Gestión de Licencias

Datos de la licencia

DNI del agente: **Buscar agente**

Agente encontrado:

Tipo de licencia: **Licencia por Enfermedad**

Fecha inicio (dd/MM/yyyy):

Fecha fin (dd/MM/yyyy):

Observaciones:

Licencias registradas

DNI	Apellido	Nombre	Tipo	Inicio	Fin	Días	Estado	Observaciones
27949990	Quinteros	Rodrigo	Licencia por Enf...	2026-05-26	2026-05-28	3	Registrada	Gripe
31454814	Granau	Cecilia	Licencia por Raz...	2026-05-25	2026-05-30	6	Registrada	Licencia tranquila
28666777	Pereyra	Carlos	Licencia por Est...	2026-08-10	2026-08-10	1	Registrada	Licencia por exa...
30111222	Gonzalez	Mariana	Licencia por Enf...	2026-06-03	2026-06-05	3	Registrada	Licencia por enf...
27949990	Quinteros	Rodrigo	Licencia Anual ...	2026-05-20	2026-05-21	2	Registrada	Prueba de licen...

Operación exitosa

Licencia registrada correctamente para el agente Quinteros, Rodrigo.

Aceptar

Registrar licencia **Limpiar** **Volver**

Imagen 60 – Captura Alta de Licencia Exitosa

Para demostrar el funcionamiento del prototipo se incorporaron capturas de las ventanas principales en ejecución y del proceso de compilación realizado desde NetBeans. Estas evidencias permiten verificar que el programa compila correctamente, que la interfaz gráfica se ejecuta y que las funcionalidades principales del prototipo pueden utilizarse desde el menú de selección.

En esta etapa se priorizó la implementación del prototipo Java y la aplicación de los conceptos solicitados: clases, objetos, constructores, encapsulamiento, herencia, abstracción, polimorfismo, estructuras condicionales y repetitivas, manejo de excepciones, búsqueda y menú de selección.

CAPÍTULO 18 - PERSISTENCIA DE DATOS MEDIANTE JDBC Y MARIADB

En ésta última etapa se incorporó un mecanismo de persistencia permanente que permitió reemplazar el almacenamiento temporal utilizado durante las etapas iniciales por una solución basada en una base de datos relacional. Esta evolución representa un paso importante en el desarrollo del sistema, ya que posibilita conservar la información registrada de forma consistente, organizada y disponible aun después de finalizar la ejecución de la aplicación.

Para implementar esta funcionalidad se utilizó la API **JDBC (Java Database Connectivity)** junto con el sistema gestor de bases de datos **MariaDB**, ejecutado localmente mediante el entorno XAMPP.

Más allá de incorporar persistencia permanente, el objetivo principal fue organizar el acceso a los datos mediante una arquitectura que separara claramente las responsabilidades de conexión, acceso a la información y presentación de la interfaz gráfica. Esta organización facilita el mantenimiento del sistema, favorece la reutilización del código y reduce el impacto que futuras modificaciones sobre la base de datos podrían producir en el resto de la aplicación.

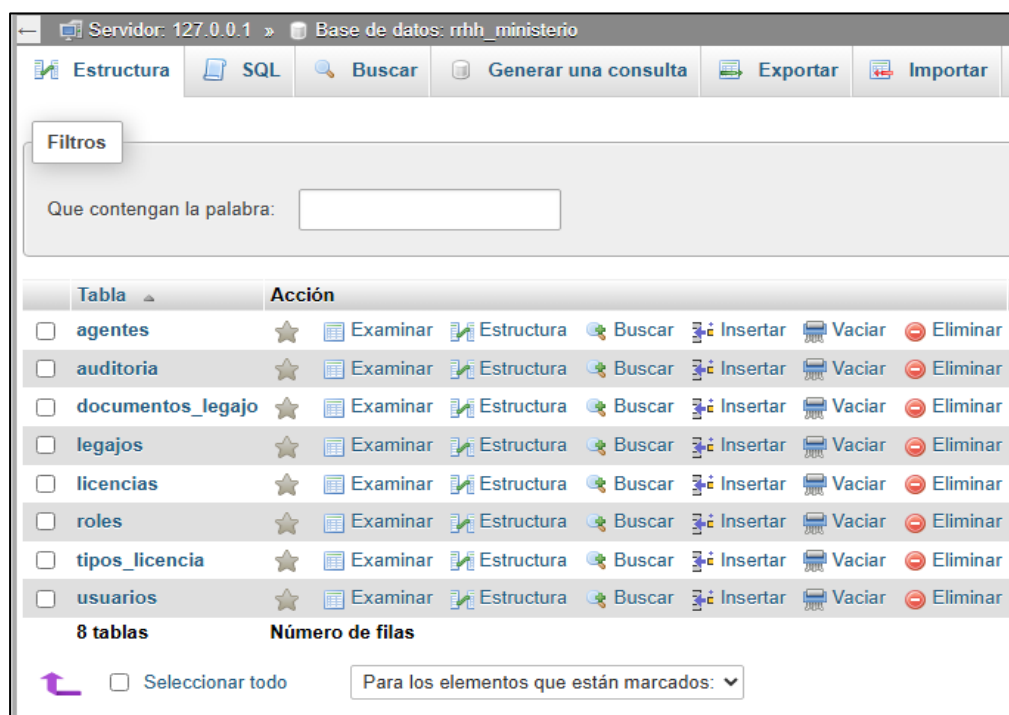


Imagen 61 – BD: rrhh_ministerio implementada en MariaDB y estructura de persistencia utilizada por el sistema

18.1 - ESTRATEGIA DE PERSISTENCIA Y ARQUITECTURA

La persistencia del sistema fue diseñada siguiendo una arquitectura en capas, donde las operaciones de acceso a la base de datos permanecen completamente desacopladas de la lógica de negocio y de las interfaces desarrolladas con Java Swing.

La administración de las conexiones se encuentra centralizada en la clase **ConexionBD**, responsable de establecer la comunicación con la base de datos **rrhh_ministerio** mediante JDBC. Concentrar esta responsabilidad en un único componente evita duplicar configuraciones dentro del sistema y simplifica futuras tareas de mantenimiento o modificación de la infraestructura de datos.

Por su parte, la capa de presentación únicamente interactúa con las clases del modelo y con el repositorio de datos, sin ejecutar directamente sentencias SQL. Esta separación permite que la interfaz permanezca enfocada exclusivamente en la interacción con el usuario, mientras que toda la lógica relacionada con la persistencia queda encapsulada en componentes específicos.

```
public class ConexionBD {

    private static final String URL = "jdbc:mariadb://localhost:3306/rrhh_ministerio";
    private static final String USUARIO = "root";
    private static final String CLAVE = "";

    /**
     * Constructor privado.
     *
     * La clase solo expone métodos estáticos y no necesita ser instanciada.
     */
    private ConexionBD() {
    }

    /**
     * Obtiene una conexión activa contra MariaDB.
     *
     * @return conexión JDBC hacia la base rrhh_ministerio.
     * @throws SQLException si ocurre un error de conexión.
     */
    public static Connection obtenerConexion() throws SQLException {
        return DriverManager.getConnection(URL, USUARIO, CLAVE);
    }

    /**
     * Verifica si la conexión con la base de datos se puede establecer.
     *
     * Este método se usa al iniciar el sistema para evitar abrir la interfaz
     * si MariaDB no está disponible.
     *
     * @return true si la conexión es correcta; false si ocurre un error.
     */
    public static boolean probarConexion() {
        try (Connection conexion = obtenerConexion()) {
            return conexion != null && !conexion.isClosed();
        } catch (SQLException e) {
            System.err.println("Error al conectar con MariaDB: " + e.getMessage());
            return false;
        }
    }
}
```

Imagen 62 – Extracto código: Clase ConexionBD: centralización de la conexión JDBC y verificación de disponibilidad de la base de datos

18.2 - CENTRALIZACIÓN DEL ACCESO A DATOS

Toda la interacción con MariaDB se encuentra concentrada en la clase **RepositorioMemoria**.

Si bien el nombre de esta clase se mantuvo para preservar la compatibilidad con las versiones desarrolladas durante las etapas anteriores del proyecto, su implementación actual dejó de utilizar almacenamiento temporal y pasó a desempeñar el rol de repositorio centralizado para el acceso a datos mediante JDBC.

Desde este componente se implementan las principales operaciones del sistema, entre ellas:

- Validación de usuarios.
- Registro y consulta de agentes.
- Búsqueda de agentes por DNI.
- Verificación de existencia de registros.
- Obtención de tipos de licencia.
- Registro de solicitudes de licencia.
- Consulta de licencias registradas.

```
public static boolean validarUsuario(...)
public static boolean agregarAgente(...)
public static List<Agente> obtenerAgentes()
public static List<Agente> obtenerAgentesOrdenadosPorApellido()
public static Agente buscarAgentePorDni(...)
public static boolean existeDni(...)
public static List<String> obtenerTiposLicencia()
public static boolean registrarLicencia(...)
public static List<Object[]> obtenerLicenciasParaTabla()
```

Imagen 63 – Métodos de acceso a datos implementados en la clase RepositorioMemoria

18.3 - GESTIÓN DINÁMICA DE LA INFORMACIÓN MEDIANTE ARRAYLIST

Se requería utilizar estructuras dinámicas de almacenamiento. Para satisfacer esto, el sistema emplea colecciones **ArrayList** para administrar los conjuntos de datos recuperados desde la base de datos.

Las principales operaciones que utilizan esta estructura corresponden a la obtención de agentes, tipos de licencia y consultas necesarias para alimentar las tablas y listas desplegables de la interfaz gráfica.

El uso de **ArrayList** permite almacenar una cantidad variable de registros sin necesidad de definir previamente un tamaño fijo, facilitando el procesamiento de la información recuperada y simplificando su posterior visualización.

La utilización de colecciones dinámicas aporta mayor flexibilidad al sistema y responde a uno de los conceptos de programación solicitados para esta etapa del proyecto.

```

124  public static List<Agente> obtenerAgentes() {
125      List<Agente> agentes = new ArrayList<>();
126
127      String sql = ""
128          SELECT
129              id_agente,
130              dni,
131              cuil,
132              apellido,
133              nombre,
134              cargo,
135              reparticion,
136              activo
137          FROM agentes
138          "";
139
140      try (Connection conexion = ConexionBD.obtenerConexion();
141          PreparedStatement ps = conexion.prepareStatement(sql);
142          ResultSet rs = ps.executeQuery()) {
143
144          while (rs.next()) {
145              agentes.add(mapearAgente(rs));
146          }
147
148      } catch (SQLException e) {
149          System.err.println("Error al obtener agentes: " + e.getMessage());
150      }
151
152      return agentes;
153  }
154

```

Imagen 64 – Extracto código RepositorioMemoria.java - Utilización de colecciones dinámicas ArrayList para almacenar registros recuperados desde MariaDB

18.4 - CONSULTAS RELACIONALES E INTEGRACIÓN CON LA INTERFAZ

Además de las operaciones básicas de inserción y consulta, el sistema implementa consultas relacionales utilizando la instrucción **INNER JOIN**, aprovechando las relaciones definidas durante el diseño de la base de datos.

Un ejemplo de ello se encuentra en la consulta utilizada para visualizar las licencias registradas, donde se integran en una única operación SQL los datos almacenados en las tablas **licencias**, **agentes** y

tipos_licencia. Este enfoque permite recuperar toda la información necesaria mediante una sola consulta, evitando consultas adicionales y manteniendo la coherencia entre los datos presentados al usuario.

Las ventanas **LoginView**, **AgenteView** y **LicenciaView** permanecen completamente desacopladas de la base de datos. Ninguna de ellas establece conexiones ni ejecuta instrucciones SQL directamente. Todas las operaciones son delegadas al repositorio, que actúa como intermediario entre la interfaz y la capa de persistencia.

Esta organización favorece la reutilización del código, simplifica el mantenimiento del sistema y mantiene claramente diferenciadas las responsabilidades de cada componente de la aplicación.

```

438  public static List<Object[]> obtenerLicenciasParaTabla() {
439
440      String sql = ""
441
442      SELECT
443          l.id_licencia,
444          a.dni,
445          a.apellido,
446          a.nombre,
447          tl.nombre AS tipo_licencia,
448          l.fecha_inicio,
449          l.fecha_fin,
450          l.dias_solicitados,
451          l.estado,
452          l.observaciones
453      FROM licencias l
454      INNER JOIN agentes a
455          ON l.id_agente = a.id_agente
456      INNER JOIN tipos_licencia tl
457          ON l.id_tipo_licencia = tl.id_tipo_licencia
458      ORDER BY l.id_licencia DESC
459      "";
460
461      try (Connection conexion = ConexionBD.obtenerConexion();
462          PreparedStatement ps = conexion.prepareStatement(sql);
463          ResultSet rs = ps.executeQuery()) {
464
465          while (rs.next()) {
466              licencias.add(new Object[]{
467                  rs.getString(columnLabel: "dni"),
468                  rs.getString(columnLabel: "apellido"),
469                  rs.getString(columnLabel: "nombre"),
470                  rs.getString(columnLabel: "tipo_licencia"),
471                  rs.getDate(columnLabel: "fecha_inicio").toString(),
472                  rs.getDate(columnLabel: "fecha_fin").toString(),
473                  rs.getInt(columnLabel: "dias_solicitados"),
474                  rs.getString(columnLabel: "estado"),
475                  rs.getString(columnLabel: "observaciones")
476              });
477          }
478
479      } catch (SQLException e) {
480          System.err.println("Error al obtener licencias: " + e.getMessage());
481      }
482
483      return licencias;
484  }

```

Imagen 65 – Extracto código RepositorioMemoria.java - Consulta relacional mediante INNER JOIN implementada en RepositorioMemoria para la recuperación de licencias

18.5 - SÍNTESIS DEL MÓDULO

La incorporación de JDBC y MariaDB permitió completar la etapa de persistencia del proyecto, integrando la aplicación Java con una base de datos relacional y reemplazando el almacenamiento temporal utilizado durante las primeras etapas del desarrollo.

La solución implementada organiza el acceso a la información mediante componentes especializados, donde **ConexionBD** administra las conexiones, **RepositorioMemoria** centraliza las operaciones de acceso a datos y las interfaces gráficas interactúan únicamente con el repositorio. Asimismo, la utilización de **PreparedStatement**, **ResultSet**, **ArrayList** y consultas relacionales, evidencia la aplicación de conceptos de programación orientada a objetos y de acceso a bases de datos.

Como resultado, el sistema es capaz de autenticar usuarios, registrar agentes, gestionar solicitudes de licencia y consultar información almacenada de forma permanente en MariaDB, cumpliendo los requerimientos funcionales y técnicos establecidos para el Trabajo Práctico Final y consolidando una arquitectura organizada, mantenible y preparada para futuras ampliaciones.

19 - JUSTIFICACIÓN DEL PATRÓN DE DISEÑO IMPLEMENTADO

Durante el desarrollo del sistema de Gestión Integral para Recursos Humanos surgió la necesidad de organizar el código fuente de manera que cada componente cumpliera una responsabilidad específica.

Con este objetivo, el proyecto fue organizado siguiendo los principios del patrón de diseño **Modelo-Vista-Controlador (MVC)**. Esta distribución de las clases permite separar la lógica del negocio, la interfaz gráfica y el acceso a los datos, favoreciendo un diseño más claro, modular y escalable.

19.1 - ORGANIZACIÓN DEL PROYECTO BASADA EN MVC

La estructura del proyecto se encuentra dividida en paquetes que agrupan clases con responsabilidades bien definidas.

Modelo

El paquete **rrhh.modelo** contiene las clases que representan las entidades principales del dominio del problema.

Entre ellas se destacan:

- Persona
- Agente

Estas clases encapsulan los datos propios de cada entidad mediante atributos privados y proporcionan los métodos necesarios para acceder y modificar dicha información, respetando los principios de encapsulamiento de la Programación Orientada a Objetos.

La clase *Persona* constituye la abstracción principal del dominio, mientras que *Agente* hereda sus características e incorpora los atributos específicos correspondientes al personal del Ministerio.

Vista

El paquete **rrhh.vista** reúne todas las interfaces.

Las principales ventanas del sistema son:

- *LoginView*
- *MenuPrincipalView*
- *AgenteView*
- *LicenciaView*

Estas clases tienen como única responsabilidad interactuar con el usuario, mostrando formularios, tablas y mensajes, además de capturar los datos ingresados desde la interfaz gráfica.

Las vistas no realizan operaciones directas sobre la base de datos, sino que delegan dichas tareas en la capa correspondiente.

Acceso a datos

La comunicación con MariaDB se concentra en el paquete **rrhh.datos**, integrado principalmente por las clases:

- *ConexionBD*
- *RepositorioMemoria*

La clase **ConexionBD** centraliza la configuración de la conexión JDBC y proporciona las conexiones necesarias hacia la base de datos.

Por su parte, **RepositorioMemoria** concentra todas las operaciones de persistencia del sistema, incluyendo la validación de usuarios, el registro y consulta de agentes, la obtención de tipos de licencia y el almacenamiento de solicitudes de licencias mediante consultas SQL parametrizadas.

De esta manera, la lógica de acceso a datos permanece aislada de la interfaz gráfica y del modelo del dominio.

Coordinación de la aplicación

La clase *Main* constituye el punto de inicio de la aplicación.

Su función consiste en verificar inicialmente la disponibilidad de la base de datos mediante el método `ConexionBD.probarConexion()`. Si la conexión resulta satisfactoria, inicia la interfaz gráfica mostrando la ventana de autenticación (`LoginView`).

Posteriormente, cada vista coordina las acciones del usuario invocando los métodos correspondientes del repositorio para realizar las operaciones necesarias sobre la base de datos.

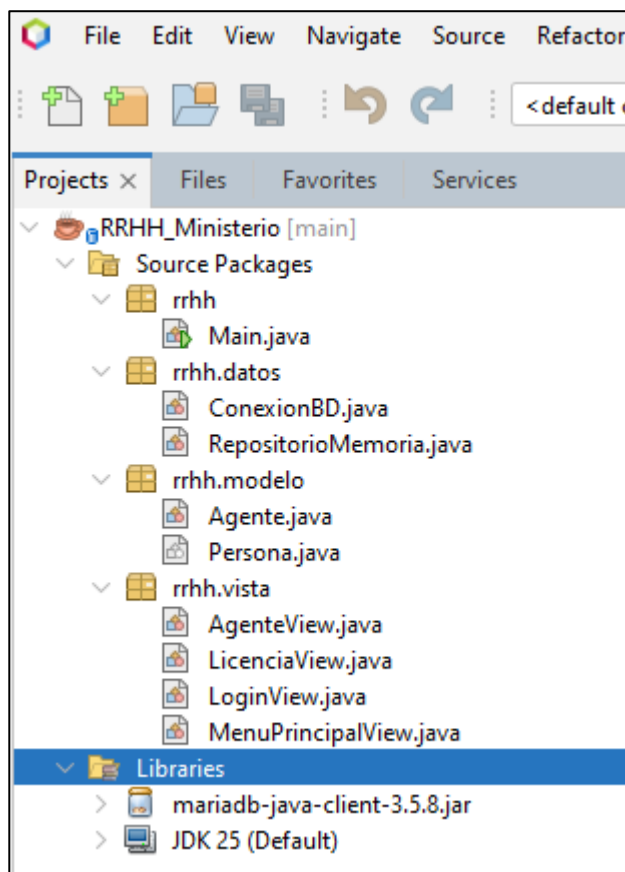
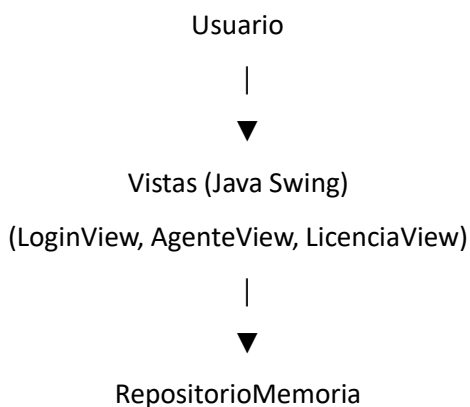
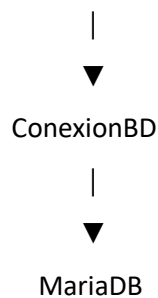


Imagen 66 – Captura de NetBeans - Organización del proyecto siguiendo una arquitectura basada en el patrón MVC

19.2 - RELACIÓN ENTRE LOS COMPONENTES

La interacción entre las distintas capas puede resumirse mediante el siguiente flujo:





Esta organización permite que cada componente conozca únicamente las responsabilidades necesarias para cumplir su función, reduciendo el acoplamiento entre las distintas partes del sistema.

19.3 - BENEFICIOS OBTENIDOS

La utilización de una organización basada en los principios del patrón Modelo-Vista-Controlador permitió obtener diversas ventajas durante el desarrollo del proyecto:

- Separación clara entre la interfaz gráfica, el modelo de datos y la persistencia.
- Mayor organización del código fuente mediante paquetes con responsabilidades específicas.
- Facilidad para incorporar la persistencia mediante JDBC sin modificar significativamente las ventanas desarrolladas durante las etapas anteriores.
- Mejor mantenimiento del proyecto al centralizar todas las operaciones de acceso a la base de datos en una única clase repositorio.
- Posibilidad de ampliar el sistema incorporando nuevos módulos sin alterar la estructura general de la aplicación.

CAPÍTULO 20 - IMPLEMENTACIÓN DE ESTRUCTURAS DE DATOS DINÁMICAS (ARRAYLIST)

En el presente proyecto se implementó la colección **ArrayList**, perteneciente al paquete `java.util`, como mecanismo para almacenar temporalmente los objetos recuperados desde la base de datos y facilitar su procesamiento dentro de la aplicación.

A diferencia de los arreglos tradicionales, cuyo tamaño debe definirse al momento de su creación, `ArrayList` permite administrar una cantidad variable de elementos, adaptándose automáticamente al crecimiento o disminución de los datos manipulados por el sistema.

Esta característica resulta especialmente adecuada para aplicaciones administrativas, donde la cantidad de registros almacenados en la base de datos puede variar constantemente.

20.1 - UTILIZACIÓN DE ARRAYLIST EN EL PROYECTO

La utilización de colecciones dinámicas se concentra principalmente en la clase **RepositorioMemoria**, responsable del acceso a los datos persistidos en MariaDB.

Cuando una consulta SQL devuelve múltiples registros mediante un objeto `ResultSet`, cada fila recuperada es transformada en un objeto del modelo y posteriormente incorporada a una colección dinámica.

De esta forma, la cantidad de elementos almacenados depende exclusivamente de la cantidad de registros existentes en la base de datos, sin necesidad de conocer previamente su tamaño.

Remitir a Imagen 64 para ver implementación de estructuras dinámicas mediante `ArrayList`

20.2 - RECUPERACIÓN DINÁMICA DE AGENTES

Los métodos:

- `obtenerAgentes()`
- `obtenerAgentesOrdenadosPorApellido()`

ejecutan consultas SQL sobre la tabla `agentes` y almacenan los resultados en una colección **`ArrayList<Agente>`**.

Una vez finalizada la consulta, la colección completa es devuelta a las ventanas del sistema, particularmente a **`AgenteView`**, donde se utiliza para cargar dinámicamente la tabla que presenta la información al usuario.

Esta estrategia desacopla la lógica de acceso a datos de la representación visual, facilitando el mantenimiento y la reutilización del código.

20.3 - RECUPERACIÓN DINÁMICA DE TIPOS DE LICENCIA

El método:

`obtenerTiposLicencia()`

también emplea una colección dinámica.

Cada nombre recuperado desde la tabla `tipos_licencia` es incorporado a la colección mediante el método **`add()`**.

Posteriormente, la vista **LicenciaView** utiliza esta lista para completar automáticamente el componente gráfico JComboBox, evitando mantener valores fijos escritos directamente en el código fuente.

Esta implementación permite que cualquier modificación realizada sobre los tipos de licencia en la base de datos se refleje automáticamente en la interfaz del sistema.

20.4 - RECUPERACIÓN DINÁMICA DE LICENCIAS

El listado histórico de licencias registradas también utiliza colecciones dinámicas.

En este caso, el método:

obtenerLicenciasParaTabla()

Cada elemento de la colección representa una fila completa destinada a ser visualizada posteriormente en la tabla de la interfaz gráfica.

La información se obtiene mediante una consulta SQL que utiliza operaciones INNER JOIN entre las tablas:

- licencias
- agentes
- tipos_licencia

permitiendo construir una vista integrada de la información necesaria para el usuario.

Posteriormente, la colección es recorrida desde LicenciaView, donde cada elemento es agregado al modelo de la JTable.

20.5 - CONCLUSIÓN

La utilización de la colección dinámica ArrayList permitió implementar un mecanismo flexible para almacenar temporalmente los datos recuperados desde MariaDB mediante JDBC. Su empleo en los módulos de agentes, tipos de licencia y registro de licencias demuestra la aplicación práctica de las estructuras de datos dinámicas estudiadas durante la asignatura, facilitando el procesamiento de información variable y contribuyendo a una arquitectura organizada y mantenible.

CAPÍTULO 21 - APLICACIÓN DE POO: ABSTRACCIÓN, HERENCIA Y POLIMORFISMO

Durante el desarrollo del proyecto se implementaron los pilares de abstracción, herencia y polimorfismo mediante las clases que representan a las personas y agentes pertenecientes a la Oficina de Personal. Esta estructura permitió evitar la duplicación de código, mejorar la organización del modelo de datos y facilitar la futura incorporación de nuevas entidades relacionadas con la gestión de recursos humanos.

21.1 - ABSTRACCIÓN MEDIANTE LA CLASE PERSONA

La abstracción se implementó mediante la definición de la clase:

public abstract class Persona

Esta clase representa una entidad genérica del dominio y concentra la información común que comparten todas las personas administradas por el sistema.

Los atributos definidos son:

- dni
- cuil
- apellido
- nombre

Estos datos constituyen la información básica de identificación de cualquier persona registrada en la aplicación.

Al declararse como una clase **abstracta**, Persona no puede ser instanciada directamente mediante la instrucción **new Persona()**. Su función consiste en servir como base para otras clases más específicas, obligando a que únicamente puedan crearse objetos pertenecientes a entidades concretas del dominio.

Remitir a Imagen 38

21.2 - HERENCIA MEDIANTE LA CLASE AGENTE

La herencia se implementó utilizando la clase:

public class Agente extends Persona

Mediante la palabra reservada **extends**, la clase Agente hereda automáticamente todos los atributos y métodos definidos en Persona, reutilizando el comportamiento común sin necesidad de volver a implementarlo.

Además de los atributos heredados, la clase incorpora información específica correspondiente al ámbito laboral del agente estatal:

- cargo
- reparticion
- activo

De esta forma, un objeto Agente reúne tanto la información personal heredada como los datos propios necesarios para la administración del personal dentro del Ministerio.

Remitir a Imagen 40

21.3 - POLIMORFISMO MEDIANTE LA SOBRESCRITURA DE MÉTODOS

El proyecto también aplica el principio de polimorfismo utilizando métodos abstractos y su implementación en las clases derivadas.

La clase Persona declara el siguiente método abstracto:

public abstract String mostrarDatos();

Al tratarse de un método abstracto, no posee implementación dentro de la clase base y obliga a todas las clases que hereden de Persona a proporcionar su propia implementación.

La clase Agente implementa este comportamiento mediante la anotación:

@Override

public String mostrarDatos()

Este método construye y devuelve una representación textual del agente utilizando tanto los atributos heredados (dni, cuil, apellido, nombre) como los atributos propios (cargo, reparticion, activo).

De esta manera, el sistema garantiza que cualquier clase derivada de Persona pueda presentar su información respetando una misma interfaz, pero adaptando su comportamiento según las necesidades particulares de cada entidad.

21.4 - ENCAPSULAMIENTO DE LA INFORMACIÓN

Todos los atributos de las clases del modelo fueron declarados con acceso **private**, impidiendo que otras clases modifiquen directamente su contenido.

El acceso a la información se realiza exclusivamente mediante los correspondientes métodos públicos:

- getDni()
- setDni()

- `getApellido()`
- `setApellido()`
- `getCargo()`
- `setCargo()`
- `isActivo()`
- `setActivo()`

Este mecanismo permite controlar la manipulación de los datos internos de cada objeto, preservando la integridad del modelo y favoreciendo un diseño orientado a objetos más seguro y mantenible.

21.5 - BENEFICIOS OBTENIDOS

La utilización de los principios de POO permitió obtener diversas ventajas durante el desarrollo del sistema:

- Reutilización de código mediante herencia.
- Eliminación de duplicación de atributos comunes.
- Organización clara del modelo del dominio.
- Separación entre información general y específica de cada entidad.
- Mayor facilidad para extender el sistema incorporando nuevas clases derivadas.
- Mejor mantenimiento del código fuente.

Asimismo, la utilización de clases abstractas permitió definir una estructura común para todas las entidades relacionadas con personas, garantizando un diseño consistente y alineado con las buenas prácticas de la Programación Orientada a Objetos.

CAPÍTULO 22 - MANEJO DE EXCEPCIONES Y VALIDACIONES DE ROBUSTEZ

Durante el desarrollo del sistema se incorporaron mecanismos de control destinados a prevenir errores de ejecución, proteger la integridad de la información y brindar una respuesta adecuada al usuario ante situaciones excepcionales.

La estrategia adoptada combina el tratamiento estructurado de excepciones mediante bloques try-catch con un conjunto de validaciones preventivas realizadas antes de ejecutar las operaciones sobre la base de datos. Este enfoque permite detectar condiciones inválidas de forma anticipada y evitar que

errores de entrada o problemas de conectividad provoquen la interrupción inesperada de la aplicación.

22.1 - MANEJO DE EXCEPCIONES EN LA CAPA DE PERSISTENCIA

La comunicación entre la aplicación Java y la base de datos MariaDB se realiza mediante la API JDBC, por lo que todas las operaciones de acceso a datos pueden generar excepciones del tipo `SQLException`. Para administrar estas situaciones, los métodos de las clases **ConexionBD** y **RepositorioMemoria** implementan bloques try-catch que encapsulan las operaciones críticas relacionadas con la conexión, consulta y actualización de la base de datos.

Entre los métodos que realizan este tratamiento se encuentran:

- `probarConexion()`
- `validarUsuario()`
- `agregarAgente()`
- `obtenerAgentes()`
- `obtenerAgentesOrdenadosPorApellido()`
- `buscarAgentePorDni()`
- `existeDni()`
- `obtenerTiposLicencia()`
- `registrarLicencia()`
- `obtenerLicenciasParaTabla()`

Cuando se produce una excepción durante la ejecución de una consulta SQL, el sistema captura el error y evita que la aplicación finalice de manera abrupta.

Esta estrategia permite preservar la estabilidad del sistema aun cuando existan inconvenientes de conectividad con MariaDB o errores durante la ejecución de las consultas.

22.2 - VALIDACIÓN DEL FORMATO DE FECHAS

El módulo de gestión de licencias requiere el ingreso manual de las fechas de inicio y finalización utilizando el formato `dd/MM/yyyy`.

Cuando se detecta un formato inválido, el sistema interrumpe el proceso de registración y presenta un mensaje informativo, solicitando al usuario que ingrese nuevamente la fecha respetando el formato establecido.

De esta manera se evita que datos inconsistentes lleguen a la capa de persistencia.

```
294         try {
295             fechaInicio = LocalDate.parse(fechaInicioTexto, formatter);
296             fechaFin = LocalDate.parse(fechaFinTexto, formatter);
297         } catch (DateTimeParseException ex) {
298             JOptionPane.showMessageDialog(
299                 this,
300                 message: "Las fechas deben tener el formato dd/MM/yyyy.",
301                 title: "Formato de fecha inválido",
302                 JOptionPane.WARNING_MESSAGE
303             );
304             return;
305         }
```

Imagen 67 – Extracto código de LicenciaView.java - Manejo de excepciones mediante try-catch para la validación de fechas en el módulo de licencias

22.3 - VALIDACIONES PREVENTIVAS DE INGRESO DE DATOS

Además del tratamiento de excepciones, la aplicación implementa diversas validaciones antes de ejecutar cualquier operación sobre la base de datos.

Estas verificaciones permiten detectar errores de ingreso y mejorar la calidad de la información almacenada.

Validación de campos obligatorios

Antes de registrar un agente o una licencia, el sistema verifica que los campos obligatorios hayan sido completados.

Si algún dato requerido no fue ingresado, la operación se cancela y se informa al usuario mediante un cuadro de diálogo.

Validación del formato del DNI

Tanto en **AgenteView** como en **LicenciaView**, el número de documento se valida.

Esta condición verifica que el DNI esté compuesto exactamente por ocho caracteres numéricos.

Si la validación falla, el sistema impide continuar con la operación hasta que el dato sea corregido.

Validación de unicidad del DNI

Antes de registrar un nuevo agente, la aplicación consulta la base de datos; Si el documento ya se encuentra registrado, la operación se cancela y el usuario recibe una notificación indicando que el agente ya existe.

Esta validación evita la incorporación de registros duplicados desde la propia lógica de la aplicación.

Validación de existencia del agente

Antes de registrar una licencia, el sistema verifica que el agente exista previamente; Si la búsqueda no devuelve resultados, la registración de la licencia es cancelada y el usuario recibe un mensaje indicando que no existe un agente asociado al DNI ingresado.

De esta manera se evita registrar licencias para agentes inexistentes.

Validación de la secuencia cronológica

Luego de convertir correctamente las fechas ingresadas, el sistema verifica que la fecha de finalización no sea anterior a la fecha de inicio.

Si la condición NO se cumple, la registración es cancelada y se informa al usuario sobre el error detectado.

Esta validación garantiza la coherencia temporal de las licencias registradas.

22.4 - COMUNICACIÓN DE ERRORES AL USUARIO

Cuando se detecta una condición inválida, se informa la situación al operador.

Según el tipo de evento, se emplean diferentes categorías de mensajes:

- Información.
- Advertencias.
- Errores.

Esta estrategia mejora la interacción con el usuario al proporcionar mensajes claros y específicos sobre la causa del problema detectado.

22.5 - BENEFICIOS OBTENIDOS

La incorporación de mecanismos de validación y manejo de excepciones permitió mejorar significativamente la confiabilidad del sistema.

Entre los principales beneficios obtenidos pueden destacarse:

- Prevención de registros inconsistentes.
- Protección frente a errores de conexión con MariaDB.
- Validación de la información antes de persistirla.
- Disminución de fallos durante la ejecución.
- Mejor experiencia de uso mediante mensajes informativos.

- Mayor estabilidad de la aplicación ante situaciones excepcionales.

CAPÍTULO 23 - JUSTIFICACIÓN DE LA NO UTILIZACIÓN DE ARCHIVOS PLANOS

Uno de los contenidos abordados durante la asignatura corresponde al manejo de archivos mediante las clases proporcionadas por Java para la lectura y escritura de información en el sistema de archivos. Sin embargo, el presente proyecto implementa un mecanismo de persistencia basado íntegramente en una base de datos relacional MariaDB, motivo por el cual no fue necesario incorporar almacenamiento adicional mediante archivos planos.

Esta decisión responde tanto a criterios técnicos como al alcance funcional definido para el sistema desarrollado.

23.1 - PERSISTENCIA IMPLEMENTADA EN EL PROYECTO

La información administrada por el sistema se almacena de manera permanente en la base de datos **MariaDB**, utilizando la API **JDBC** como mecanismo de comunicación entre la aplicación Java y el motor relacional.

Los datos correspondientes a usuarios, agentes, tipos de licencia y licencias se encuentran distribuidos en tablas relacionadas entre sí, permitiendo que toda la información del sistema permanezca centralizada en un único repositorio de datos.

Las operaciones de consulta, inserción y validación son realizadas mediante sentencias SQL ejecutadas desde la clase **RepositorioMemoria**, mientras que la clase **ConexionBD** administra la apertura de las conexiones necesarias para acceder al servidor de base de datos.

En consecuencia, el proyecto ya dispone de un mecanismo completo de persistencia permanente, por lo que incorporar archivos planos para almacenar la misma información habría significado duplicar innecesariamente la funcionalidad implementada.

23.2 - RAZONES PARA NO UTILIZAR ARCHIVOS PLANOS

La decisión de no incorporar almacenamiento mediante archivos responde a los siguientes fundamentos técnicos:

- La persistencia permanente ya se encuentra completamente resuelta mediante MariaDB y JDBC.
- Toda la información del sistema se administra de forma centralizada dentro de una única base de datos.

- La estructura relacional facilita la vinculación entre entidades mediante claves primarias y foráneas.
- El lenguaje SQL permite realizar consultas, búsquedas y relaciones entre tablas de forma eficiente.
- La utilización simultánea de archivos planos hubiera implicado mantener información duplicada, incrementando la complejidad del sistema y el riesgo de inconsistencias entre ambos mecanismos de almacenamiento.
- El uso exclusivo de la base de datos simplifica el mantenimiento y la administración de la información.

23.3 - CONCLUSIÓN

El proyecto implementa un mecanismo de persistencia permanente utilizando MariaDB y JDBC, permitiendo administrar de forma centralizada la información correspondiente a usuarios, agentes y licencias. Debido a que la base de datos satisface completamente los requerimientos de almacenamiento definidos para el sistema, no fue necesario incorporar archivos planos como mecanismo adicional de persistencia. Esta decisión permitió mantener una arquitectura más simple, consistente y acorde con el alcance funcional del proyecto, sin dejar de contemplar los conceptos teóricos relacionados con el manejo de archivos estudiados durante la asignatura.

CAPÍTULO 24 - CONCLUSIONES

El desarrollo del **Sistema de Gestión Integral para Recursos Humanos del Ministerio de Gobierno, Infraestructura y Desarrollo Territorial del Gobierno de Mendoza** permitió aplicar de manera integrada los conocimientos adquiridos durante las distintas etapas de la carrera, abarcando actividades de análisis, diseño, implementación, pruebas y persistencia de datos.

A partir del relevamiento realizado sobre los procesos administrativos de la Oficina de Personal, se identificaron diversas problemáticas asociadas al uso simultáneo de documentación física, planillas de cálculo y sistemas aislados, generando dificultades para la gestión, consulta y resguardo de la información administrativa de los agentes. Como respuesta a esta situación, se diseñó y desarrolló una propuesta orientada a centralizar la información, mejorar la trazabilidad de los procesos y facilitar el acceso a los datos administrativos.

Durante el desarrollo del proyecto se aplicaron metodologías de análisis y diseño de sistemas, permitiendo elaborar el modelo de dominio, los requerimientos funcionales y no funcionales, los casos de uso, los diagramas UML, el modelo entidad-relación y la arquitectura general del sistema. Estas etapas proporcionaron una base sólida para la construcción del prototipo funcional.

La implementación fue realizada utilizando el lenguaje Java, mediante una aplicación de escritorio desarrollada con Java Swing, aplicando conceptos fundamentales de Programación Orientada a Objetos, tales como encapsulamiento, abstracción, herencia y polimorfismo. Asimismo, se incorporaron mecanismos de validación, manejo de excepciones y estructuras dinámicas de almacenamiento mediante colecciones ArrayList, cumpliendo con los requisitos técnicos establecidos para el desarrollo del Trabajo Práctico Final.

En la etapa final del proyecto se implementó la persistencia permanente de datos utilizando la API JDBC y el sistema gestor de bases de datos MariaDB, permitiendo reemplazar el almacenamiento temporal utilizado durante las etapas iniciales. La incorporación de una arquitectura organizada, basada en la separación de responsabilidades entre las capas de presentación, modelo y acceso a datos, contribuyó a mejorar la mantenibilidad, reutilización y escalabilidad de la aplicación.

Las pruebas realizadas permitieron verificar el correcto funcionamiento de las funcionalidades implementadas, validando tanto los procesos de autenticación, gestión de agentes y registro de licencias, como los mecanismos de validación y persistencia de la información. Los resultados obtenidos demuestran que el prototipo desarrollado satisface los objetivos definidos para esta instancia académica y constituye una base sólida para futuras ampliaciones funcionales.

Finalmente, el desarrollo de este proyecto permitió integrar conocimientos de análisis de sistemas, modelado UML, bases de datos, programación orientada a objetos, persistencia de datos y pruebas de software, consolidando una experiencia práctica de desarrollo de sistemas de información aplicados a problemáticas reales de la administración pública. Asimismo, el sistema propuesto presenta una arquitectura que posibilita su evolución futura mediante la incorporación de nuevos módulos, funcionalidades y mecanismos de integración con otros sistemas institucionales.